



ROYAUME DU MAROC
MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE



Université Abdelmalek Essaâdi Faculté des Sciences et Techniques – Tanger Master
Sciences et Techniques Intelligence Artificielle et Sciences des Données

Rapport de Projet de Fin d'Année

Master Sciences et Techniques

Conception et développement d'une plateforme intelligente d'analyse conversationnelle des données de santé

*Architecture agentique, moteur d'analyse déterministe
et modèles de langage locaux*

Organisme d'accueil :



المجموعة الصحية الترابية
طنجة-تطوان-الحسيمة
+XO:CC. +A:O. +C:O. +
E. I. - +E. U. - H:O. C.

Groupement Sanitaire Territorial
Tanger-Tétouan-Al Hoceïma

Réalisé par :

ZARQI Ezzoubair
El Attar Mohamed
El Gharib Mahmoud
Etudiants en Master Intelligence Artificielle
et Sciences des Données

Encadré par :

Pr. Benabdoulah Ikram
Encadrant pédagogique
FANGACHI Oussama
Encadrant technique – GST-TTA

Structure d'accueil : Direction de la Formation, de la Recherche et de l'Innovation

Année Universitaire 2025–2026

Avant-Propos

Nom et Prénom de l'Étudiant Stagiaire :

El Gharib Mahmoud, ZARQI Ezzoubair, El Attar Mohamed

Intitulé du sujet :

Conception et Développement d'une Plateforme Intelligente d'Analyse
Conversationnelle des données de santé

Établissement d'accueil :

Groupe Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)
Direction de la Formation, de la Recherche et de l'Innovation (DFRI)

Encadrant Pédagogique :

Pr. Benabdoulah Ikram

Encadrant Technique de Stage :

FANGACHI Oussama

Période de projet :

Du 20 juin 2026 au 01 septembre 2026

Cadre du Stage :

Projet de Fin d'Année présenté en vue de l'obtention du diplôme de Master Sciences et
Techniques

Filière :

Master Sciences et Techniques – Intelligence Artificielle et Sciences des Données
(IASD)

Cet avant-propos présente les informations essentielles relatives au projet de fin d'année, incluant l'identité des stagiaires, l'intitulé du sujet, l'organisme d'accueil, les encadrants ainsi que le cadre académique dans lequel s'inscrit ce travail.

Dédicace

*À nos chers parents,
qui nous ont soutenus durant la réalisation de ce projet
et pour tous les sacrifices qu'ils ont consentis
afin de nous faciliter la tâche,
en témoignage de la profonde affection que nous leur portons.*

*À nos frères et sœurs,
à qui nous souhaitons un avenir prospère.*

*À nos chers amis et amies,
merci pour leur présence et leurs encouragements ;
qu'ils trouvent ici toute notre gratitude.*

*À nos enseignants,
qui n'ont cessé de nous conseiller, encourager et soutenir
tout au long de nos études.*

*À tous ceux qui ont contribué,
de près ou de loin,
à la réalisation de ce projet.*

Merci

Remerciements

Avant tout développement sur cette expérience professionnelle, il apparaît opportun de commencer ce rapport de stage par des remerciements.

Nous tenons à remercier dans un premier temps, toute l'équipe pédagogique de la Faculté des Sciences et Techniques et les intervenants professionnels responsables de la filière Master Sciences et Techniques – Intelligence Artificielle et Sciences des Données (IASD), qui ont contribué à nous donner une formation solide tout au long de nos années d'études.

Nous remercions tout particulièrement notre encadrante académique **Pr. Benabdoulah Ikram**, pour son encadrement, tous ses conseils, le soutien dont elle nous a fait bénéficier, ses encouragements, les aides précieuses qu'elle n'a cessé de nous apporter tout au long de la période de ce projet et pour ses efforts afin d'assurer le bon déroulement de ce stage.

Nous tenons également à remercier et à témoigner toute notre reconnaissance à **FANGACHI Oussama**, notre encadrant technique, pour son accueil et la confiance qu'il nous a accordée dès notre arrivée au sein du Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma ; pour le temps qu'il nous a consacré tout au long de cette période, sachant répondre à toutes nos interrogations ; sans oublier sa participation au cheminement de ce rapport.

Enfin, nous remercions toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail, ainsi qu'au bon déroulement du stage, et dont les noms ne figurent pas dans ce document.

Résumé

Dans le cadre de la transformation numérique du **Groupe Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)**, ce projet porte sur la conception et le développement d'une **plateforme intelligente d'analyse conversationnelle de données de santé**. Son objectif est de faciliter l'exploitation des données hospitalières en permettant aux professionnels de santé et aux décideurs de formuler leurs demandes en langage naturel et d'obtenir des analyses statistiques, des visualisations et des interprétations adaptées à leurs besoins, sans nécessiter une maîtrise approfondie des outils traditionnels d'analyse de données.

La solution proposée repose sur une architecture modulaire combinant une interface conversationnelle, un orchestrateur agentique basé sur **LangGraph**, un moteur d'analyse déterministe et une infrastructure de modèles de langage locaux. Cette architecture établit une séparation entre l'intelligence chargée de comprendre et de planifier les demandes et les composants responsables de l'exécution effective des traitements. Le moteur d'analyse s'appuie notamment sur **pandas** et **DuckDB** afin de produire des résultats calculés de manière reproductible. Un mécanisme d'*evidence* permet en outre de relier les réponses générées aux résultats effectivement obtenus lors de l'analyse, renforçant ainsi leur traçabilité et leur vérifiabilité.

La plateforme a été conçue avec une attention particulière portée à la **confidentialité et à la souveraineté des données**. L'utilisation de modèles de langage locaux permet de limiter l'exposition des données sensibles à des services externes, tandis que l'architecture modulaire facilite l'évolution et l'intégration de nouvelles capacités d'analyse.

La réalisation a suivi une démarche itérative fondée sur une approche Agile, depuis l'analyse des besoins jusqu'à la conception, l'implémentation et la validation de la plateforme. Les tests réalisés ont permis de vérifier le fonctionnement des principaux composants et leur adéquation avec les besoins identifiés. Certaines limites restent toutefois à traiter, notamment l'authentification et la gestion fine des droits d'accès, l'industrialisation des tests et l'extension des capacités analytiques. Ces éléments constituent des perspectives naturelles pour les évolutions futures de la solution.

Mots-clés : Intelligence artificielle générative, analyse conversationnelle, données de santé, grands modèles de langage, agents intelligents, LangGraph, analyse de données, souveraineté des données.

Abstract

As part of the digital transformation of the **Tanger–Tétouan–Al Hoceïma Territorial Health Group (GST-TTA)**, this project focuses on the design and development of an **intelligent conversational health data analysis platform**. Its objective is to facilitate the exploitation of hospital data by enabling healthcare professionals and decision-makers to formulate requests in natural language and obtain statistical analyses, visualizations, and interpretations adapted to their needs, without requiring advanced expertise in traditional data analysis tools.

The proposed solution is based on a modular architecture combining a conversational interface, an agentic orchestrator built with **LangGraph**, a deterministic analysis engine, and a local language-model infrastructure. This architecture separates the intelligence responsible for understanding and planning user requests from the components responsible for executing the actual computations. The analysis engine relies in particular on **pandas** and **DuckDB** to produce reproducible results. An *evidence* mechanism further links generated responses to the results effectively obtained during the analysis, thereby improving their traceability and verifiability.

Particular attention was given to **data confidentiality and sovereignty**. The use of local language models limits the exposure of sensitive data to external services, while the modular architecture facilitates future evolution and the integration of additional analytical capabilities.

The project was developed through an iterative Agile approach, covering requirements analysis, system design, implementation, and platform validation. The tests conducted verified the operation of the main components and their compliance with the identified requirements. Some limitations remain to be addressed, particularly authentication and fine-grained access control, test industrialization, and the extension of analytical capabilities. These aspects constitute natural directions for future development.

Keywords : Generative artificial intelligence, conversational analytics, health data, large language models, intelligent agents, LangGraph, data analysis, data sovereignty.

الملخص

في إطار التحول الرقمي الذي يشهده التجمع الصحي الترابي طنجة--تطوان--الحسيمة (GST-TTA)، يهدف هذا المشروع إلى تصميم وتطوير منصة ذكية لتحليل الحوارات للبيانات الصحية. تتمثل الغاية الأساسية في تسهيل استغلال البيانات الاستشفائية، من خلال تمكين مهني الصحة وصناع القرار من التفاعل مع مجموعات البيانات باستخدام اللغة الطبيعية، والحصول على تحليلات إحصائية موثوقة وتمثيلات بيانية ملائمة، مع ضمان سرية البيانات وسيادتها. تعتمد المنصة على بنية معيارية تجمع بين واجهة مستخدم مبنية باستخدام SvelteKit، وخلفية تعتمد على FastAPI، ومنسق للوكلاء الذكيين مبني على LangGraph، بالإضافة إلى محرك حتمي لتحليل البيانات يعتمد بشكل أساسي على pandas و DuckDB. وتفصل هذه البنية بين دور نموذج اللغة ودور المحرك التحليلي؛ إذ يتولى النموذج فهم طلب المستخدم وتنسيق مراحل المعالجة، بينما ينفذ محرك التحليل العمليات الحسابية بطريقة حتمية وقابلة للتحقق. كما يتيح نظام evidence ربط الإجابات المقدمة بالنتائج المحسوبة فعلياً. ويساهم اعتماد نماذج لغوية محلية في تعزيز سيادة البيانات والحد من الاعتماد على الخدمات الخارجية. تم تطوير المنصة بطريقة تكرارية، ثم التحقق من وظائفها من خلال مجموعة من السيناريوهات العملية. وتبين النتائج إمكانية الجمع بين الذكاء الاصطناعي التوليدي وتحليل البيانات والتفاعل الحوارية في بيئة صحية. وتشمل آفاق التطوير المستقبلية تعزيز آليات المصادقة والتحكم في الوصول، وإضفاء الطابع الصناعي على الاختبارات، وتوسيع القدرات التحليلية، وتحسين التقييم المستمر لجودة الإجابات المولدة.

الكلمات المفتاحية: الذكاء الاصطناعي التوليدي، التحليل الحوارية، البيانات الصحية، النماذج اللغوية الكبيرة، الوكلاء الذكيون، LangGraph تحليل البيانات، سيادة البيانات.

Table des matières

Avant-Propos	i
Dédicace	ii
Remerciements	iii
Résumé	iv
Abstract	v
الملخص	vi
Liste des abréviations	xii
Table des figures	xiii
Liste des tableaux	xvi
1 Introduction générale	1
1.1 Organisation du rapport	2
2 Présentation et Contexte général	3
2.1 Introduction	3
2.2 Présentation de l'organisme d'accueil	3
2.2.1 Présentation du GST-TTA	3
2.2.2 Fiche signalétique	4
2.2.3 Missions	5
2.2.4 Organisation	5
2.2.5 Direction de la Formation, de la Recherche et de l'Innovation ..	6
2.3 Contexte général et analyse de l'existant	7
2.3.1 Transformation numérique au GST-TTA	7
2.3.2 Données et systèmes d'information hospitaliers	8
2.3.3 Processus actuel et difficultés rencontrées	8
2.3.4 Limites des solutions existantes	9
2.4 Problématique	10
2.5 Objectifs du projet	11
2.5.1 Objectif général	11
2.5.2 Objectifs spécifiques	11

TABLE DES MATIÈRES

2.6	Méthodologie adoptée	12
2.6.1	Approche Agile adaptée	13
2.6.2	Organisation en sprints	14
2.6.3	Phases de réalisation	14
2.7	Management du projet	15
2.7.1	Équipe du projet	15
2.7.2	Diagramme de Gantt	16
2.8	Conclusion	17
3	Analyse des besoins et étude de l'existant	19
3.1	Introduction	19
3.2	Analyse des besoins	19
3.2.1	Identification des acteurs	19
3.2.2	Cas d'utilisation	20
3.2.3	Besoins fonctionnels	20
3.2.4	Besoins non fonctionnels	21
3.3	État de l'art	22
3.3.1	Intelligence artificielle générative	22
3.3.2	Grands modèles de langage (LLMs)	22
3.3.3	Agents intelligents	23
3.3.4	Model Context Protocol (MCP)	23
3.3.5	Framework LangGraph	23
3.3.6	Retrieval-Augmented Generation (RAG)	24
3.3.7	Analyse conversationnelle de données	24
3.4	Étude comparative des solutions existantes	24
3.5	Positionnement de la solution proposée	25
3.6	Conclusion	26
4	Conception et architecture de la solution	27
4.1	Introduction	27
4.2	Conception générale de la solution	27
4.2.1	Choix technologiques	27
4.2.2	Architecture globale	30
4.2.3	Principes de conception	32
4.3	Modélisation du système	33
4.3.1	Modélisation fonctionnelle	33
4.3.2	Modélisation UML	34
4.4	Architecture logicielle	37
4.4.1	Couche présentation	38
4.4.2	Couche API et services	38
4.4.3	Couche orchestration	39

4.4.4	Couche moteur d'analyse	39
4.4.5	Couche persistance	39
4.4.6	Couche modèles IA	39
4.5	Architecture agentique et moteur d'analyse	40
4.5.1	Architecture des agents	40
4.5.2	Orchestration avec LangGraph	41
4.5.3	Gestion du contexte et harness LLM	42
4.5.4	Moteur d'analyse déterministe	46
4.6	Communication et gestion des données	47
4.6.1	Échanges entre composants	47
4.6.2	Streaming et événements	47
4.6.3	Persistance et gestion des artefacts	48
4.7	Conclusion	48
5	Réalisation de la plateforme	49
5.1	Introduction	49
5.2	L'intelligence des données appliquée au projet	49
5.3	Environnement et organisation de la réalisation	50
5.3.1	Environnement technique	51
5.3.2	Environnement et dépendances	51
5.3.3	Organisation du travail	51
5.3.4	Gestion de la configuration	51
5.4	Réalisation du backend et du streaming	52
5.4.1	L'API FastAPI	52
5.4.2	Les services applicatifs	53
5.4.3	La gestion des conversations	53
5.4.4	La chaîne d'ingestion et de profilage des données	54
5.4.5	Le streaming temps réel	54
5.4.6	Sécurité et gestion des erreurs	55
5.5	Réalisation de l'orchestration agentique et de l'evidence	55
5.5.1	L'orchestration avec LangGraph	55
5.5.2	Le workflow d'analyse	55
5.5.3	L'ingénierie des prompts de l'agent	56
5.5.4	Les outils de l'agent analyste de données	58
5.5.5	Implémentation du moteur d'analyse	59
5.5.6	Le cadre d'exécution (<i>harness</i>) en pratique	60
5.5.7	La traçabilité par l'evidence	61
5.6	Réalisation du frontend et de l'internationalisation	62
5.6.1	L'interface SvelteKit	62
5.6.2	L'interface conversationnelle	62

TABLE DES MATIÈRES

5.6.3	La gestion des sessions et des conversations	62
5.6.4	La visualisation des résultats	62
5.6.5	L'internationalisation (i18n)	63
5.7	Déploiement et intégration	63
5.7.1	L'intégration des composants	63
5.7.2	Le déploiement conteneurisé	64
5.7.3	L'infrastructure de modèles de langage locaux	64
5.8	Conclusion	64
6	Tests et validation	66
6.1	Introduction	66
6.2	Stratégie et environnement de test	66
6.3	Tests unitaires des composants	67
6.3.1	Exécution des outils et tolérance aux pannes	67
6.3.2	Tests de bornes et robustesse	67
6.4	Tests d'intégration et validation fonctionnelle	68
6.4.1	Validation de l'API et du streaming	68
6.4.2	Validation de la chaîne d'ingestion	68
6.4.3	Validation du workflow d'analyse	68
6.4.4	Validation utilisateur	68
6.5	Tests de la passerelle de modèles locaux	69
6.5.1	Suite de tests de la passerelle	69
6.5.2	Intégration continue	69
6.6	Sécurité, performance et qualité	69
6.6.1	Sécurité	69
6.6.2	Performance	70
6.6.3	Qualité des réponses	70
6.7	Résultats, limites et perspectives	70
6.7.1	Résultats et synthèse	70
6.7.2	Limites de la solution	71
6.7.3	Perspectives d'amélioration	71
6.8	Conclusion	72
	Conclusion générale	73
A	Annexes	76
A.1	Captures d'écran de la plateforme	76
A.1.1	Tableau de bord	76
A.1.2	Gestion des jeux de données	77
A.1.3	Espace conversationnel	80
A.1.4	Artefacts	85

A.1.5	Paramètres et internationalisation	86
A.1.6	Passerelle de modèles locaux	87
A.2	Structure du projet	87
A.3	Projets liés.	88

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface (interface de programmation applicative)
CI/CD	Continuous Integration / Continuous Deployment (intégration et déploiement continus)
CORS	Cross-Origin Resource Sharing (partage de ressources entre origines)
CRUD	Create, Read, Update, Delete (créer, lire, modifier, supprimer)
CSV	Comma-Separated Values (valeurs séparées par des virgules)
DB	Database (base de données)
DFRI	Direction de la Formation, de la Recherche et de l'Innovation
GGUF	GPT-Generated Unified Format (format de fichiers pour modèles de langage)
GST-TTA	Groupeement Sanitaire Territorial Tanger–Tétouan–Al Hoceïma
HTTP	HyperText Transfer Protocol (protocole de transfert hypertexte)
IA	Intelligence Artificielle
IQR	Interquartile Range (écart interquartile)
i18n	Internationalisation
JSON	JavaScript Object Notation (notation d'objet JavaScript)
JWT	JSON Web Token (jeton d'authentification)
LLM	Large Language Model (grand modèle de langage)
MCP	Model Context Protocol (protocole de contexte pour les modèles)
NLP	Natural Language Processing (traitement automatique du langage naturel)
OpenAPI	Open Application Programming Interface (spécification de description d'API)
ORM	Object-Relational Mapping (mapping objet-relationnel)
PFA	Projet de Fin d'Année
RAG	Retrieval-Augmented Generation (génération augmentée par récupération)
REST	Representational State Transfer (style architectural pour les services Web)
SQL	Structured Query Language (langage de requête structuré)
SSE	Server-Sent Events (événements envoyés par le serveur)
UML	Unified Modeling Language (langage de modélisation unifié)
UUID	Universally Unique Identifier (identifiant unique universel)
XLSX	Office Open XML Spreadsheet (format de tableur Excel)
YAML	YAML Ain't Markup Language (format de sérialisation de données)

Table des figures

2.1	Présentation du Groupement Sanitaire Territorial Tanger–Tétouan–Al Hoceïma	4
2.2	Organigramme du Groupement Sanitaire Territorial Tanger–Tétouan–Al Hoceïma	6
2.3	Démarche générale adoptée pour la réalisation du projet	13
2.4	Structure des sprints : de la vision globale à la spécialisation santé . . .	13
2.5	Diagramme de Gantt du projet	17
3.1	Acteurs principaux de la plateforme	20
3.2	Diagramme des cas d'utilisation principaux	20
4.1	Architecture globale : de la requête en langage naturel à la restitution des résultats	31
4.2	Diagramme de séquence du traitement d'une requête conversationnelle	34
4.3	Diagramme d'activité du processus d'analyse	36
4.4	Diagramme de classes simplifié du modèle de données	37
4.5	Architecture logicielle en couches et relations entre composants	38
4.6	Graphes d'états de l'orchestrateur (planification, exécution, synthèse) .	41
4.7	Cadre d'exécution (<i>harness</i>) de l'agent et supervision du cycle de vie (LLM Ops)	42
A.1	Tableau de bord : vue d'ensemble de la plateforme. Cette page sert de point d'entrée et présente les jeux de données disponibles, les conversations récentes et les indicateurs clés de l'activité.	76
A.2	Tableau de bord (thème clair) : indicateurs clés de la plateforme — jeux de données connectés, volume de lignes, conversations actives et rapports générés.	77
A.3	Page de gestion des jeux de données : liste des jeux de données importés dans la plateforme, avec les actions disponibles (ouvrir, créer une conversation, supprimer).	77
A.4	Page de détail d'un jeu de données : informations générales (nom, taille, format, source) et accès aux différentes vues d'analyse (schéma, aperçu, statistiques).	78

A.5	Mappage des colonnes : lors de l'import, la plateforme associe les colonnes du fichier aux types de données attendus, garantissant une interprétation correcte du contenu.	78
A.6	Aperçu des données : les premières lignes du jeu de données sont affichées pour vérifier le contenu importé avant analyse.	79
A.7	Schéma du jeu de données : liste des colonnes avec leurs types et leurs caractéristiques, offrant une vision structurée des données.	79
A.8	Statistiques du jeu de données : indicateurs descriptifs calculés par le profilage (moyenne, médiane, écart-type, valeurs manquantes, valeurs uniques).	80
A.9	Nouvelle conversation : espace de discussion vide, point de départ d'une analyse en langage naturel.	80
A.10	Nouvelle conversation associée à un jeu de données : l'utilisateur sélectionne le contexte de l'analyse avant de poser sa question.	81
A.11	Requête en langage naturel : exemple de question posée par l'utilisateur, qui sera décomposée par l'agent en étapes d'analyse.	81
A.12	Planification : l'agent décompose la requête en un plan d'étapes ordonné, conformément à l'orchestration LangGraph.	82
A.13	Exécution des outils : déroulement des étapes du plan avec l'affichage des appels d'outils, garantissant la traçabilité du traitement.	82
A.14	Raisonnement de l'agent : présentation du cheminement de l'agent au cours de l'analyse, renforçant la transparence du processus.	83
A.15	Génération de graphiques : l'agent produit des visualisations en appui de son analyse, accompagnées de leur interprétation.	83
A.16	Génération de graphiques (thème clair) : l'agent présente une visualisation (répartition des cas par pays) accompagnée de son interprétation directement dans la conversation.	84
A.17	Résultat de l'agent : réponse finale présentée à l'utilisateur, fondée sur l'evidence accumulée au cours de l'analyse.	84
A.18	Explication et interprétation : détail des conclusions de l'agent et de leur signification pour l'utilisateur.	85
A.19	Liste des artefacts : panneau présentant l'ensemble des artefacts produits par l'analyse (graphiques, exports, rapports).	85
A.20	Aperçu d'un artefact : consultation d'un graphique ou d'un rapport produit par l'agent, avec possibilité de le télécharger.	86
A.21	Paramètres : gestion des préférences de l'espace de travail (langue et thème). Cette page permet de basculer l'interface entre le français et l'anglais.	86
A.22	Internationalisation : les mêmes paramètres affichés en français avec le thème sombre, illustrant la bascule de langue et d'apparence de l'interface.	87

A.23 Passerelle de modèles locaux (ModelCtl) : tableau de bord de gestion
de l’infrastructure d’inférence — modèles GGUF installés, espace de
stockage consommé et état de l’API. 87

Liste des tableaux

1.1	Structure du mémoire	2
2.1	Fiche signalétique du GST-TTA	4
2.2	Composition de l'équipe projet et répartition des responsabilités	16
3.1	Comparaison des principales plateformes d'analyse conversationnelle	25
4.1	Principes de conception et mise en œuvre	32
4.2	Traduction des besoins fonctionnels en fonctionnalités du système . . .	33
4.3	Approches de gestion du contexte d'un LLM et limites	43
5.1	L'intelligence des données et sa mise en œuvre dans la plateforme . . .	50
5.2	Principaux points de terminaison de l'API REST	53
5.3	Les outils de l'agent analyste de données	59
6.1	Synthèse des tests et validation	71

Introduction générale

Au cours des dernières années, la transformation numérique est devenue un levier stratégique pour les organisations publiques et privées. Cette évolution s'accompagne d'une croissance exponentielle du volume de données produites quotidiennement par les différents systèmes d'information. Dans le domaine de la santé, cette quantité importante de données constitue une ressource précieuse pour le suivi des activités hospitalières, l'évaluation des performances, la recherche scientifique ainsi que l'aide à la prise de décision.

Parallèlement, les progrès réalisés dans le domaine de l'intelligence artificielle, notamment avec l'émergence des grands modèles de langage (*Large Language Models* – LLM), ont profondément modifié les modes d'interaction entre l'utilisateur et les systèmes informatiques. Ces modèles permettent aujourd'hui de comprendre des requêtes formulées en langage naturel, de raisonner sur des informations complexes et de générer des réponses cohérentes tout en facilitant l'accès à des connaissances auparavant réservées à des utilisateurs spécialisés.

Malgré ces avancées, l'analyse de données demeure une tâche relativement complexe. Les outils traditionnels nécessitent généralement la maîtrise de langages de programmation tels que Python ou R, de bibliothèques spécialisées comme pandas, ou encore de plateformes décisionnelles dédiées. Cette dépendance aux compétences techniques constitue un frein important pour de nombreux utilisateurs métiers souhaitant exploiter leurs données de manière autonome.

Dans ce contexte, le **Groupe Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)** poursuit sa stratégie de transformation numérique afin d'améliorer l'exploitation des données issues de ses établissements de santé. Cette démarche vise à développer des solutions innovantes permettant de faciliter l'accès à l'information, d'améliorer les capacités d'analyse et de soutenir la prise de décision.

C'est dans cette perspective que s'inscrit ce projet de fin d'année, dont l'objectif est de concevoir une **plateforme intelligente d'analyse de données conversationnelle**. La solution développée permet aux utilisateurs d'interagir avec leurs jeux de données en langage naturel, de réaliser automatiquement des analyses statistiques, de générer des visualisations pertinentes et d'obtenir des rapports synthétiques grâce à l'intégration d'agents d'intelligence artificielle.

1.1 Organisation du rapport

Le présent mémoire est organisé en six chapitres complémentaires, suivis d'une conclusion générale.

TAB. 1.1 : Structure du mémoire

Partie	Contenu
Chapitre 1 — Introduction générale	Situe le contexte de la transformation numérique du secteur de la santé, présente la problématique et l'organisation du mémoire.
Chapitre 2 — Contexte général	Présente l'organisme d'accueil (GST-TTA), l'analyse de l'existant, la problématique, les objectifs et la méthodologie adoptée.
Chapitre 3 — Analyse des besoins	Décrit les besoins fonctionnels et non fonctionnels, les acteurs, l'état de l'art, l'étude comparative et le positionnement de la solution.
Chapitre 4 — Conception et architecture	Détaille la conception générale, la modélisation UML, l'architecture logicielle, l'architecture agentique, le moteur d'analyse et la gestion des données.
Chapitre 5 — Réalisation	Présente la mise en œuvre effective : backend, orchestration et evidence, moteur d'analyse, interface et internationalisation, déploiement et infrastructure de modèles locaux.
Chapitre 6 — Tests et validation	Expose la stratégie de test, les scénarios réalisés, les résultats obtenus, les limites et les perspectives d'amélioration.
Conclusion générale	Synthétise les travaux réalisés, les apports du projet et les perspectives d'évolution.

Synthèse

Cette organisation permet de présenter progressivement les différentes étapes ayant conduit à la conception et au développement de la plateforme tout en mettant en évidence les contributions techniques et scientifiques apportées par ce projet.

Chapitre 2

Présentation et Contexte général

2.1 Introduction

Ce chapitre présente le contexte institutionnel et technologique dans lequel s'inscrit le présent projet. Il débute par une présentation du Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA), organisme d'accueil ayant encadré la réalisation du projet. Il expose ensuite les principales missions de cette institution, son organisation ainsi que le rôle de la Direction de la Formation, de la Recherche et de l'Innovation, au sein de laquelle le projet a été développé.

Ce chapitre introduit également les enjeux liés à la transformation numérique du secteur de la santé et met en évidence les motivations ayant conduit au développement d'une plateforme intelligente d'analyse conversationnelle de données. L'analyse de l'existant permet d'identifier les limites des solutions actuelles, justifiant la formulation de la problématique et des objectifs du projet.

Enfin, ce chapitre présente l'organisation du travail à travers la structure de l'équipe projet, la répartition des tâches et la planification temporelle via un diagramme de Gantt.

2.2 Présentation de l'organisme d'accueil

2.2.1 Présentation du GST-TTA

Le présent projet de fin d'année a été réalisé au sein du **Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)**, établissement public de santé placé sous la tutelle du **Ministère de la Santé et de la Protection Sociale**.

Créé dans le cadre de la réforme du système national de santé et de la mise en œuvre de la régionalisation avancée, le GST-TTA constitue une nouvelle structure de gouvernance visant à améliorer la coordination des établissements de santé, renforcer la qualité des soins et optimiser la gestion des ressources hospitalières à l'échelle régionale.

Le Groupement couvre l'ensemble de la région Tanger-Tétouan-Al Hoceïma, représentant près de 3,7 millions d'habitants, et fédère plusieurs établissements hospitaliers, centres spécialisés ainsi que des structures de formation et de recherche.

Au-delà de ses missions de coordination, le GST-TTA conduit une politique de modernisation des systèmes d'information afin d'accompagner la transformation numérique des établissements de santé et de favoriser une prise de décision fondée sur l'exploitation des données.

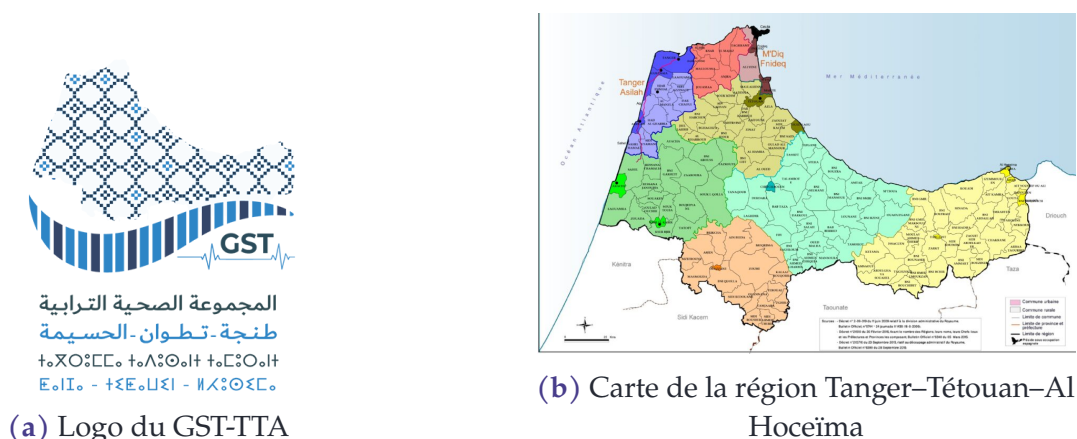


FIG. 2.1 : Présentation du Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma

2.2.2 Fiche signalétique

Les principales informations relatives à l'organisme d'accueil sont présentées dans le tableau 2.1.

Élément	Description
Raison sociale	Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)
Tutelle	Ministère de la Santé et de la Protection Sociale
Statut juridique	Établissement public de santé, sous la tutelle de l'État
Région couverte	Tanger-Tétouan-Al Hoceïma
Population desservie	Environ 3,7 millions d'habitants
Siège	Tanger
Site web	https://gst-tta.ma
Direction d'accueil	Direction de la Formation, de la Recherche et de l'Innovation

TAB. 2.1 : Fiche signalétique du GST-TTA

2.2.3 Missions

Le GST-TTA assure la gouvernance des établissements de santé de la région et veille à l'amélioration continue de la qualité des services proposés aux citoyens.

Ses principales missions sont les suivantes :

- coordonner les établissements hospitaliers de la région ;
- améliorer la qualité, la sécurité et la continuité des soins ;
- optimiser la gestion des ressources humaines, matérielles et financières ;
- accompagner la transformation numérique du système hospitalier ;
- promouvoir la recherche scientifique et l'innovation médicale ;
- développer les programmes de formation destinés aux professionnels de santé.

L'ensemble de ces missions contribue à renforcer l'efficacité du système de santé régional tout en favorisant l'introduction de solutions numériques innovantes. Ces missions, notamment l'accompagnement de la transformation numérique et la promotion de l'innovation, justifient le développement d'une plateforme comme celle présentée dans ce mémoire.

2.2.4 Organisation

Le Groupement Sanitaire Territorial est organisé autour d'une direction générale appuyée par plusieurs directions fonctionnelles couvrant les domaines médicaux, administratifs, financiers, techniques ainsi que les activités liées à la recherche et à l'innovation.

Cette organisation favorise une gouvernance intégrée des établissements de santé et facilite la conduite de projets transversaux, notamment ceux liés à la transformation numérique.



Le projet a été réalisé au sein de la **Direction de la Formation, de la Recherche et de l'Innovation (DFRI)**.

Cette direction constitue un acteur majeur de la stratégie d'innovation du Groupe. Elle accompagne les établissements de santé dans leurs activités de formation, coordonne les projets de recherche et favorise le développement de solutions numériques répondant aux besoins du secteur hospitalier.

- la coordination des activités de formation ;
- le développement de projets de recherche scientifique ;
- la promotion de l'innovation technologique ;
- la valorisation des données de santé ;
- l'accompagnement des projets de transformation numérique.

6

Positionnement du projet

Le projet a été réalisé au sein de la Direction de la Formation, de la Recherche et de l'Innovation du GST-TTA. Il contribue à la stratégie de transformation numérique de l'établissement en proposant une plateforme conversationnelle capable de faciliter l'exploitation des données grâce à l'intelligence artificielle générative.

2.3 Contexte général et analyse de l'existant

Le secteur de la santé connaît actuellement une profonde transformation numérique, marquée par la généralisation des systèmes d'information hospitaliers et la multiplication des données produites au quotidien. Cette évolution offre de nouvelles perspectives pour le pilotage des établissements de santé, l'amélioration de la qualité des soins et l'aide à la prise de décision.

Dans ce contexte, le Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA) poursuit la modernisation de son infrastructure numérique afin de renforcer la gestion des établissements hospitaliers de la région. Toutefois, malgré la disponibilité d'un volume considérable de données, leur exploitation demeure complexe en raison des outils utilisés, de la diversité des sources d'information et des contraintes propres au domaine de la santé.

Cette section présente le contexte numérique du GST-TTA, les caractéristiques des données manipulées, le processus actuel d'analyse ainsi que les principales limites des solutions existantes.

2.3.1 Transformation numérique au GST-TTA

Dans le cadre de la réforme du système national de santé, le GST-TTA s'inscrit dans une stratégie visant à accélérer la transformation numérique des établissements hospitaliers de la région. Cette stratégie repose sur la modernisation des processus administratifs, le développement des systèmes d'information hospitaliers ainsi que l'amélioration de la gouvernance fondée sur les données.

La digitalisation progressive des activités permet aujourd'hui de dématérialiser de nombreux processus auparavant réalisés de manière manuelle, tout en facilitant la circulation de l'information entre les différents établissements du groupement.

Parmi les principaux axes de cette transformation figurent notamment :

- la numérisation des processus administratifs et médicaux ;
- l'amélioration de la qualité et de la disponibilité des données ;
- le développement d'outils décisionnels ;

- la promotion des projets d'innovation numérique ;
- l'intégration progressive des technologies d'intelligence artificielle.

Cette évolution permet au GST-TTA de disposer d'un environnement numérique de plus en plus riche, capable de soutenir efficacement les activités de pilotage et d'aide à la décision.

2.3.2 Données et systèmes d'information hospitaliers

Les établissements hospitaliers produisent quotidiennement un volume important de données provenant de multiples systèmes d'information. Ces données concernent aussi bien les activités médicales que les aspects administratifs, logistiques et financiers.

Parmi les principales catégories de données exploitées, on distingue :

- les données relatives aux patients et aux hospitalisations ;
- les consultations et actes médicaux ;
- les résultats des laboratoires ;
- les données de pharmacie ;
- les ressources humaines ;
- les indicateurs de performance hospitalière ;
- les données financières et logistiques.

Ces informations présentent plusieurs caractéristiques qui rendent leur exploitation particulièrement complexe :

- un volume très important de données ;
- une mise à jour continue des informations ;
- une diversité des formats et des sources ;
- un niveau élevé de sensibilité nécessitant des mesures strictes de confidentialité ;
- des besoins croissants en matière d'analyse et de visualisation.

L'ensemble de ces données constitue une ressource stratégique permettant d'améliorer le suivi des activités hospitalières et de soutenir les décisions des responsables médicaux et administratifs.

2.3.3 Processus actuel et difficultés rencontrées

À l'heure actuelle, l'exploitation des données repose principalement sur une succession d'étapes réalisées à l'aide de plusieurs outils distincts.

Le processus classique comprend généralement :

1. l'extraction des données depuis les différents systèmes d'information ;
2. le nettoyage et la préparation des jeux de données ;
3. la réalisation des traitements statistiques ;
4. la création des tableaux et graphiques ;
5. l'interprétation des résultats ;
6. la rédaction des rapports.

Bien que cette approche permette d'obtenir les analyses souhaitées, elle présente plusieurs inconvénients majeurs.

Les utilisateurs doivent souvent maîtriser plusieurs logiciels spécialisés tels que Microsoft Excel, SQL, Python ou encore Power BI. Cette multiplicité d'outils implique des manipulations répétitives, augmente les risques d'erreurs et allonge considérablement le temps nécessaire à la production des résultats.

Par ailleurs, la majorité des professionnels de santé ne disposent pas nécessairement des compétences techniques requises pour réaliser des analyses complexes, ce qui limite leur autonomie dans l'exploitation des données et crée une dépendance vis-à-vis des services spécialisés.

2.3.4 Limites des solutions existantes

Les solutions actuellement disponibles présentent trois limites majeures qui freinent l'exploitation efficace des données de santé.

Barrière technique

Les outils traditionnels d'analyse de données (Excel, SQL, Power BI, Python) exigent des compétences techniques avancées en programmation, en statistiques ou en gestion de bases de données. Leur utilisation reste donc largement réservée aux profils spécialisés, limitant ainsi l'accès à l'information pour les utilisateurs métiers (médecins, personnels administratifs, décideurs).

Problèmes de confidentialité

L'émergence des assistants conversationnels basés sur les grands modèles de langage (LLMs), tels que ChatGPT, Claude ou Gemini, a ouvert de nouvelles perspectives pour l'analyse de données en langage naturel. Ces interfaces réduisent considérablement la barrière technique et permettent à des utilisateurs non spécialistes d'interroger leurs données.

Toutefois, ces plateformes publiques ne répondent pas aux exigences de confidentialité imposées par le secteur de la santé. Les données médicales réelles, contenant des

informations sensibles relatives aux patients, sont soumises à des contraintes réglementaires et éthiques strictes. Leur transmission vers des services externes hébergés sur des infrastructures publiques représente un risque majeur pour la sécurité, la confidentialité et la souveraineté des données.

Fragmentation des outils

Les utilisateurs sont souvent contraints de manipuler plusieurs outils distincts afin de préparer les données, réaliser les analyses statistiques, produire des visualisations puis rédiger des rapports. Cette fragmentation du processus complexifie le travail, augmente le risque d'erreurs et ralentit considérablement la prise de décision.

De plus, les assistants conversationnels existants reposent principalement sur des mécanismes de génération de texte. Ils ne garantissent pas toujours la reproductibilité des traitements analytiques, la traçabilité des calculs effectués ni la transparence des résultats obtenus.

Synthèse de l'analyse

L'analyse de l'existant montre que les établissements de santé disposent aujourd'hui d'un patrimoine informationnel considérable, mais que son exploitation demeure freinée par trois obstacles majeurs : (i) la complexité technique des outils, (ii) les contraintes de confidentialité des données de santé, et (iii) la fragmentation des processus d'analyse.

Ces limites mettent en évidence la nécessité de disposer d'une plateforme intégrée, capable de combiner les avantages des interfaces conversationnelles avec la fiabilité d'un moteur d'analyse déterministe, tout en assurant la confidentialité des données manipulées.

2.4 Problématique

L'exploitation des données constitue aujourd'hui un enjeu majeur pour les établissements de santé. Avec la généralisation des systèmes d'information hospitaliers, les établissements produisent quotidiennement un volume considérable de données issues de multiples sources : dossiers médicaux électroniques, systèmes d'information hospitaliers, laboratoires, services administratifs, indicateurs de performance et données de recherche.

Ces données sont non seulement **massives**, mais également **dynamiques**, puisqu'elles sont continuellement mises à jour pour refléter l'évolution des activités médicales et administratives.

Les constats établis dans l'analyse de l'existant — barrière technique, problèmes de confidentialité et fragmentation des outils — mettent en évidence la nécessité de disposer d'une solution capable d'exploiter efficacement des données sensibles tout en restant simple d'utilisation, sécurisée et adaptée aux besoins des établissements de santé.

Ainsi, la problématique centrale de ce projet peut être formulée comme suit :

Comment concevoir une plateforme intelligente permettant aux professionnels de santé d'interagir avec des jeux de données massifs et continuellement mis à jour en langage naturel, tout en garantissant la confidentialité des données sensibles, la fiabilité des traitements analytiques, la transparence des résultats, la modularité du système et la reproductibilité des analyses ?

Cette problématique soulève plusieurs défis complémentaires :

- **Défi technique** : concevoir une architecture modulaire séparant l'orchestration pilotée par l'IA de l'exécution computationnelle déterministe ;
- **Défi sécuritaire** : garantir la confidentialité et la souveraineté des données de santé tout en offrant des fonctionnalités d'analyse avancées ;
- **Défi d'usage** : proposer une interface intuitive permettant à des utilisateurs non techniques d'exploiter efficacement leurs données ;
- **Défi d'extensibilité** : développer une solution facilement adaptable à l'évolution des besoins et des technologies.

2.5 Objectifs du projet

2.5.1 Objectif général

L'objectif général de ce projet consiste à **concevoir et développer une plateforme conversationnelle d'analyse de données** reposant sur une architecture modulaire intégrant des agents d'intelligence artificielle. Cette plateforme doit permettre aux utilisateurs d'exploiter efficacement leurs données sans nécessiter de compétences avancées en programmation ou en science des données.

2.5.2 Objectifs spécifiques

Afin d'atteindre cet objectif général, plusieurs objectifs spécifiques ont été définis :

1. **Conception architecturale** : concevoir une architecture logicielle modulaire favorisant la maintenabilité et l'évolutivité de la plateforme ;

2. **Interface conversationnelle** : développer une interface conversationnelle intuitive permettant d'interroger les données en langage naturel ;
3. **Orchestration intelligente** : intégrer un agent intelligent capable d'orchestrer les différentes étapes d'une analyse de données ;
4. **Automatisation des traitements** : automatiser les traitements statistiques ainsi que les opérations courantes de préparation et d'exploration des données ;
5. **Visualisation automatique** : générer automatiquement des visualisations adaptées aux résultats obtenus ;
6. **Persistance des données** : permettre la gestion et la persistance des conversations ainsi que des artefacts produits au cours des analyses ;
7. **Communication fluide** : assurer une communication fluide entre les différents composants de la plateforme grâce à une architecture moderne basée sur des API REST et le streaming des réponses ;
8. **Extensibilité** : développer une solution facilement extensible permettant l'ajout de nouveaux outils d'analyse et de nouveaux modèles d'intelligence artificielle.

2.6 Méthodologie adoptée

La réalisation de ce projet s'est appuyée sur une démarche de développement agile, adaptée aux spécificités d'un projet de recherche et développement en intelligence artificielle. Cette approche itérative et incrémentale a permis une progression maîtrisée, depuis une vision large de la plateforme jusqu'à une spécialisation progressive dans le domaine de la santé, en intégrant les retours des parties prenantes à chaque étape.

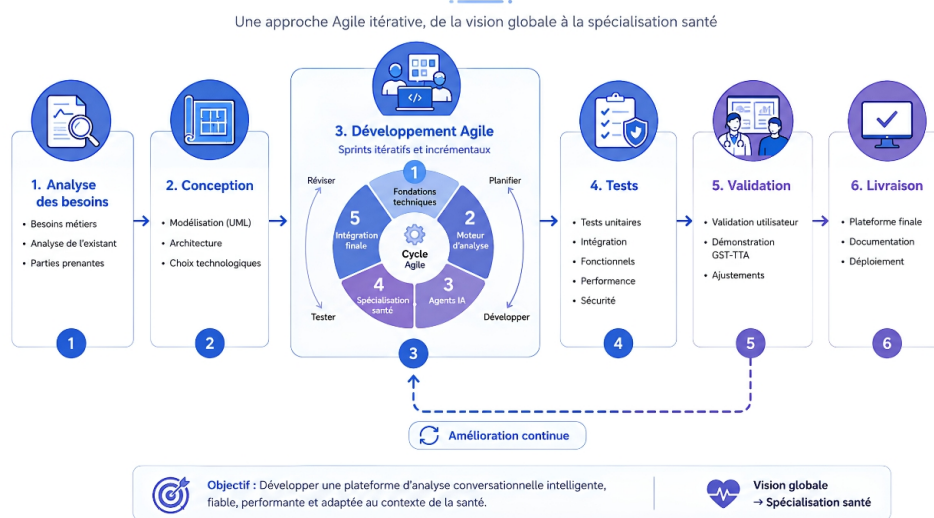


FIG. 2.3 : Démarche générale adoptée pour la réalisation du projet

2.6.1 Approche Agile adaptée

Notre méthodologie s'inscrit dans une logique Agile, caractérisée par des cycles de développement courts (sprints) et une livraison incrémentale des fonctionnalités. Cette approche a été particulièrement adaptée à notre contexte, où les besoins ont évolué d'une vision générique vers une spécialisation progressive.

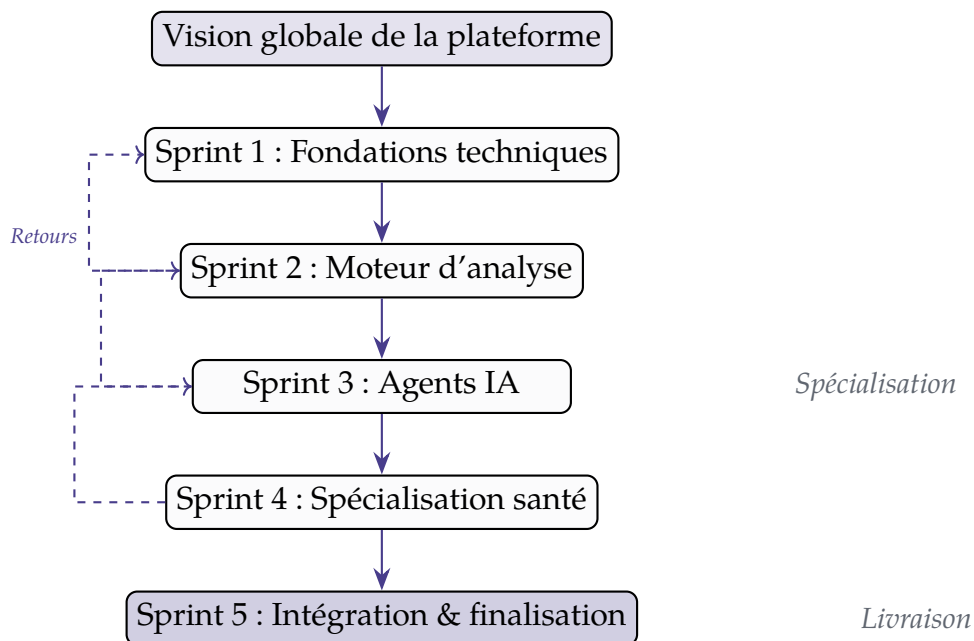


FIG. 2.4 : Structure des sprints : de la vision globale à la spécialisation santé

2.6.2 Organisation en sprints

Le projet a été structuré en cinq sprints de deux à trois semaines, chacun ayant un objectif spécifique et livrant un incrément fonctionnel de la plateforme :

1. **Sprint 1 : Fondations techniques** — Mise en place de l’infrastructure de base : structure du monorepo, environnement de développement, configuration des outils, et premier prototype d’interface.
2. **Sprint 2 : Moteur d’analyse** — Développement du moteur computationnel (pandas, matplotlib) avec support multi-formats et fonctions statistiques essentielles.
3. **Sprint 3 : Agents IA et orchestration** — Implémentation du framework agent (LangGraph), création des outils d’analyse, et mise en place de l’orchestrateur en trois phases.
4. **Sprint 4 : Spécialisation santé** — Adaptation de la plateforme aux données du GST-TTA : intégration de jeux de données réels, personnalisation des analyses pour le contexte médical, et mise en place de la persistance.
5. **Sprint 5 : Intégration et finalisation** — Assemblage complet des composants, tests d’intégration, optimisation des performances, et préparation de la livraison.

Cette organisation a permis de maintenir un rythme de développement soutenu tout en assurant une validation continue des fonctionnalités produites.

2.6.3 Phases de réalisation

Au-delà de l’organisation en sprints, la réalisation du projet s’est articulée autour de quatre grandes phases complémentaires, permettant de structurer progressivement le développement de la plateforme.

Phase 1 : Analyse des besoins

Cette première phase a consisté à identifier les besoins des utilisateurs, à étudier le contexte du projet et à analyser les solutions existantes. Elle a permis de définir les objectifs du système ainsi que les principales exigences fonctionnelles et non fonctionnelles.

Phase 2 : Conception et architecture

La seconde phase a été consacrée à la modélisation de la solution et à la définition de son architecture logicielle. Les différents composants du système ont été conçus selon une approche modulaire afin de garantir leur maintenabilité, leur évolutivité et leur réutilisabilité.

Phase 3 : Développement

La troisième phase correspond à l'implémentation progressive de la plateforme selon une approche Agile. Les fonctionnalités ont été développées de manière incrémentale au cours de plusieurs sprints, permettant une validation continue des résultats obtenus et une adaptation aux besoins identifiés.

Phase 4 : Tests et validation

Enfin, une phase de validation a été menée afin de vérifier le bon fonctionnement de la plateforme. Différents niveaux de tests ont été réalisés pour évaluer la qualité de la solution, sa robustesse, ses performances ainsi que son adéquation aux objectifs fixés.

Synthèse méthodologique

La méthodologie adoptée combine une approche Agile basée sur des développements itératifs avec une progression structurée en quatre phases : analyse, conception, développement et validation. Cette organisation a permis de faire évoluer progressivement la plateforme, depuis une vision générale jusqu'à une solution adaptée aux besoins du GST-TTA.

2.7 Management du projet

La bonne conduite d'un projet de cette envergure nécessite une organisation rigoureuse, une répartition claire des responsabilités et un suivi temporel des différentes tâches. Cette section présente les principaux aspects du management du projet : l'équipe impliquée, la répartition des tâches et la planification temporelle.

2.7.1 Équipe du projet

Le projet a été mené par une équipe pluridisciplinaire, encadrée par des professionnels du GST-TTA et de l'établissement d'enseignement supérieur. L'équipe se compose comme suit :

Rôle	Nom	Responsabilités
Stagiaires	El Gharib Mahmoud	- Analyse des structures et des sources de données ; - Analyse et conception fonctionnelle et technique ; - Modélisation et conception de l'architecture du système ; - Développement de la plateforme et de ses différentes fonctionnalités ; - Mise en place du moteur d'analyse et de l'orchestration agentique ; - Intégration des modèles de langage et des composants d'intelligence artificielle ; - Mise en place de la persistance, du cache et de la gestion des artefacts ; - Réalisation des tests et validation des fonctionnalités ; - Suivi des performances et amélioration de la solution ; - Documentation et préparation de la soutenance.
	Zarqi Ezzoubair	
	El Attar Mohamed	
Encadrante pédagogique	Pr. Benabdoulah Ikram	Suivi académique du projet, orientation méthodologique, correction et suivi du mémoire ainsi que préparation de la soutenance.
Encadrant technique	FANGACHI Oussama	Orientation technique, validation des fonctionnalités, accompagnement du développement et exécution des tests.

TAB. 2.2 : Composition de l'équipe projet et répartition des responsabilités

2.7.2 Diagramme de Gantt

La planification temporelle du projet a été établie sous forme de diagramme de Gantt, permettant de visualiser la séquence et la durée des différentes tâches. Ce diagramme couvre l'ensemble de la période de réalisation du projet, depuis l'analyse des besoins jusqu'à la livraison finale.

Planification du projet

Diagramme de Gantt • 20 juin 2026 — 1 septembre 2026

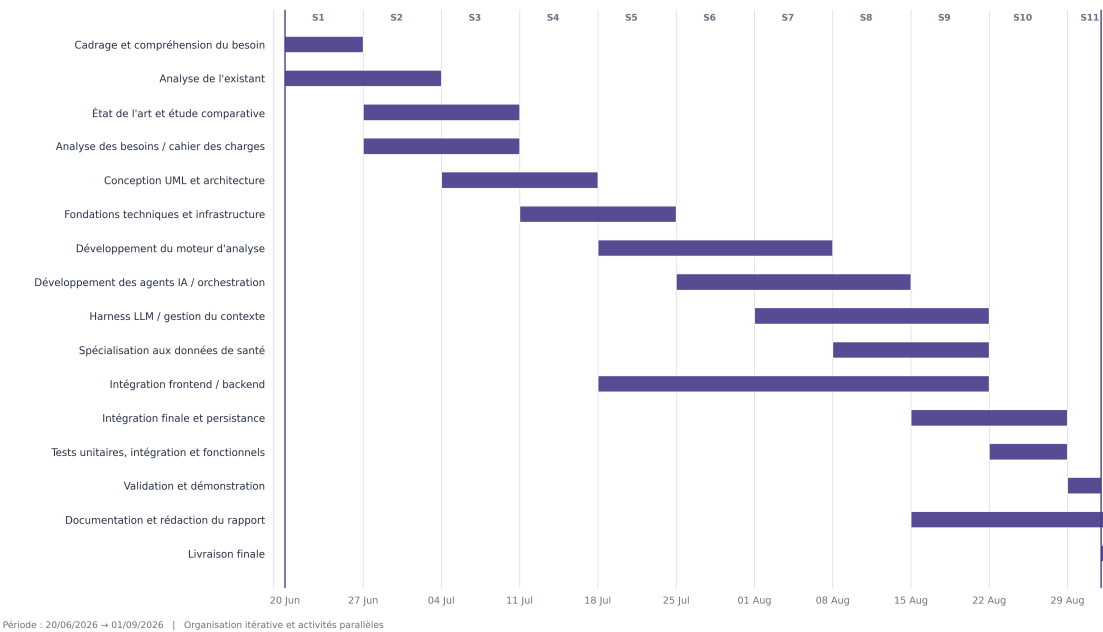


FIG. 2.5 : Diagramme de Gantt du projet

La planification suit une organisation itérative dans laquelle plusieurs activités peuvent être menées en parallèle. Cette organisation permet notamment d’articuler progressivement la conception, le développement du moteur d’analyse, l’intégration des agents d’intelligence artificielle, l’implémentation du mécanisme de gestion du contexte, puis les phases d’intégration, de validation et de documentation

Synthèse du management

L’organisation du projet, structurée autour d’une équipe aux rôles clairement définis et d’une planification rigoureuse, a permis de mener à bien les différentes phases dans le respect des délais impartis. La répartition des tâches a favorisé une collaboration efficace entre les acteurs académiques et professionnels.

2.8 Conclusion

Le Groupement Sanitaire Territorial Tanger–Tétouan–Al Hoceïma constitue un environnement particulièrement favorable au développement de projets innovants destinés à améliorer l’exploitation des données de santé. Grâce à sa stratégie de transformation numérique et à l’implication de la Direction de la Formation, de la Recherche et de l’Innovation, il offre un cadre propice à la conception de solutions d’aide à la décision

intégrant les technologies de l'intelligence artificielle.

L'analyse du contexte et de l'existant a mis en évidence les limites des solutions actuelles, justifiant la nécessité de développer une plateforme adaptée aux besoins du secteur hospitalier. La problématique, les objectifs et la méthodologie adoptée constituent les fondements de la démarche entreprise.

L'organisation du projet, structurée autour d'une équipe pluridisciplinaire, d'une répartition claire des tâches et d'une planification temporelle rigoureuse, a permis d'assurer une conduite maîtrisée du projet dans le respect des échéances fixées.

Le chapitre suivant sera consacré à l'état de l'art des technologies et solutions existantes dans le domaine de l'analyse de données conversationnelle, des grands modèles de langage et des architectures d'agents intelligents qui constituent les fondements scientifiques de la plateforme développée.

Chapitre 3

Analyse des besoins et étude de l'existant

3.1 Introduction

Ce chapitre a pour objectif d'analyser les besoins auxquels doit répondre la plateforme proposée et d'étudier les solutions existantes dans le domaine de l'analyse conversationnelle de données. Dans un premier temps, les différents acteurs ainsi que leurs besoins fonctionnels et non fonctionnels sont identifiés afin de définir le périmètre du projet. Dans un second temps, un état de l'art présente les principales technologies mobilisées ainsi qu'une étude comparative des plateformes existantes. Cette analyse permet enfin de justifier les choix retenus pour la conception de la solution proposée.

3.2 Analyse des besoins

Cette section présente les besoins identifiés auprès des futurs utilisateurs de la plateforme. L'objectif est de définir les fonctionnalités attendues ainsi que les contraintes auxquelles le système devra répondre.

3.2.1 Identification des acteurs

L'identification des acteurs constitue une étape préliminaire essentielle pour comprendre les interactions entre les utilisateurs et le système. La plateforme s'adresse à plusieurs catégories d'utilisateurs ayant des rôles et des attentes distincts.

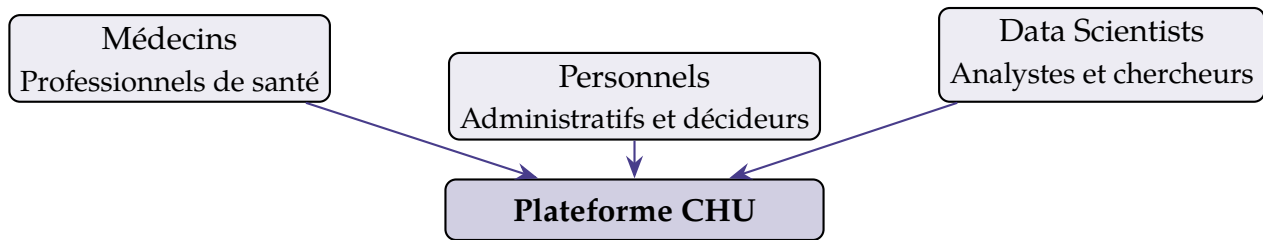


FIG. 3.1 : Acteurs principaux de la plateforme

Les acteurs identifiés sont les suivants :

- **Médecins et professionnels de santé** : utilisateurs finaux souhaitant interroger les données en langage naturel pour obtenir des analyses rapides sur leurs activités ;
- **Personnels administratifs et décideurs** : responsables du pilotage des établissements cherchant à suivre les indicateurs de performance et à prendre des décisions fondées sur les données ;
- **Data scientists et analystes** : utilisateurs techniques nécessitant des fonctionnalités avancées pour des analyses complexes et reproductibles.

3.2.2 Cas d'utilisation

Les cas d'utilisation principaux de la plateforme sont présentés dans le diagramme ci-dessous :

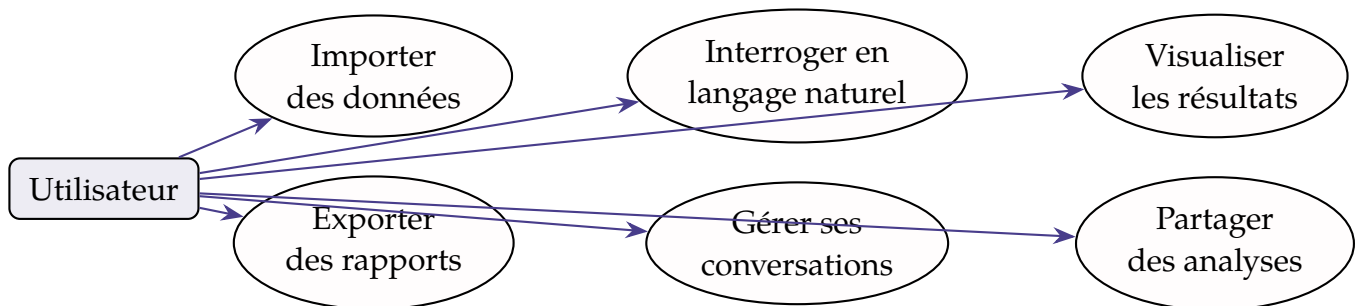


FIG. 3.2 : Diagramme des cas d'utilisation principaux

3.2.3 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que la plateforme doit offrir aux utilisateurs :

- **Importation de données** : support multi-formats (CSV, XLSX, parquet, JSON) avec détection automatique de l'encodage et du séparateur ;
- **Interaction en langage naturel** : interface conversationnelle permettant de formuler des requêtes analytiques sans compétences techniques ;

- **Planification automatique** : décomposition automatique d'une requête complexe en une séquence d'étapes ordonnées et traçables ;
- **Analyse statistique** : calculs statistiques rigoureux (descriptives, corrélations, agrégations, profilage) via un moteur déterministe ;
- **Visualisation automatique** : génération automatique de graphiques adaptés (histogrammes, diagrammes de dispersion, matrices de corrélation) ;
- **Génération de rapports** : production de rapports synthétiques intégrant résultats, graphiques et interprétations ;
- **Streaming en temps réel** : diffusion progressive des résultats (plan, étapes, graphiques, tokens) pour une expérience interactive ;
- **Persistance** : sauvegarde des conversations et des artefacts (graphiques, rapports) entre les sessions ;
- **Gestion des erreurs** : messages d'erreur clairs et suggestions pour guider l'utilisateur.

3.2.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités et contraintes du système :

- **Sécurité** : confidentialité et intégrité des données sensibles, authentification des utilisateurs, chiffrement des communications ;
- **Performance** : réactivité optimale avec un temps de réponse inférieur à 5 secondes pour les opérations courantes ;
- **Modularité** : architecture permettant l'ajout de nouveaux outils, agents ou visualisations sans modifications majeures ;
- **Extensibilité** : intégration facilitée de nouveaux modèles de langage et de nouvelles sources de données ;
- **Traçabilité** : journalisation des actions et des résultats pour l'audit et la reproductibilité des analyses ;
- **Maintenabilité** : bonnes pratiques de développement (tests unitaires, documentation, intégration continue) ;
- **Portabilité** : déploiement sur différentes infrastructures via la conteneurisation (Docker) ;
- **Souveraineté des données** : hébergement exclusif sur l'infrastructure du GST-TTA, sans transmission vers des services externes.

Synthèse des besoins

L'analyse des besoins a permis d'identifier une plateforme intégrée, sécurisée et modulaire, capable de répondre aux exigences du contexte hospitalier tout en offrant une interface intuitive en langage naturel. Les besoins non fonctionnels, notamment la sécurité et la souveraineté des données, constituent des contraintes fortes qui orientent les choix architecturaux.

3.3 État de l'art

Afin de concevoir une solution adaptée aux besoins identifiés, il est nécessaire d'étudier les principales technologies utilisées dans le domaine de l'intelligence artificielle générative et de l'analyse conversationnelle de données.

3.3.1 Intelligence artificielle générative

L'intelligence artificielle générative désigne l'ensemble des modèles capables de produire du contenu original — textes, images, code, etc. — à partir d'une entrée donnée. Contrairement aux modèles discriminants qui classifient ou prédisent, les modèles génératifs apprennent la distribution sous-jacente des données pour en générer de nouvelles.

Les avancées récentes dans ce domaine, notamment avec l'architecture Transformers et le mécanisme d'attention, ont permis l'émergence de modèles de plus en plus performants, capables de comprendre et de générer un langage naturel de qualité.

3.3.2 Grands modèles de langage (LLMs)

Les grands modèles de langage (*Large Language Models*) sont des modèles d'IA générative entraînés sur des corpus textuels massifs. Ils sont capables de comprendre des requêtes en langage naturel, de raisonner sur des informations complexes et de générer des réponses cohérentes.

Parmi les LLMs les plus connus, on peut citer :

- **GPT (OpenAI)** : modèles généralistes aux capacités étendues ;
- **Claude (Anthropic)** : modèle axé sur la sécurité et la fiabilité ;
- **Gemini (Google)** : modèle multimodal intégrant texte et image ;
- **Llama (Meta)** : modèle open source permettant un déploiement local.

Les LLMs ouvrent la voie à de nouvelles interfaces conversationnelles où l'utilisateur interagit en langage naturel, sans nécessiter de compétences techniques particulières.

3.3.3 Agents intelligents

Un agent intelligent est un système capable de percevoir son environnement, de raisonner sur les actions à entreprendre et d'agir de manière autonome pour atteindre un objectif. Dans le contexte de l'analyse de données, un agent peut orchestrer différentes étapes du traitement, appeler des outils spécialisés et synthétiser les résultats.

Les frameworks d'agents permettent aujourd'hui de :

- décomposer des tâches complexes en sous-objectifs ;
- sélectionner dynamiquement les outils adaptés ;
- maintenir un état de la conversation ;
- assurer la traçabilité des actions effectuées.

3.3.4 Model Context Protocol (MCP)

Le Model Context Protocol est un protocole standardisé permettant aux modèles de langage d'interagir avec des outils et des sources de données externes de manière structurée. Il définit des interfaces communes pour :

- la découverte des outils disponibles ;
- l'invocation des outils avec des paramètres validés ;
- le retour des résultats dans un format exploitable.

Ce protocole facilite l'intégration de nouvelles capacités aux agents sans modifier leur logique centrale.

3.3.5 Framework LangGraph

LangGraph est un framework open source permettant de construire des applications d'agents avec des graphes d'états. Il étend les capacités de LangChain en offrant :

- une modélisation explicite du flux de contrôle ;
- la persistance de l'état entre les étapes ;
- la gestion de cycles et de conditionnels ;
- une traçabilité complète des actions.

Cette approche est particulièrement adaptée aux workflows analytiques complexes où chaque étape doit être documentée et vérifiable.

3.3.6 Retrieval-Augmented Generation (RAG)

Le RAG est une approche combinant la génération de texte avec la recherche d'informations dans des bases de données. Elle permet aux LLMs de produire des réponses basées sur des documents spécifiques plutôt que sur leur seule connaissance interne.

Les bénéfices du RAG incluent :

- une réduction des hallucinations du modèle ;
- la capacité à répondre sur des données spécifiques non incluses dans l'entraînement ;
- la traçabilité des sources d'information.

Dans le contexte de l'analyse de données, le RAG peut être utilisé pour fournir des informations contextuelles sur les données (métadonnées, schémas, descriptions) lors de l'interprétation des résultats.

3.3.7 Analyse conversationnelle de données

L'analyse conversationnelle de données est un domaine émergent qui combine interfaces en langage naturel, intelligence artificielle et visualisation de données. Elle permet aux utilisateurs de dialoguer avec leurs données sans avoir à maîtriser des langages de programmation ou des outils statistiques complexes.

Les systèmes d'analyse conversationnelle reposent généralement sur :

- une compréhension du langage naturel pour interpréter les requêtes ;
- un moteur d'exécution pour réaliser les traitements demandés ;
- une génération de langage naturel pour expliquer les résultats ;
- une interface de visualisation interactive.

3.4 Étude comparative des solutions existantes

Cette section présente une comparaison des principales plateformes actuellement disponibles pour l'analyse conversationnelle de données.

TAB. 3.1 : Comparaison des principales plateformes d'analyse conversationnelle

Solution	Chat	Analyse	Graphiques	Local	Open source	Sécurité
ChatGPT ADA	✓	✓	✓	✗	✗	Moyen
Claude Analysis	✓	✓	✓	✗	✗	Moyen
Julius AI	✓	✓	✓	✗	✗	Moyen
Power BI Copilot	✓	✓	✓	Entreprise	✗	Élevée
Tableau AI	✓	✓	✓	Entreprise	✗	Élevée
Notre plateforme	✓	✓	✓	✓	✓	Très élevée

L'analyse comparative met en évidence plusieurs constats :

- **Solutions cloud** : les plateformes existantes (ChatGPT ADA, Claude Analysis, Julius AI) sont majoritairement hébergées sur des infrastructures cloud publiques, ce qui pose des problèmes de confidentialité pour les données de santé ;
- **Solutions propriétaires** : Power BI Copilot et Tableau AI sont des solutions d'entreprise propriétaires, coûteuses et non modifiables ;
- **Manque de transparence** : les mécanismes de traitement sont souvent opaques, ne permettant pas de vérifier la reproductibilité des analyses ;
- **Faible extensibilité** : l'ajout de nouveaux outils ou de nouvelles sources de données est généralement limité.

3.5 Positionnement de la solution proposée

À partir de l'analyse des besoins et de l'étude comparative réalisée, plusieurs limites des solutions existantes ont été identifiées. Bien que les plateformes actuelles proposent des fonctionnalités avancées d'analyse conversationnelle, elles reposent majoritairement sur des services cloud propriétaires, offrent une faible maîtrise des traitements réalisés et répondent difficilement aux exigences de confidentialité imposées par le domaine de la santé.

La plateforme développée dans le cadre de ce projet adopte une approche différente reposant sur :

- une **architecture modulaire** facilitant l'évolution du système et l'ajout de nouvelles fonctionnalités ;
- une **exécution locale** garantissant la souveraineté des données au sein de l'infrastructure du GST-TTA ;

- un **moteur d'analyse déterministe** assurant la reproductibilité et la vérifiabilité des résultats ;
- une **orchestration basée sur des agents intelligents** (LangGraph) pour structurer le processus d'analyse ;
- une **intégration adaptée** aux données hospitalières du GST-TTA, avec persistance des conversations et des artefacts.

Ces caractéristiques permettent de répondre aux besoins identifiés tout en garantissant la confidentialité, la traçabilité et l'évolutivité de la solution. En pratique, les modèles de langage sont servis localement au sein de l'infrastructure du groupement au travers d'une passerelle d'inférence auto-hébergée, conformément à l'exigence de souveraineté. Les API externes compatibles OpenAI ne sont mobilisées que ponctuellement, durant les phases de développement et de test.

Positionnement technologique

La plateforme se distingue par son approche hybride : elle combine la puissance des LLMs pour l'interaction en langage naturel avec un moteur d'analyse computationnel déterministe (pandas, matplotlib...), offrant ainsi à la fois la flexibilité du langage naturel et la rigueur des méthodes statistiques.

3.6 Conclusion

Ce chapitre a permis d'identifier les besoins fonctionnels et non fonctionnels de la plateforme ainsi que les contraintes propres au contexte hospitalier. L'étude des technologies et la comparaison des solutions existantes ont mis en évidence leurs avantages ainsi que leurs limites. Ces analyses justifient les choix technologiques retenus pour le développement de la plateforme et constituent la base de la conception présentée dans le chapitre suivant.

Le chapitre suivant détaille l'architecture logicielle complète de la plateforme, en présentant les différentes composantes développées, leurs interactions et les mécanismes de persistance et de communication.

Chapitre 4

Conception et architecture de la solution

4.1 Introduction

Ce chapitre a pour objectif de présenter la conception technique et l'architecture de la plateforme d'analyse conversationnelle de données. Il constitue le passage des besoins identifiés au chapitre précédent vers une solution technique concrète, en explicitant les choix d'ingénierie qui ont guidé la mise en œuvre.

Le chapitre s'organise en six parties complémentaires. La conception générale de la solution est d'abord présentée, en incluant les choix technologiques, l'architecture globale et les principes de conception retenus. La modélisation du système est ensuite détaillée à travers une modélisation fonctionnelle et les principaux diagrammes UML. L'architecture logicielle est décrite par couches, en précisant les responsabilités de chaque composant et leurs relations. Une attention particulière est ensuite accordée au cœur du système : l'architecture agentique et le moteur d'analyse. Enfin, les mécanismes de communication et de gestion des données sont présentés avant de conclure sur les choix effectués.

4.2 Conception générale de la solution

Cette section présente les choix de conception de haut niveau qui traduisent les besoins fonctionnels et non fonctionnels identifiés au chapitre précédent en décisions techniques. Elle s'articule autour des choix technologiques, de l'architecture globale et des principes de conception.

4.2.1 Choix technologiques

Le choix des technologies constitue une étape décisive de la conception. Il doit permettre de satisfaire simultanément les exigences fonctionnelles de la plateforme (interaction en langage naturel, analyse statistique, visualisation et streaming) et ses

contraintes non fonctionnelles (sécurité, souveraineté des données, modularité, extensibilité et performance).

Les principales technologies retenues sont présentées ci-dessous selon leur domaine d'utilisation. Cette organisation permet de mettre en évidence à la fois leur rôle individuel et leur complémentarité au sein de la plateforme.

01 Interface utilisateur



SvelteKit 5 / Svelte 5



TailwindCSS

Justification

Interface réactive et fluide, adaptée à l'affichage en temps réel du streaming. L'approche par composants permet de conserver une architecture modulaire et réutilisable.

02 API et communication



FastAPI

REST

SSE

Justification

FastAPI fournit une API asynchrone avec validation des données via Pydantic et génération automatique de la documentation OpenAPI. REST assure les échanges classiques avec le frontend, tandis que SSE permet la diffusion des étapes de traitement, des tokens et des artefacts en temps réel.

03 Intelligence artificielle



LangGraph / LangChain

LLM compatible API OpenAI

Justification

LangGraph permet de modéliser le processus d'analyse sous forme de graphe d'états, facilitant l'organisation des différentes étapes, la persistance de l'état, la gestion des erreurs et la traçabilité du traitement. L'architecture reste indépendante du fournisseur de modèles afin de privilégier l'utilisation de modèles locaux, en cohérence avec les exigences de confidentialité et de souveraineté des données. La mise en service des modèles locaux s'appuie sur une passerelle d'inférence auto-hébergée (projet `llms-gateway`) qui sert des modèles GGUF via `llama.cpp` et expose une interface compatible OpenAI. Des modèles accessibles via une API compatible OpenAI ont été utilisés ponctuellement durant les phases de développement et de test afin d'accélérer les expérimentations.

04 Analyse des données



pandas



NumPy



DuckDB

Justification

Ces bibliothèques assurent les opérations de manipulation, de calcul statistique et d'interrogation des données. DuckDB permet notamment l'exécution de requêtes SQL directement sur les fichiers, favorisant la reproductibilité et la vérifiabilité des résultats.

05 Persistance des données



PostgreSQL



SQLAlchemy 2.0

Justification

PostgreSQL assure le stockage fiable des conversations, messages et artefacts. SQLAlchemy 2.0 fournit une couche d'accès aux données modulaire et compatible avec le fonctionnement asynchrone du backend.

06 Visualisation



Plotly



Matplotlib

Justification

Plotly permet de produire des graphiques interactifs sous forme de spécifications JSON, tandis que Matplotlib répond aux besoins de visualisation statique et de génération de figures contrôlées.

07 Déploiement et infrastructure



Docker



Nginx

uv

Justification

Docker assure la conteneurisation des services, Nginx joue le rôle de reverse proxy et uv permet une gestion rapide et reproductible des dépendances Python.

Les choix technologiques présentent ainsi plusieurs cohérences notables. L'utilisation de Python pour le backend, l'orchestration et le moteur d'analyse évite les ruptures de contexte entre les différentes couches computationnelles, tandis que TypeScript, utilisé par SvelteKit, garantit une interface typée et maintenable.

Par ailleurs, les principales briques logicielles retenues sont open source et peuvent être déployées sur une infrastructure locale. Cette caractéristique répond directement aux exigences de souveraineté des données identifiées précédemment, tout en conservant une architecture modulaire et extensible.

4.2.2 Architecture globale

L'architecture globale de la plateforme s'organise autour d'un flux unique qui relie l'utilisateur aux données et aux résultats. Une requête formulée en langage naturel traverse successivement l'interface conversationnelle, l'API backend et l'orchestration pilotée par l'IA, avant d'être exécutée par le moteur d'analyse déterministe. Les résultats produits sont ensuite interprétés par le modèle de langage et restitués à l'utilisateur en temps réel.

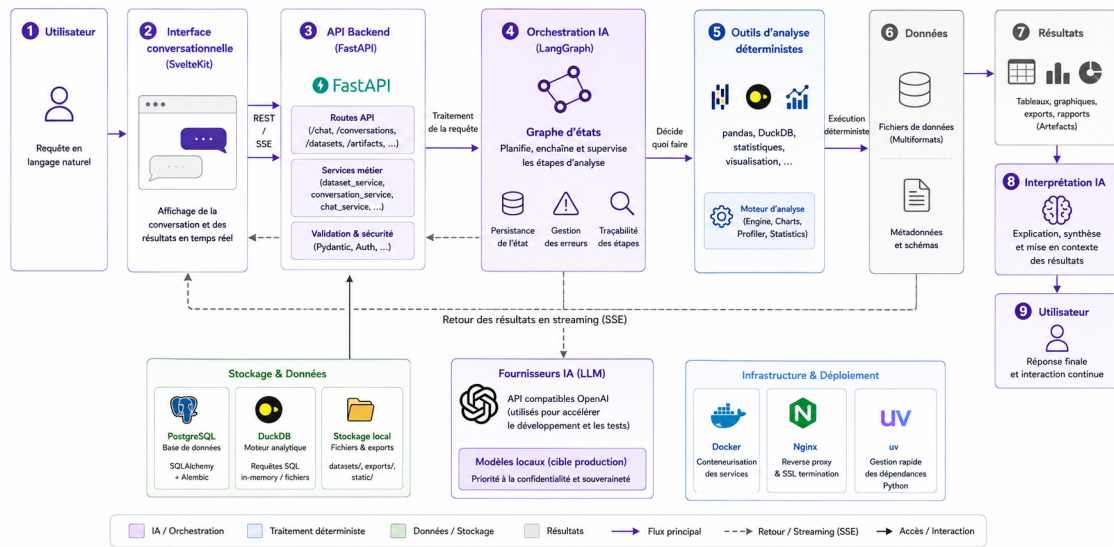


FIG. 4.1 : Architecture globale : de la requête en langage naturel à la restitution des résultats

Comme l'illustre la figure 4.1, l'architecture du système repose sur une séparation claire entre les responsabilités liées à l'intelligence artificielle et celles liées au traitement déterministe des données.

- **Zone IA et orchestration** : elle regroupe l'interface conversationnelle, l'API backend et l'orchestration assurée par LangGraph. Cette zone prend en charge la compréhension de la requête, la planification des étapes d'analyse, l'appel aux outils appropriés ainsi que l'interprétation et la restitution des résultats ;
- **Zone de traitement déterministe** : elle regroupe les outils d'analyse tels que pandas, DuckDB et les différents modules statistiques et de visualisation. Cette zone réalise les calculs effectifs sur les données selon des opérations déterministes, indépendamment du modèle de langage ;
- **Zone données et stockage** : elle assure l'accès aux fichiers de données, aux métadonnées ainsi qu'aux différents espaces de stockage nécessaires au fonctionnement de la plateforme. Elle constitue la source sur laquelle s'appuient les traitements analytiques et le stockage des informations produites.

Cette séparation constitue l'un des choix architecturaux les plus structurants du projet. Le modèle de langage n'effectue pas directement les calculs statistiques ou les opérations analytiques critiques : il intervient principalement pour comprendre la demande, planifier le traitement et interpréter les résultats produits par les outils déterministes. Les calculs sont ainsi réalisés par des composants spécialisés, ce qui permet de garantir leur reproductibilité et leur vérifiabilité.

4.2.3 Principes de conception

La conception de la plateforme repose sur un ensemble de principes qui ont guidé l'ensemble des choix d'ingénierie. Ces principes, présentés dans le tableau 4.1, découlent directement des besoins non fonctionnels identifiés au chapitre précédent.

TAB. 4.1 : Principes de conception et mise en œuvre

Principe	Mise en œuvre
Modularité	Décomposition du système en paquets indépendants (interface, API, agents, moteur d'analyse, outils) dont chacun expose une responsabilité unique.
Séparation des responsabilités	Organisation en couches distinctes où chaque composant assure une fonction précise, sans chevauchement des responsabilités.
Séparation IA / calcul déterministe	L'IA assure la compréhension, la planification et l'interprétation ; le moteur exécute les calculs de manière déterministe et traçable.
Extensibilité	Registre d'outils permettant d'ajouter de nouvelles capacités d'analyse sans modifier l'orchestration ; intégration facilitée de nouveaux modèles de langage.
Sécurité et souveraineté	Déploiement local, données hébergées exclusivement sur l'infrastructure du GST-TTA, aucune transmission vers des services externes.
Traçabilité et reproductibilité	Journalisation des étapes, des outils appelés et des résultats ; persistance des artefacts pour l'audit et la reproductibilité.
Temps réel	Diffusion progressive des résultats (plan, étapes, tokens, artefacts) via le streaming pour une expérience interactive.

Synthèse de la conception générale

La conception générale de la plateforme repose sur une architecture modulaire et sécurisée, articulée autour de la séparation entre le pilotage par l'IA et le calcul déterministe. Les choix technologiques, présentés dans la section consacrée aux choix technologiques, répondent aux besoins fonctionnels et non fonctionnels identifiés au chapitre précédent tout en garantissant la souveraineté des données de santé.

4.3 Modélisation du système

La modélisation du système permet de traduire les besoins identifiés au chapitre précédent en une représentation structurée des fonctionnalités, des interactions et des données de la plateforme. Elle s'articule autour d'une modélisation fonctionnelle et d'une modélisation UML.

4.3.1 Modélisation fonctionnelle

La modélisation fonctionnelle consiste à établir la correspondance entre les besoins fonctionnels exprimés au chapitre précédent et les fonctionnalités concrètes du système. Le tableau 4.2 présente cette traduction en associant chaque besoin à la fonctionnalité qui le satisfait et au composant responsable.

TAB. 4.2 : Traduction des besoins fonctionnels en fonctionnalités du système

Besoin (chapitre précédent)	Fonctionnalité	Composant responsable
Importation multi-formats	Upload, profilage et détection automatique des types	API datasets + moteur d'analyse
Interaction en langage naturel	Interface conversationnelle avec streaming	Frontend + API chat
Planification automatique	Décomposition d'une requête en étapes ordonnées	Planificateur (LLM)
Analyse statistique	Calculs descriptifs, corrélations, agrégations	Moteur d'analyse (outils)
Visualisation automatique	Génération de graphiques adaptés aux résultats	Outils de visualisation
Génération de rapports	Synthèse des résultats et des interprétations	Synthétiseur (LLM)
Streaming temps réel	Diffusion des étapes et des résultats	API chat + Frontend
Persistance	Sauvegarde des conversations et des artefacts	Base de données PostgreSQL
Traçabilité	Journalisation des étapes, outils et résultats	État d'exécution + événements

D'un point de vue fonctionnel, la plateforme se décompose en quatre grands ensembles : un **front-office** assurant l'interaction avec l'utilisateur (upload des données, interface conversationnelle, affichage des résultats), une **orchestration** pilotant le dé-

roulement des analyses (planification, exécution, synthèse), un **moteur d'analyse** réalisant les traitements computationnels et un **service de persistance** garantissant la conservation des conversations et des artefacts produits.

4.3.2 Modélisation UML

La modélisation UML permet de représenter de manière normalisée les différents aspects du système. Le diagramme des cas d'utilisation présenté au chapitre précédent (figure 3.2) définit les interactions entre l'utilisateur et la plateforme. Afin de compléter cette vision, trois diagrammes complémentaires sont présentés : le diagramme de séquence, qui décrit le déroulement d'une requête, le diagramme d'activité, qui modélise le processus d'analyse, et le diagramme de classes, qui représente le modèle de données.

Diagramme de séquence

Le diagramme de séquence de la figure 4.2 illustre le déroulement temporel d'une requête conversationnelle, depuis la formulation de la question jusqu'à l'affichage de la réponse. Il traduit directement les besoins d'interaction en langage naturel et de streaming temps réel.

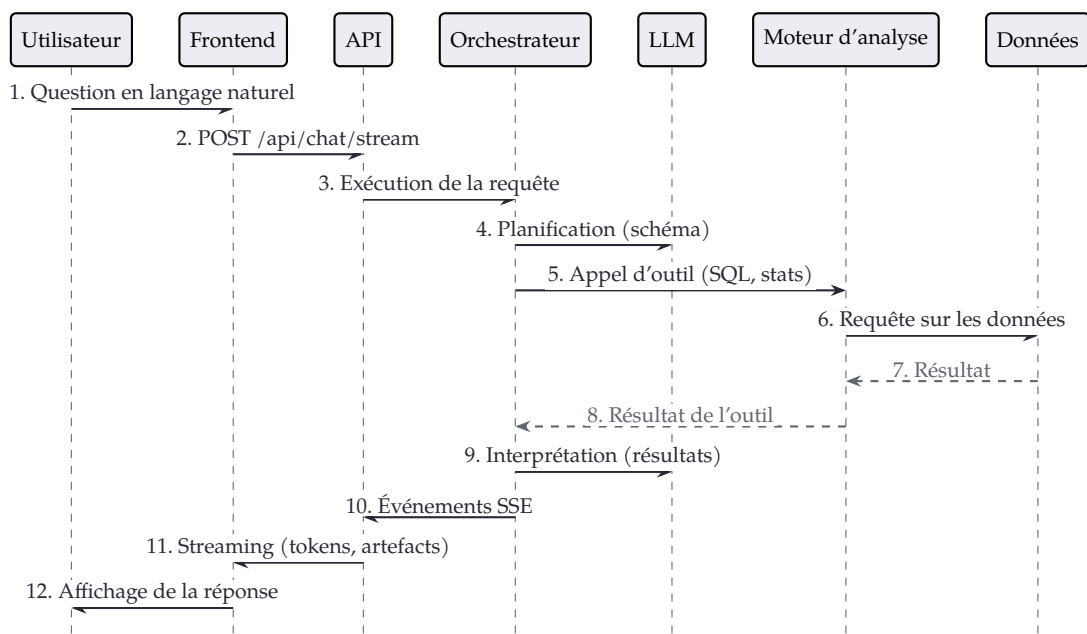


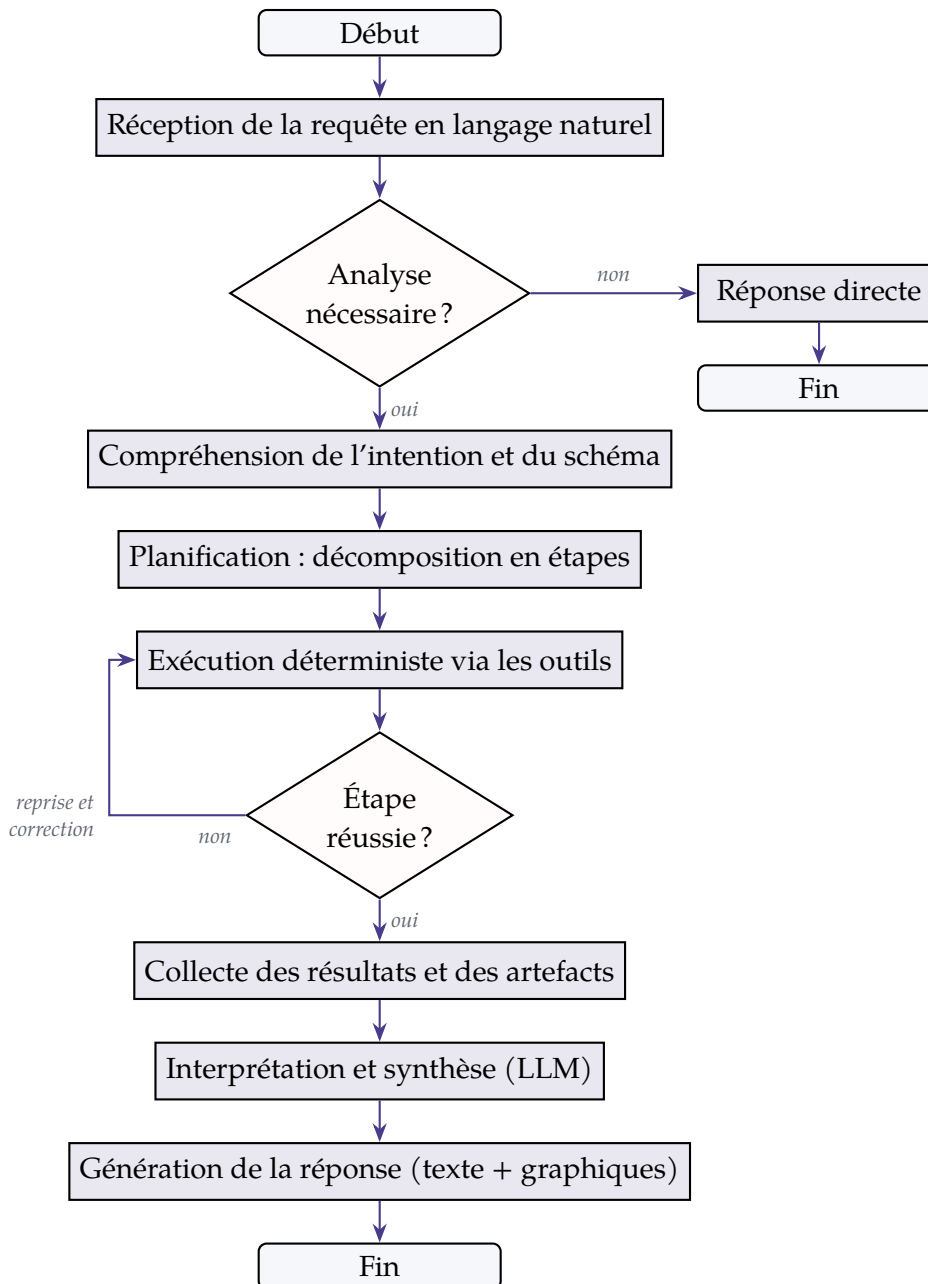
FIG. 4.2 : Diagramme de séquence du traitement d'une requête conversationnelle

Le déroulement met en évidence le rôle central de l'orchestrateur, qui coordonne les appels au modèle de langage pour la planification et l'interprétation, ainsi que les appels au moteur d'analyse pour l'exécution déterministe. Les étapes 5 à 8 sont

répétées pour chaque étape du plan d'exécution. Chaque appel d'outil produit un résultat transmis à l'orchestrateur, puis interprété. L'ensemble des événements (étapes, appels d'outils, artefacts, tokens) est diffusé en temps réel au frontend via le streaming, conformément au besoin de réactivité identifié au chapitre précédent.

Diagramme d'activité

Le diagramme d'activité de la figure 4.3 modélise le processus d'analyse complet, depuis la réception de la requête jusqu'à la génération de la réponse. Il traduit les besoins de planification automatique, d'exécution rigoureuse et de gestion des erreurs.

**FIG. 4.3 :** Diagramme d'activité du processus d'analyse

Le processus distingue le cas où la requête appelle une analyse (chemin principal) de celui où une réponse directe suffit. Pour les requêtes analytiques, le système procède à la compréhension de l'intention, à la planification en étapes ordonnées, puis à l'exécution de chaque étape par les outils d'analyse. En cas d'échec d'une étape, un mécanisme de reprise et de correction est déclenché, conformément au besoin de robustesse et de traçabilité. Les résultats collectés sont ensuite interprétés et synthétisés par le modèle de langage, puis restitués à l'utilisateur.

Diagramme de classes

Le diagramme de classes de la figure 4.4 représente le modèle de données de la plateforme, qui traduit les besoins de persistance des conversations et des artefacts.

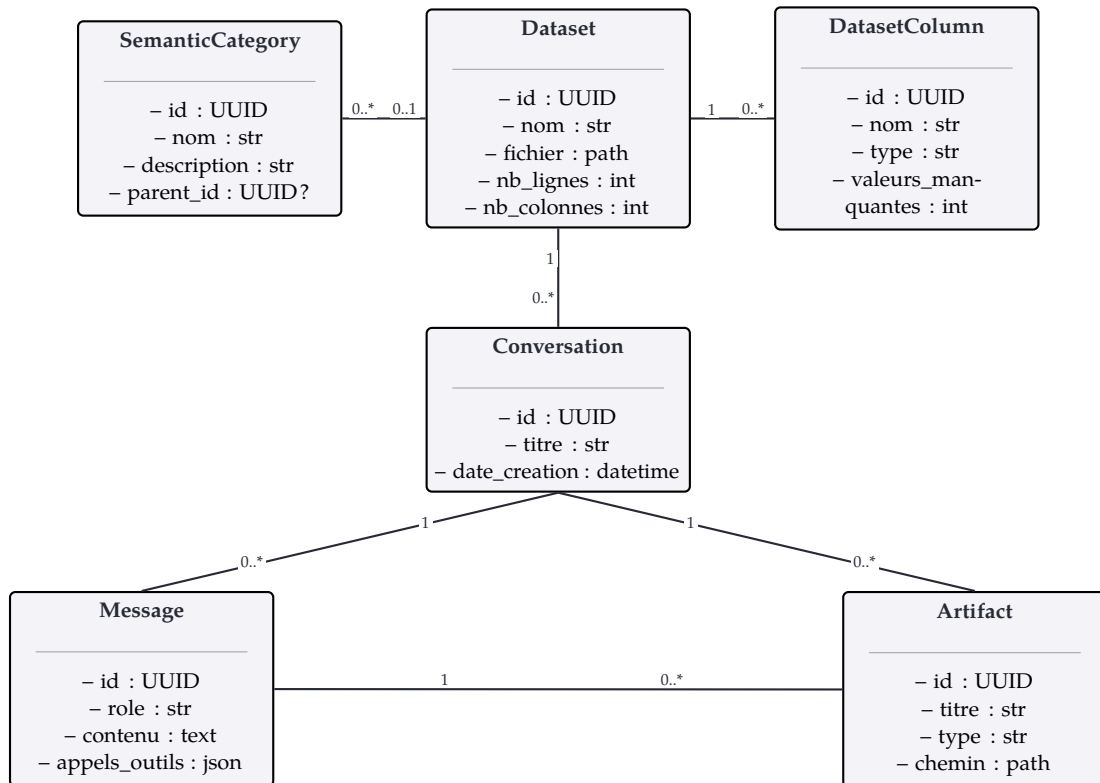


FIG. 4.4 : Diagramme de classes simplifié du modèle de données

Le modèle distingue les entités relatives au catalogue des données (SemanticCategory, Dataset, DataColumn) de celles relatives aux échanges conversationnels (Conversation, Message, Artifact). Un jeu de données est associé à une catégorie sémantique et décrit par ses colonnes profilées. Une conversation est rattachée à un jeu de données et regroupe les messages échangés ; les artefacts produits (graphiques, exports, rapports) sont rattachés à la conversation et au message qui les a générés.

4.4 Architecture logicielle

L'architecture logicielle de la plateforme est organisée en couches, chacune assurant une responsabilité précise. Cette organisation permet de maintenir une séparation claire des préoccupations et de faire évoluer chaque composant indépendamment.

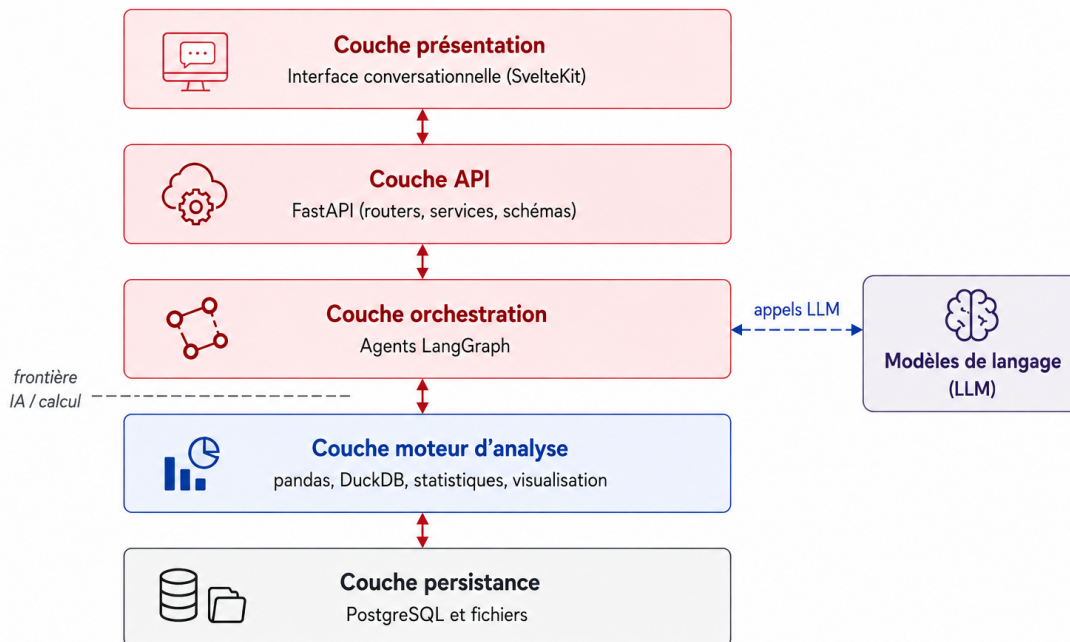


FIG. 4.5 : Architecture logicielle en couches et relations entre composants

La figure 4.5 présente l'architecture en couches et les relations entre les principaux composants. Chaque couche est décrite ci-dessous.

4.4.1 Couche présentation

La couche présentation correspond à l'interface utilisateur développée avec SvelteKit. Elle est responsable de l'affichage des données, de la gestion de l'interaction conversationnelle et de la restitution temps réel des événements diffusés par le backend. Elle intègre également le rendu interactif des graphiques produits par le moteur d'analyse, à partir des spécifications Plotly reçues au cours du streaming.

4.4.2 Couche API et services

La couche API, développée avec FastAPI, constitue le point d'entrée unique du backend. Elle expose des interfaces REST pour la gestion des jeux de données, des conversations, des artefacts et des catégories sémantiques, ainsi qu'un point de terminaison de streaming pour le traitement des requêtes conversationnelles. La logique métier est isolée dans une couche de services, ce qui permet de maintenir des routeurs légers et dédiés aux préoccupations HTTP.

4.4.3 Couche orchestration

La couche d'orchestration, basée sur LangGraph, constitue le cerveau du système. Elle coordonne le déroulement des analyses en pilotant les appels au modèle de langage et aux outils. Cette couche est décrite en détail dans la section consacrée à l'architecture agentique.

4.4.4 Couche moteur d'analyse

La couche moteur d'analyse regroupe les opérations computationnelles déterministes : chargement et profilage des données, calculs statistiques, agrégations, requêtes SQL et génération de visualisations. Elle est volontairement indépendante du modèle de langage, ce qui garantit la fiabilité et la reproductibilité des traitements.

4.4.5 Couche persistance

La couche persistance assure le stockage des données relationnelles (catalogue des jeux de données, conversations, messages, artefacts) dans PostgreSQL, ainsi que des fichiers bruts et des artefacts produits sur le système de fichiers. Le moteur DuckDB est utilisé pour exécuter des requêtes analytiques directement sur les fichiers, sans recourir à la base relationnelle pour les charges de calcul lourdes.

4.4.6 Couche modèles IA

Enfin, la couche des modèles de langage fournit les capacités de compréhension, de planification et de synthèse. Elle est consommée exclusivement par la couche d'orchestration, qui l'invoque via une interface de programmation standardisée. Cette isolation permet de remplacer ou de déployer localement le modèle sans impact sur le reste du système.

Architecture en couches

L'architecture logicielle repose sur une séparation stricte entre les couches de présentation, d'API, d'orchestration, de moteur d'analyse, de persistance et de modèles IA. Cette organisation garantit la modularité, la maintenabilité et l'évolutivité de la plateforme, tout en respectant le principe fondamental de séparation entre le pilotage par l'IA et le calcul déterministe.

4.5 Architecture agentique et moteur d'analyse

Cette section présente le cœur du système : l’architecture agentique qui orchestre le processus d’analyse, le cadre d’exécution (*harness*) qui encadre les appels au modèle de langage ainsi que le moteur d’analyse qui réalise les traitements. Elle montre comment une instruction formulée en langage naturel aboutit à une analyse réelle et vérifiable, en s’appuyant sur une séparation claire entre les responsabilités de l’IA et celles du calcul déterministe.

4.5.1 Architecture des agents

L’architecture agentique repose sur un agent spécialisé, l’**agent analyste de données** (*DataAnalystAgent*), construit avec LangGraph. Cet agent n’effectue pas lui-même les calculs : il constitue le **pilote** du processus, chargé de comprendre la requête, de décider des opérations à réaliser et d’interpréter les résultats. L’exécution effective des traitements est déléguée à un ensemble d’outils spécialisés.

L’architecture des agents s’articule autour de trois familles de composants :

- **L’orchestrateur** : composant central qui organise le déroulement d’une requête en coordonnant les appels au modèle de langage et aux outils. Il garantit l’enchaînement des phases et la cohérence de l’ensemble du traitement ;
- **Les nœuds spécialisés** : le planificateur, l’exécuteur, le synthétiseur et le module de récupération d’erreur, qui incarnent chacune des étapes du processus d’analyse ;
- **Le registre d’outils** : catalogue centralisé des capacités analytiques accessibles à l’agent (inspection et profilage, statistiques et agrégation, nettoyage, requêtes SQL, visualisation). Chaque outil expose une interface normalisée et exécute un traitement déterministe sur les données.

L’état de l’agent (*working memory*) constitue la mémoire de travail de l’exécution en cours. Il conserve l’ensemble des informations nécessaires au traitement : la requête originale, le jeu de données associé, le plan d’exécution, l’étape courante, les résultats intermédiaires, les artefacts générés et les éventuelles erreurs. Cet état est propagé d’un nœud à l’autre par le graphe d’orchestration et persisté via un mécanisme de *checkpoint*, ce qui permet de maintenir la continuité entre les tours de la conversation et de reprendre une exécution en cas de besoin.

Le registre d’outils répond au besoin d’**extensibilité** identifié au chapitre précédent : l’ajout d’une nouvelle capacité d’analyse se limite à l’enregistrement d’un nouvel outil, sans modifier la logique de l’orchestration. Cette organisation permet également de maîtriser précisément les opérations que l’agent est autorisé à déclencher, renforçant ainsi la sécurité et la traçabilité de la plateforme.

4.5.2 Orchestration avec LangGraph

L'orchestration du processus d'analyse est modélisée sous la forme d'un **graphe d'états** construit avec LangGraph. Ce choix permet de représenter explicitement le flux de contrôle, de persister l'état entre les étapes et de gérer les cycles et les conditions de manière robuste. Le graphe d'orchestration, illustré par la figure 4.6, organise le traitement d'une requête selon trois phases principales décrites ci-dessous.

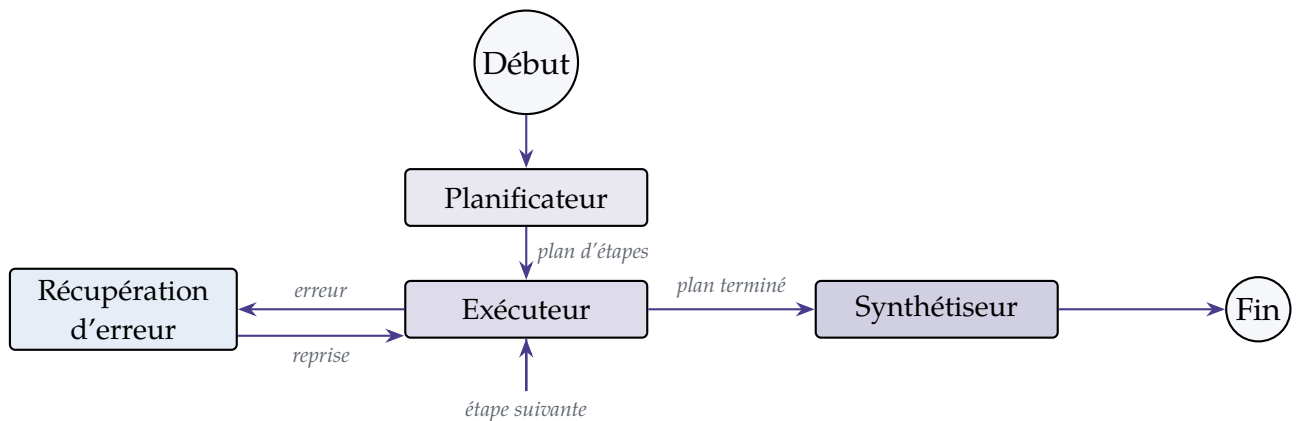


FIG. 4.6 : Graphe d'états de l'orchestrateur (planification, exécution, synthèse)

Phase de planification

La phase de planification a pour objectif de transformer la requête de l'utilisateur en un plan d'exécution structuré. Le modèle de langage analyse l'intention de l'utilisateur, le schéma du jeu de données concerné, et décompose la requête en une séquence d'étapes ordonnées. Chaque étape spécifie l'outil à utiliser et les paramètres de l'appel. Cette phase traduit le besoin de planification automatique identifié au chapitre précédent.

Phase d'exécution

La phase d'exécution traite chaque étape du plan en invoquant l'outil approprié au travers du registre d'outils. Le moteur d'analyse exécute alors le traitement de manière déterministe — requête SQL sur les données, calcul statistique, génération d'un graphique — et retourne le résultat à l'orchestrateur. Un mécanisme de tolérance aux pannes permet de retenter une étape en cas d'erreur, avec une correction automatique guidée par le modèle de langage, avant de poursuivre ou d'abandonner la séquence. Cette phase traduit les besoins d'analyse rigoureuse, de robustesse et de traçabilité.

Phase de synthèse

La phase de synthèse a pour objectif de transformer les résultats obtenus en une réponse compréhensible pour l'utilisateur. Le modèle de langage interprète les résultats, produit une synthèse rédigée en langage naturel et, le cas échéant, ordonne la génération de visualisations. Les étapes, les résultats intermédiaires et les artefacts produits sont conservés dans l'état de l'exécution, ce qui permet de retracer l'intégralité du raisonnement et de garantir la reproductibilité.

4.5.3 Gestion du contexte et harness LLM

L'utilisation d'un modèle de langage dans une application de production ne se limite pas à l'invocation d'un modèle : elle requiert un **cadre d'exécution** (*harness*) qui encadre chaque appel, construit le contexte et organise la mémoire. Ce cadre, illustré par la figure 4.7, s'inspire des pratiques des plateformes d'agents modernes et a été adapté aux spécificités de la plateforme d'analyse conversationnelle.

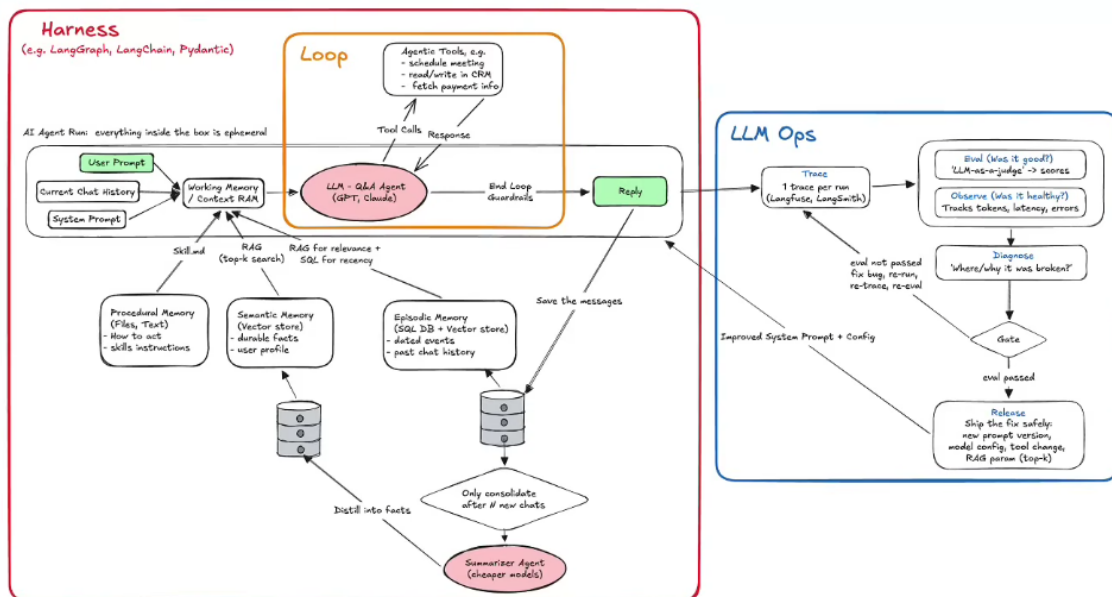


FIG. 4.7 : Cadre d'exécution (*harness*) de l'agent et supervision du cycle de vie (LLM Ops)

Pourquoi un harness : la gestion du contexte des LLM

Le recours à un cadre d'exécution dédié ne relève pas d'un choix cosmétique : il répond à une limitation fondamentale des grands modèles de langage. Un LLM est intrinsèquement **sans état** (*stateless*) : chaque appel est indépendant du précédent et le modèle ne conserve aucune mémoire interne des échanges antérieurs. En l'absence de dispositif externe, il perd donc le fil de la conversation dès qu'un nouveau tour

commence. Par ailleurs, la fenêtre de contexte d'un modèle est **finie** : elle ne peut contenir qu'un nombre limité de tokens, ce qui borne la quantité d'historique, de données et de résultats qu'il est possible de lui transmettre.

Une intégration naïve du modèle, sans harness, conduirait ainsi à plusieurs échecs :

- **Perte de la continuité conversationnelle** : l'utilisateur serait contraint de reformuler son contexte à chaque tour, le modèle ne gardant aucune trace de la discussion précédente ;
- **Absence d'accès aux données** : le modèle ne peut pas exécuter de requête SQL ni de calcul statistique ; il se contenterait de simuler une analyse sans garantie de véracité ;
- **Risque d'hallucination** : sans ancrage dans le schéma réel du jeu de données, le modèle peut inventer des colonnes, des formats ou des résultats ;
- **Absence de traçabilité** : rien ne permettrait de savoir quelles opérations ont réellement été effectuées, ni de reproduire un résultat.

Plusieurs approches alternatives pourraient être envisagées pour répondre à ces limites. Le tableau 4.3 les compare et justifie le choix d'un harness.

TAB. 4.3 : Approches de gestion du contexte d'un LLM et limites

Approche	Principe	Limites
Appel sans mémoire	Chaque requête est traitée isolément, sans historique	Aucune continuité ; reformulation complète à chaque tour ; aucun accès aux données.
Historique complet	Envoi de l'intégralité de la conversation à chaque tour	Dépassement de la fenêtre de contexte ; coût et latence croissants ; résultats dilués.
Fine-tuning du modèle	Adaptation des poids du modèle à un domaine	Coûteux et complexe ; ne résout ni l'accès aux outils ni la mémoire par conversation.
RAG seul	Enrichissement du prompt par des documents pertinents	Améliore l'ancrage, mais pas la continuité de la conversation ni le calcul déterministe.
Harness proposé	Reconstruction du contexte, mémoire structurée, outils	Assure continuité, ancrage, traçabilité et maîtrise des coûts.

Le harness proposé surmonte ces difficultés en combinant trois mécanismes complémentaires. La **reconstruction systématique du contexte** rend l'exécution reproductible malgré le caractère sans état du modèle; la **mémoire structurée** (procédurale, sémantique, épisodique) préserve la continuité de la conversation et les connaissances durables; l'**ancrage dans les données** via le RAG et les outils déterministes garantit que le modèle raisonne sur des faits réels et non sur des inférences approximatives. Ce choix répond directement aux besoins de fiabilité, de transparence et de reproductibilité identifiés au chapitre précédent, tout en maîtrisant les coûts grâce à la consolidation du contexte.

Principe du harness

Le harness repose sur un principe structurant : **chaque exécution de l'agent est éphémère**. Les informations manipulées au cours d'un tour — prompt, contexte, mémoire de travail, résultats d'outils — sont reconstruites à chaque invocation à partir de sources durables situées à l'extérieur de la boîte d'exécution. Cette conception garantit que l'agent ne dépend d'aucun état résiduel implicite et que chaque exécution peut être reproduite et tracée.

Construction du contexte

À chaque exécution, le harness construit le **contexte** transmis au modèle de langage à partir de plusieurs sources complémentaires :

- la **requête de l'utilisateur** formulée en langage naturel;
- le **jeu de données courant**, représenté par son schéma et son profilage (l'équivalent du fichier courant dans un environnement de développement);
- le **prompt système**, qui définit le rôle de l'agent, ses règles de comportement et les contraintes à respecter;
- la **mémoire de travail** (état de l'agent) et le résumé consolidé de la conversation.

Afin de guider le modèle sans dépendre de sa seule connaissance générale, la plateforme exploite une forme de **RAG** (*Retrieval-Augmented Generation*) sur les métadonnées des données : le schéma des colonnes, les types détectés et les catégories sémantiques sont récupérés et injectés dans le prompt. Le modèle peut ainsi formuler des requêtes valides et adaptées au jeu de données considéré.

Gestion de la mémoire

La gestion de la mémoire distingue trois types de mémoire complémentaires, conformément au modèle du harness :

- la **mémoire procédurale** : fichiers et textes qui définissent *comment* l'agent doit agir (prompt système, instructions, configuration de l'agent) ;
- la **mémoire sémantique** : faits durables relatifs aux données et aux utilisateurs (catalogue des jeux de données, catégories, profils), conservés dans la base de données ;
- la **mémoire épisodique** : historique des conversations passées, persisté dans PostgreSQL et exploité par le *checkpoint* de LangGraph (en mémoire ou sur PostgreSQL) pour assurer la continuité des échanges.

Chaque tour est **sauvegardé** en base de données (messages, appels d'outils, artefacts), ce qui constitue la source de vérité de la mémoire épisodique. Afin de maîtriser la taille du contexte, un **agent de résumé** consolide périodiquement l'historique : après un certain nombre de tours, les échanges antérieurs sont condensés en un résumé courant, injecté dans le contexte des tours suivants. Cette consolidation est réalisée par un appel dédié au modèle de langage, ce qui permet de conserver l'essentiel de la conversation tout en limitant la consommation de contexte.

Supervision du cycle de vie (LLM Ops)

La mise en production d'un agent requiert enfin une supervision continue de son comportement. La plateforme intègre les principes du *LLM Ops*, qui structurent le cycle de vie d'un agent :

- **Tracer** : chaque exécution produit une trace complète (configuration, étapes, appels d'outils), permettant de reconstituer le déroulement d'une requête ;
- **Observer** : les réponses sont évaluées afin de mesurer leur qualité, notamment par des critères objectifs (*LLM-as-a-judge*) ;
- **Diagnostiquer** : les traces et les métriques (tokens, latence, erreurs) permettent d'identifier l'origine d'un échec ;
- **Valider** : toute évolution (prompt, modèle, configuration) n'est déployée qu'après validation ;
- **Livrer** : les améliorations sont mises en production de manière progressive et réversible.

Ces mécanismes, combinés à la traçabilité des étapes et des outils décrite précédemment, garantissent que le comportement de l'agent reste maîtrisé et évolutif dans un contexte hospitalier exigeant.

4.5.4 Moteur d'analyse déterministe

Le moteur d'analyse constitue la partie déterministe du système. Il est chargé de réaliser l'ensemble des traitements computationnels sur les données réelles, à l'opposé du modèle de langage qui se limite à la compréhension, la planification et l'interprétation. Cette séparation garantit qu'un résultat présenté à l'utilisateur a bien été calculé par un algorithme vérifiable, et non généré de manière approximative par le modèle.

Le moteur d'analyse prend en charge :

- le **chargement** des jeux de données aux formats CSV, XLSX, Parquet ou JSON, avec détection automatique de l'encodage et du séparateur ;
- le **profilage** des colonnes : types, valeurs manquantes, statistiques descriptives (min, max, moyenne, écart-type, quantiles) ;
- les **calculs statistiques** : statistiques descriptives, corrélations, agrégations et opérations de transformation ;
- les **requêtes SQL** exécutées directement sur les fichiers via DuckDB, avec des limites de sécurité sur le volume des résultats ;
- la **génération de visualisations** : histogrammes, diagrammes de dispersion, matrices de corrélation et autres graphiques, produits sous forme de spécifications Plotly interactives ou de rendus statiques.

Le déroulement d'une requête met en évidence le rôle complémentaire des deux composants. À partir d'une instruction en langage naturel, le modèle de langage produit une **compréhension** de l'intention, puis une **planification** en étapes. Le moteur d'analyse assure l'**exécution** réelle de chaque étape sur les données. Les **résultats** obtenus sont ensuite transmis au modèle de langage pour leur **interprétation**, puis restitués à l'utilisateur. Ainsi, le modèle de langage oriente le travail mais ne calcule jamais directement : il reçoit des résultats vérifiables qu'il se contente d'expliquer.

Cette architecture garantit plusieurs propriétés essentielles. La **fiabilité** des traitements, puisque les calculs sont réalisés par des algorithmes éprouvés ; la **transparence**, puisque chaque étape est tracée et chaque résultat associé à l'outil qui l'a produit ; la **reproductibilité**, puisque le même plan d'exécution appliqué aux mêmes données produit les mêmes résultats.

Séparation IA et calcul

Le point le plus structurant de l'architecture réside dans la séparation entre l'orchestration pilotée par l'IA et l'exécution déterministe du moteur d'analyse. Le LLM oriente et orchestre le travail, tandis que le moteur effectue les traitements réels de manière contrôlée et vérifiable. Cette distinction permet de concilier la flexibilité du langage naturel avec la rigueur et la fiabilité des méthodes statistiques.

4.6 Communication et gestion des données

Cette section présente les mécanismes de communication entre les différents composants de la plateforme ainsi que la gestion des données, de leur ingestion à la production des artefacts.

4.6.1 Échanges entre composants

Les échanges entre le frontend et le backend s'effectuent selon deux modes complémentaires. Le premier repose sur des **interfaces REST** pour les opérations de gestion : importation et profilage des jeux de données, gestion des conversations, récupération des artefacts et gestion des catégories sémantiques. Le second repose sur le **streaming** (*Server-Sent Events*) pour le traitement des requêtes conversationnelles : le frontend ouvre un flux continu sur lequel le backend diffuse progressivement les événements du traitement.

Le backend constitue l'intermédiaire entre le frontend et l'orchestration agentique. Il reçoit la requête de l'utilisateur, initialise l'exécution, collecte les événements produits par l'orchestrateur et les convertit en messages de streaming. L'orchestrateur, quant à lui, communique avec le modèle de langage et avec le moteur d'analyse au travers du registre d'outils.

4.6.2 Streaming et événements

Le streaming constitue un élément clé de l'expérience utilisateur. Il permet de diffuser en temps réel l'ensemble des événements du traitement :

- `step_start` : annonce le début d'une étape du plan ;
- `tool_call` : indique l'invocation d'un outil d'analyse ;
- `token` : diffusion progressive du texte généré par le modèle ;
- `artifact` : notification de la production d'un artefact (graphique, export) ;
- `error` : signalement d'une erreur éventuelle ;
- `done` : fin du traitement.

Ce mécanisme répond au besoin de réactivité et de transparence identifié au chapitre précédent : l'utilisateur observe en direct le déroulement de l'analyse, les étapes franchies et la construction progressive de la réponse.

4.6.3 Persistance et gestion des artefacts

La gestion des données s’articule autour d’une architecture duale. La **base de données relationnelle** (PostgreSQL) conserve les métadonnées du catalogue : jeux de données, colonnes profilées, catégories sémantiques, conversations, messages et artefacts. Les **fichiers bruts** des jeux de données et les **artefacts produits** (spécifications de graphiques, exports, rapports) sont stockés sur le système de fichiers, leurs références étant conservées en base de données.

Pour les opérations analytiques, le moteur **DuckDB** exécute les requêtes directement sur les fichiers, sans transiter par la base relationnelle. Cette approche permet de traiter efficacement de grands volumes de données en mémoire tout en préservant la base de données pour la gestion des métadonnées.

Les artefacts produits au cours des analyses — graphiques interactifs, tableaux exportés, rapports — sont enregistrés avec leurs métadonnées (titre, type, chemin) et associés à la conversation et au message qui les ont générés. Le frontend les récupère via des interfaces dédiées et les affiche directement dans l’interface conversationnelle. Cette organisation garantit la conservation des résultats et leur réutilisation ultérieure, conformément aux besoins de persistance et de traçabilité.

4.7 Conclusion

Ce chapitre a présenté la conception et l’architecture de la plateforme d’analyse conversationnelle de données, en explicitant les choix d’ingénierie qui traduisent les besoins identifiés au chapitre précédent en une solution technique cohérente.

La conception générale a justifié les choix technologiques et défini une architecture globale articulée autour de trois zones : l’orchestration par l’IA, le traitement déterministe des données et la persistance. La modélisation UML et l’architecture logicielle en couches ont précisé les fonctionnalités et les responsabilités de chaque composant.

L’architecture agentique et le moteur d’analyse constituent le cœur du système. L’agent, construit avec LangGraph, orchestre le processus en trois phases — planification, exécution et synthèse — tandis que le moteur déterministe réalise les traitements computationnels, garantissant fiabilité et reproductibilité.

Le fil conducteur du chapitre repose sur une distinction structurante : **l’IA pilote et orchestre l’analyse, tandis que les calculs critiques sont exécutés par un moteur déterministe**. Cette séparation, motivée par les besoins de fiabilité et de transparence, concilie la flexibilité du langage naturel avec la rigueur des méthodes statistiques.

Ces choix fondent la réalisation pratique de la plateforme, dont la mise en œuvre est détaillée dans le chapitre suivant.

Chapitre 5

Réalisation de la plateforme

5.1 Introduction

Ce chapitre présente la réalisation effective de la plateforme, c'est-à-dire la manière dont la solution conçue au chapitre précédent a été implémentée, intégrée et préparée pour le déploiement. Il répond à la question centrale : **comment avons-nous effectivement développé la solution ?**

Le chapitre s'organise comme suit. La notion d'**intelligence des données** appliquée au projet est d'abord introduite, afin de situer la plateforme dans la stratégie d'exploitation des données du GST-TTA. L'environnement de réalisation et l'organisation du travail sont ensuite présentés. Les principales réalisations sont ensuite détaillées : le backend et le streaming temps réel, l'orchestration agentique et le mécanisme d'**evidence**, le moteur d'analyse, puis l'interface et son **internationalisation** (i18n). Enfin, l'intégration de la solution et les aspects de déploiement sont abordés avant la conclusion.

Conformément aux exigences d'un rapport académique, seuls les extraits de code indispensables sont présentés, afin d'illustrer les mécanismes les plus structurants — streaming, evidence et internationalisation — sans transformer le chapitre en documentation technique. Les choix d'architecture ayant déjà été justifiés au chapitre précédent, ce chapitre se concentre sur la mise en œuvre concrète et ses résultats.

5.2 L'intelligence des données appliquée au projet

L'**intelligence des données** (*data intelligence*) désigne la capacité d'une organisation à transformer ses données brutes en connaissances exploitables, en combinant la collecte et la préparation des données, l'analyse statistique, la visualisation et l'intelligence artificielle. Dans le secteur de la santé, elle constitue un levier essentiel pour le pilotage des établissements, l'amélioration de la qualité des soins et l'aide à la décision.

Dans le cadre de ce projet, la plateforme opérationnalise cette démarche pour le compte du GST-TTA. Elle ne se limite pas à l'exécution de requêtes : elle accompagne

l'utilisateur sur l'ensemble du cycle d'exploitation des données. Le tableau 5.1 met en correspondance les dimensions de l'intelligence des données avec les composants développés de la plateforme.

TAB. 5.1 : L'intelligence des données et sa mise en œuvre dans la plateforme

Dimension	Apport	Composants
Compréhension des données	Profilage automatique : types, valeurs manquantes, statistiques descriptives	Moteur d'analyse (profilage), interface jeux de données
Organisation sémantique	Catégorisation hiérarchique des jeux de données pour la recherche et le contexte	Catégories sémantiques
Exploitation statistique	Statistiques descriptives, corrélations, agrégations, détection d'anomalies	Moteur d'analyse et outils (pandas, DuckDB)
Accès conversationnel	Interrogation en langage naturel, sans compétences techniques	Orchestration agentique, interface de chat
Génération d'insights	Synthèses fondées sur des résultats calculés et vérifiables	Synthétiseur et mécanisme d' <i>evidence</i>
Diffusion des résultats	Graphiques interactifs, rapports, streaming temps réel	Frontend et artefacts (Plotly, exports)

Ce positionnement répond directement aux besoins identifiés au chapitre consacré à l'analyse des besoins : réduire la barrière technique, garantir la confidentialité et fournir des analyses fiables et reproductibles. L'intelligence des données constitue ainsi le fil conducteur de la réalisation : chaque composant développé contribue à l'une de ses dimensions, depuis la compréhension des données jusqu'à la diffusion des résultats.

Positionnement du projet

La plateforme constitue un outil d'intelligence des données pour le GST-TTA : elle transforme des données hospitalières brutes en connaissances exploitables, accessibles en langage naturel et fondées sur des résultats calculés de manière vérifiable.

5.3 Environnement et organisation de la réalisation

5.3.1 Environnement technique

La réalisation s'appuie sur un monorepo Python géré par uv, organisé en applications et paquets partagés. Cette organisation permet de développer, tester et déployer les différents composants de manière cohérente tout en conservant une séparation claire des responsabilités, conformément au principe de modularité défini au chapitre précédent.

Listing 5.1 : Organisation du monorepo

```
CHU-Platform/
|-- apps/
|   |-- api/          # API FastAPI (routers, services, modèles)
|   |-- ui/           # Mockup / maquette de l'interface utilisateur
|   |-- web/          # Application SvelteKit (interface)
|-- packages/
|   |-- agents/       # Orchestration agentique (LangGraph)
|   |-- analysis/     # Moteur d'analyse (statistiques, profilage)
|   |-- tools/        # Outils accessibles aux agents
|-- nginx/           # Reverse proxy
`-- docs/            # Documentation du projet
```

5.3.2 Environnement et dépendances

Les dépendances du projet sont gérées de manière reproductible : le workspace Python est décrit dans un fichier `pyproject.toml` à la racine, et les versions exactes des paquets sont épinglées dans un fichier de verrouillage (`uv.lock`) résolu par uv. Côté frontend, les dépendances Node sont gérées par bun ou npm. Cette organisation garantit que l'ensemble de l'équipe et les environnements de déploiement utilisent les mêmes versions de dépendances, ce qui contribue à la reproductibilité de la réalisation.

5.3.3 Organisation du travail

Le développement a été mené selon la démarche Agile présentée au chapitre consacré au contexte général : des sprints itératifs ont permis de faire évoluer la plateforme depuis les fondations techniques jusqu'à la spécialisation santé, chaque itération livrant un incrément fonctionnel validé. Les outils de versionnement (Git), de gestion des dépendances (uv) et de conteneurisation (Docker) ont été utilisés tout au long du processus, garantissant la traçabilité des évolutions et la reproductibilité de l'environnement.

5.3.4 Gestion de la configuration

La configuration de l'application est centralisée et externalisée : les paramètres sont définis par variables d'environnement (fichier `.env`) et chargés au démarrage

via Pydantic Settings. Cette approche permet de configurer le modèle de langage (OPENAI_MODEL), la chaîne de connexion à la base de données (DATABASE_URL), les répertoires de stockage des fichiers (FILES_DIR, DATASETS_DIR, EXPORTS_DIR) ou encore le niveau de journalisation (LOG_LEVEL), sans modification du code. L'accès au modèle de langage est également configurable : l'adresse de l'endpoint (OPENAI_BASE_URL) et le modèle (AGENT_MODEL) peuvent pointer vers l'infrastructure locale de modèles ou, en phase de test, vers un fournisseur externe. Elle facilite également le déploiement de la plateforme sur différentes infrastructures, conformément au besoin de portabilité identifié au chapitre consacré à l'analyse des besoins.

5.4 Réalisation du backend et du streaming

5.4.1 L'API FastAPI

Le backend expose une API REST structurée autour de routeurs dédiés, dont les principaux points de terminaison sont synthétisés dans le tableau 5.2. La logique métier est isolée dans des services, les échanges sont validés par des schémas Pydantic, et l'accès aux données s'appuie sur SQLAlchemy 2.0 en mode asynchrone avec PostgreSQL. Les évolutions du schéma de la base sont gérées par des migrations Alembic. Enfin, l'API est auto-documentée : la documentation OpenAPI (interface Swagger) est générée automatiquement et accessible à l'adresse /docs, ce qui facilite l'exploration et l'intégration de la plateforme.

TAB. 5.2 : Principaux points de terminaison de l'API REST

Ressource	Points de terminaison	Rôle
Jeux de données	POST /api/datasets/upload, GET /api/datasets/, GET /api/datasets/{id}, GET .../preview, PATCH, DELETE	Importer, lister, consulter, prévisualiser et supprimer les jeux de données.
Conversations	POST/GET /api/conversations/, GET/DELETE /api/conversations/{id}	Gérer les fils de discussion et leur historique.
Chat (streaming)	POST /api/chat/stream	Exécuter une requête conversationnelle et diffuser les événements en temps réel (SSE).
Artefacts	GET /api/artifacts/{id}, GET /api/artifacts/conversation/{id}	Récupérer les artefacts produits (graphiques, exports).
Catégories sémantiques	GET/POST /api/semantic-categories/, PATCH/DELETE .../{id}	Gérer la catégorisation des jeux de données.

5.4.2 Les services applicatifs

La logique métier du backend est organisée en services, séparés des routeurs qui ne gèrent que les préoccupations HTTP. Trois services principaux structurent cette couche : le service des jeux de données, responsable de la validation des fichiers, de leur enregistrement et de leur profilage ; le service des conversations, qui gère le cycle de vie des fils de discussion et leur historique ; et le service de chat, qui assure l'intermédiaire entre la requête HTTP et l'orchestrateur et convertit les événements produits par l'agent en messages de streaming.

5.4.3 La gestion des conversations

Une conversation constitue un fil de discussion associé à un jeu de données précis. Son cycle de vie est géré par le service dédié : création d'un nouveau fil, liste des conversations existantes, récupération d'une conversation avec ses messages et suppression. Chaque échange — message de l'utilisateur et réponse de l'assistant — est persisté en base de données avec les traces des appels d'outils, ce qui alimente la mémoire épisodique décrite précédemment et permet de reprendre une discussion à tout moment.

5.4.4 La chaîne d’ingestion et de profilage des données

L’importation d’un jeu de données constitue la première étape de l’exploitation des données. Lors de l’upload d’un fichier, le service dédié enregistre le fichier brut dans un répertoire de stockage (`files/datasets/`) sous un identifiant unique, puis lance en arrière-plan une tâche de traitement : chargement de la table avec détection automatique du format (CSV, TSV, XLSX, XLS, Parquet, JSON, Feather) et des encodages, puis **profilage** des colonnes. Le profilage produit, pour chaque colonne, son type, le nombre de valeurs manquantes, le nombre de valeurs uniques et un exemple de valeur, ainsi que la forme globale de la table (nombre de lignes et de colonnes, volume mémoire). Ces métadonnées sont ensuite persistées en base de données et mises à disposition de l’interface (schéma et statistiques) ainsi que du moteur d’analyse.

Le stockage des fichiers suit une organisation simple et centralisée : les jeux de données bruts sont conservés dans `files/datasets/`, tandis que les artefacts produits au cours des analyses — spécifications de graphiques Plotly, exports CSV, rendus PNG — sont enregistrés dans `files/exports/`. Chaque artefact est référencé en base de données (identifiant, type, chemin, conversation associée), ce qui permet de le retrouver et de le diffuser via les interfaces dédiées. Cette organisation, dont le principe a été justifié au chapitre précédent, est ici mise en œuvre par les services de persistance et les routeurs d’artefacts.

5.4.5 Le streaming temps réel

Le streaming constitue un élément central de l’expérience utilisateur. Il repose sur le protocole *Server-Sent Events* (SSE) : le backend diffuse progressivement les événements du traitement — plan, étapes, appels d’outils, tokens, artefacts, erreurs et fin. L’orchestrateur émet ces événements via un mécanisme unifié, comme l’illustre l’extrait suivant.

Listing 5.2 : Émission d’un événement de streaming

```
async def _emit(self, event_type: str, data: str | dict):
    """Émet un événement consommé par le flux SSE."""
    await adispatch_custom_event(
        "orchestrator_event", {"type": event_type, "data": data}
    )
```

Ce mécanisme garantit que l’utilisateur observe en direct le déroulement de l’analyse — étapes franchies, outils invoqués, construction progressive de la réponse — conformément au besoin de réactivité et de transparence identifié au chapitre consacré à l’analyse des besoins.

5.4.6 Sécurité et gestion des erreurs

La réalisation intègre plusieurs mesures de sécurité et un traitement rigoureux des erreurs, indispensables dans un contexte hospitalier. L'exécution des requêtes générées par le modèle de langage est **encadrée** : le moteur DuckDB fonctionne en mode lecture seule, ce qui interdit toute opération destructive sur les fichiers, et le volume des résultats est limité afin d'éviter une consommation excessive de mémoire. Les formats des fichiers importés sont contrôlés et les échanges validés par des schémas.

La gestion des erreurs distingue trois niveaux complémentaires. Au niveau de l'API, les erreurs sont retournées sous un format JSON homogène avec des codes HTTP explicites (400, 404, 422, 500). Au niveau du chat, une erreur survenant au cours du streaming est transmise au frontend sous la forme d'un événement `error`, sans interrompre la connexion : l'utilisateur est informé et peut reformuler sa demande. Enfin, au niveau de l'agent, l'échec d'un appel d'outil (par exemple une requête SQL invalide) est renvoyé dans la mémoire de travail de l'exécution : le planificateur analyse l'erreur et corrige la requête lors de l'itération suivante, conformément au mécanisme de reprise présenté au chapitre précédent.

Sécurité par conception

Les calculs exécutés sur les données sont strictement contrôlés : le moteur interdit les opérations destructives, borne les volumes traités et valide les entrées, tandis que la traçabilité des étapes et l'evidence garantissent la fiabilité des résultats.

5.5 Réalisation de l'orchestration agentique et de l'evidence

5.5.1 L'orchestration avec LangGraph

L'orchestration a été implémentée sous la forme d'un graphe d'états LangGraph (figure 4.6), comprenant le planificateur, l'exécuteur, le module de récupération d'erreur et le synthétiseur. Le graphe est compilé avec un *checkpoint* qui assure la persistance de l'état entre les étapes et la continuité entre les tours de conversation. Selon l'environnement, la mémoire est conservée en mémoire ou sur PostgreSQL.

5.5.2 Le workflow d'analyse

Le déroulement d'une requête suit le workflow planification-exécution-synthèse présenté au chapitre précédent (figure 4.3). Concrètement, l'orchestrateur commence

par demander au modèle de générer un plan d'étapes ordonné ; chaque étape est ensuite exécutée par l'outil adapté, dont le résultat alimente l'evidence ; en cas d'échec, une reprise est tentée avec correction ; enfin, la synthèse produit la réponse finale à partir de l'ensemble des preuves accumulées. Ce workflow garantit qu'une question en langage naturel se traduit systématiquement en une séquence d'opérations tracées et vérifiables.

5.5.3 L'ingénierie des prompts de l'agent

Le comportement de l'agent est défini par un ensemble de prompts système qui constituent sa **mémoire procédurale** : ils encodent les règles de bonnes pratiques — ne jamais inventer de statistique, fonder chaque conclusion sur des résultats calculés, suivre un cycle de vie de visualisation — et sont versionnés avec le code source de l'agent.

Le prompt système général de l'analyste fixe le workflow complet de l'analyse et les règles d'intégrité statistique, dont les principes fondamentaux sont reproduits ci-dessous.

Listing 5.3 : Prompt système de l'analyste (extrait)

```
## Core Principles
- Never guess.
- Never fabricate statistics, values, trends, or conclusions.
- Every conclusion must be supported by tool output.
- If the available data is insufficient, explicitly say so.

## Analysis Workflow
1. Inspect the dataset.
2. Assess data quality.
3. Clean or prepare the data if necessary.
4. Select appropriate statistical methods.
5. Perform the requested analysis.
6. Generate visualizations when useful.
7. Verify important conclusions.
8. Present findings.
```

En complément, trois prompts spécialisés correspondent aux trois phases du workflow : la planification, l'exécution et la synthèse.

Le prompt de planification (`planner_prompt`) demande au modèle de décomposer la requête utilisateur en une liste ordonnée d'étapes actionnables, en tirant parti du profilage pré-calculé lorsqu'il est fourni, et de retourner un plan au format JSON (extrait ci-dessous).

Listing 5.4 : Prompt de planification (extrait)

```
You are an execution planner for a data analysis agent.
Your job is to break down a user's request into a clear,
```

```
ordered list of execution steps.

## Rules
1. Produce exactly the steps needed - no more, no less.
2. Each step must be actionable.
3. Include a final "Synthesize findings" step.
4. Visualization is evidence, NOT a standalone step. Set
   needs_visualization: true only if a chart adds insight.

## Output Format
Return ONLY a JSON object:
{
  "plan_title": "Short title",
  "steps": [
    {"id": 1, "title": "...", "description": "...",
     "tool_hint": "...", "needs_visualization": false,
     "visualization_rationale": ""}
  ]
}
```

Le prompt d'exécution (step_prompt) oriente le modèle sur une étape précise du plan : il limite le périmètre de la tâche, impose l'usage séquentiel des outils et rappelle le cycle de vie des visualisations — calculer avant de générer, fournir une interprétation, référencer le graphique.

Listing 5.5 : Prompt d'exécution (extrait)

```
You are an expert data analyst executing a specific step in
an analysis plan.

## Current Task
{step_title}: {step_description}

## Rules
- Use the available tools to gather evidence for this step.
- VERY IMPORTANT: Call tools sequentially.
- Be thorough but focused.
- Do NOT make a plan or list next steps - just execute this step.

## Chart Lifecycle (when visualization is required)
1. COMPUTE FIRST - run the relevant statistics tool.
2. DECIDE - would a chart add insight the numbers cannot convey?
3. INSIGHT FIRST - form a 1-2 sentence interpretation.
4. GENERATE - call generate_chart with the insight.
5. REFERENCE - mention the chart by title in your narrative.
```

Enfin, le prompt de synthèse (synthesis_prompt) reçoit le **manifeste d'evidence** accumulé pendant l'exécution et impose au modèle de rédiger la réponse finale en s'appuyant exclusivement sur ces preuves, sans appeler de nouveaux outils et en intégrant les graphiques au fil du texte, comme des figures dans un article scientifique.

Listing 5.6 : Prompt de synthèse (extrait)

```
You are an expert data analyst writing a final analytical report.

## Evidence Manifest
All analysis steps have been completed. The evidence below contains
statistics, findings, and chart references gathered step by step.

## Report Structure
Write a professional, research-paper-style report. For each section:
1. Open with a paragraph describing the findings (cite numbers).
2. Embed the chart's Markdown exactly where it is most relevant.
3. Provide the interpretation and business impact.

## Hard Rules
- Every conclusion MUST be traceable to evidence above.
- Never fabricate statistics or trends not present in the evidence.
- Do NOT call any more tools - just write the final report.
```

5.5.4 Les outils de l'agent analyste de données

L'exécution des étapes du plan repose sur un ensemble d'outils spécialisés, implémentés de manière autonome dans le paquet `tools`, indépendamment de l'orchestration, puis exposés à l'agent au travers d'un registre centralisé. Cette conception permet de développer et de tester chaque capacité d'analyse de manière isolée, puis de la rendre disponible à l'agent par simple enregistrement, conformément au principe d'extensibilité défini au chapitre précédent.

Le tableau 5.3 recense les familles d'outils implémentées pour l'agent analyste de données.

TAB. 5.3 : Les outils de l'agent analyste de données

Famille	Outils	Rôle
Inspection et profilage	DescribeDataset, DatasetSummary, DatasetHead, DatasetShape, ListColumns, ColumnInfo	Comprendre la structure, les types et la qualité d'un jeu de données.
Statistiques et agrégation	Mean, Median, Min, Max, Std, Quantiles, Correlation, Aggregate, Filter, Sort, OutlierDetection	Calculer des indicateurs, croiser les variables et détecter les anomalies.
Nettoyage	MissingValues, Duplicates, DropColumns	Préparer et fiabiliser les données avant analyse.
Requêtes SQL (DuckDB)	QuerySQL, DuckDBAggregate, DuckDBFilter, DuckDBWindowStats	Interroger directement les fichiers via SQL (DuckDB).
Visualisation	GenerateChart, CorrelationHeatmap	Produire des preuves visuelles au format Plotly.
Planification	CreateBlueprint	Structurer le déroulement d'une analyse en étapes.

Chaque outil expose une interface normalisée — nom, description et schéma des paramètres — et est enregistré dans le registre au démarrage de l'application. Au cours d'une exécution, l'exécuteur sélectionne l'outil adapté à chaque étape du plan et reçoit un résultat structuré (tableau, statistique, spécification de graphique). Ce résultat alimente directement l'evidence de l'étape, garantissant que les conclusions reposent sur des traitements réellement exécutés.

5.5.5 Implémentation du moteur d'analyse

Le moteur d'analyse (paquet `analysis`) regroupe les traitements déterministes qui sous-tendent les outils de l'agent. Quatre domaines structurent sa réalisation.

Préparation et exploration des données

La préparation des données couvre le chargement multi-formats avec détection des encodages et des séparateurs, ainsi que le profilage des colonnes (types, valeurs manquantes, valeurs uniques, exemples). Les outils d'inspection permettent d'explorer un jeu de données : aperçu des premières lignes, forme de la table, résumé descriptif et liste des colonnes.

Analyses statistiques

Le moteur statistique fournit les calculs descriptifs (moyenne, médiane, écart-type, quantiles), les corrélations entre variables, les agrégations et la détection d'anomalies. Ces opérations sont implémentées à l'aide de pandas et numpy, puis exposées à l'agent sous forme d'outils normalisés.

Génération des visualisations

Les visualisations sont produites sous forme de spécifications Plotly interactives (histogrammes, diagrammes de dispersion, matrices de corrélation) ou de rendus Matplotlib statiques. Les spécifications générées sont enregistrées comme artefacts et rendues côté interface.

Génération des artefacts et rapports

Enfin, le moteur assure la production des artefacts : exports CSV des données traitées, spécifications de graphiques et synthèses. Chaque artefact est accompagné d'un résumé d'evidence associant le résultat à l'opération qui l'a produit, ce qui alimente le manifeste d'evidence de la réponse finale.

5.5.6 Le cadre d'exécution (*harness*) en pratique

Le cadre d'exécution décrit au chapitre précédent (figure 4.7) est mis en œuvre par l'orchestrateur à travers quatre mécanismes complémentaires : la conservation du contexte, sa construction pour le modèle, la gestion de l'historique et des états, et le contrôle des interactions avec le modèle.

Gestion et conservation du contexte

Le contexte durable de la plateforme repose sur les trois mémoires présentées au chapitre précédent. La mémoire épisodique — l'historique des échanges — est alimentée par la persistance des messages, des appels d'outils et des artefacts en base de données. La mémoire sémantique est portée par le catalogue des jeux de données et des catégories. La mémoire procédurale est constituée des prompts et de la configuration de l'agent. Ces sources sont conservées hors de la boîte d'exécution et reconstruites à chaque tour, conformément au principe d'exécution éphémère.

Construction du contexte pour le modèle

À chaque exécution, l'orchestrateur assemble le contexte transmis au modèle de langage : la requête de l'utilisateur, le schéma et le profilage du jeu de données courant, le prompt système définissant le rôle et les règles de l'agent, ainsi que la mémoire de

travail et le résumé consolidé de la conversation. La langue de l'interface est également intégrée à la formulation des prompts, afin que l'agent réponde dans la langue de l'utilisateur.

Gestion de l'historique et des états

L'état de l'exécution est conservé par le *checkpoint* de LangGraph, qui enregistre l'avancement du traitement (plan, étape courante, résultats intermédiaires, artefacts). Un résumé de conversation est consolidé périodiquement afin de maîtriser la taille du contexte : les échanges antérieurs sont condensés et injectés dans les tours suivants, tout en conservant l'historique complet en base de données.

Contrôle des interactions avec le modèle

Enfin, les interactions avec le modèle sont contrôlées : l'agent n'accède aux données que par l'intermédiaire d'outils validés, les appels au modèle sont configurés (modèle, paramètres de génération) et chaque exécution produit une trace exploitable pour le diagnostic et l'amélioration continue, conformément aux principes du *LLM Ops* décrits au chapitre précédent. Le modèle est invoqué au travers d'une interface compatible OpenAI : en production, il est servi localement par la passerelle d'inférence décrite dans la section consacrée au déploiement, tandis que les phases de test ont recours à des API externes.

5.5.7 La traçabilité par l'evidence

L'**evidence** constitue le mécanisme garantissant la véracité des réponses. À chaque étape du plan, les résultats réels produits par les outils — résultats de requêtes, statistiques calculées, résumés de graphiques — sont accumulés dans l'état de l'exécution. La synthèse finale est construite à partir de ce manifeste d'evidence : le modèle de langage est explicitement invité à fonder ses conclusions sur les preuves calculées et à indiquer le niveau de confiance associé, comme l'illustre l'extrait suivant.

Listing 5.7 : Accumulation de l'evidence et synthèse

```
# Chaque étape accumule les preuves réellement calculées par les outils
full_evidence = "".join(evidence_tokens)
if full_evidence.strip():
    step_evidence = f"\n## Étape {step.id}: {step.title}\n{full_evidence.strip()}"

# La synthèse ne reçoit que l'evidence calculée, jamais un résultat inventé
synthesis_prompt = SYNTHESIS_SYSTEM_PROMPT.format(evidence=evidence)
```

Ainsi, chaque conclusion présentée à l'utilisateur est traçable vers des résultats calculés de manière déterministe. Le modèle de langage interprète et rédige, mais ne produit jamais lui-même les chiffres : il s'appuie exclusivement sur l'evidence

accumulée, conformément aux exigences de transparence et de reproductibilité définies au chapitre précédent.

L’evidence au cœur de la fiabilité

Le mécanisme d’evidence garantit que toute réponse est fondée sur des résultats réellement calculés par le moteur d’analyse. Les graphiques et statistiques produits constituent des preuves associées aux conclusions, et non des illustrations décoratives.

5.6 Réalisation du frontend et de l’internationalisation

5.6.1 L’interface SvelteKit

L’interface a été développée avec SvelteKit 5, organisée autour de plusieurs espaces : un tableau de bord offrant une vue d’ensemble des jeux de données, des conversations et des rapports ; une gestion des jeux de données présentant le schéma, le profilage et les statistiques ; un espace de conversation ; ainsi qu’une page de paramètres. La navigation et les composants sont construits sur la bibliothèque de composants partagés du projet.

5.6.2 L’interface conversationnelle

L’espace de conversation constitue le cœur de l’expérience utilisateur. Il consomme le flux d’événements SSE diffusé par le backend et affiche en temps réel le déroulement de l’analyse : étapes du plan, appels d’outils, tokens de la réponse et artefacts produits. Les messages sont rendus au format Markdown et les appels d’outils sont présentés sous forme de panneaux déplaçables, offrant ainsi une transparence totale sur le traitement en cours.

5.6.3 La gestion des sessions et des conversations

La plateforme permet de créer, lister, reprendre et supprimer des conversations. Chaque conversation est associée à un jeu de données, ce qui définit le contexte de l’analyse. L’historique des échanges est rechargé depuis le backend, permettant à l’utilisateur de reprendre une discussion là où elle s’est arrêtée.

5.6.4 La visualisation des résultats

Les graphiques produits par le moteur d’analyse sont rendus de manière interactive à partir de leurs spécifications Plotly, directement dans la conversation. Les artefacts —

graphiques, exports, rapports — sont affichés sous forme de cartes avec la possibilité de les télécharger ou de les consulter.

5.6.5 L'internationalisation (i18n)

La plateforme est **internationalisée** en français et en anglais. L'internationalisation repose sur le framework *inlang* et *Paraglide JS* : les textes de l'interface sont centralisés dans des fichiers de messages par langue (`messages/en.json`, `messages/fr.json`), puis compilés en fonctions typées utilisées dans les composants. L'utilisateur peut basculer la langue de l'interface à tout moment depuis les paramètres, comme l'illustre l'extrait suivant.

Listing 5.8 : Utilisation de l'internationalisation dans un composant

```
<script>
  import { i18n, m } from '$lib/i18n';
</script>

<h1>{m.overview_title()}</h1>
<button onclick={() => i18n.setLocale('fr')}>Français</button>
```

Au-delà de l'interface, la langue sélectionnée est intégrée à la construction des prompts adressés au modèle de langage : l'agent répond ainsi dans la langue de l'utilisateur, ce qui améliore l'accessibilité de la plateforme pour les professionnels de santé francophones et anglophones du GST-TTA.

5.7 Déploiement et intégration

5.7.1 L'intégration des composants

L'intégration de la solution s'est faite à trois niveaux complémentaires. L'intégration frontend/backend repose sur les interfaces REST et le streaming SSE : l'interface consomme directement les événements diffusés par l'API. L'intégration agent/moteur d'analyse passe par le registre d'outils : l'orchestrateur invoque les capacités du moteur sans dépendre de leur implémentation et récupère les résultats sous une forme structurée alimentant l'evidence. Enfin, l'intégration du modèle de langage est assurée par le cadre d'exécution (harness) : les appels au modèle sont configurés, encadrés et tracés, et les réponses sont produites dans la langue de l'utilisateur.

5.7.2 Le déploiement conteneurisé

La plateforme est conteneurisée et déployée au travers de Docker : une image pour l'API, une image pour l'application web, et une configuration Nginx assurant le reverse proxy. Un fichier de composition (docker-compose) orchestre l'ensemble des services — API, interface et base de données PostgreSQL — tandis que la configuration est externalisée par variables d'environnement. Cette approche garantit la portabilité et la reproductibilité du déploiement sur l'infrastructure locale, en cohérence avec l'exigence de souveraineté des données identifiée au chapitre consacré à l'analyse des besoins.

5.7.3 L'infrastructure de modèles de langage locaux

En production, la plateforme s'appuie sur une infrastructure de modèles de langage entièrement locale, qui garantit la souveraineté des traitements. Cette infrastructure repose sur une passerelle d'inférence auto-hébergée (projet llms-gateway) qui sert des modèles GGUF via le moteur llama.cpp et expose une API compatible OpenAI. Un reverse proxy Nginx constitue le point d'entrée unique (port 6060) et route les requêtes vers des conteneurs d'inférence dédiés par capacité — chat, embeddings, reranking, vision — gérés dynamiquement par le biais du SDK Docker.

L'intégration avec la plateforme est transparente : le backend est configuré pour adresser la passerelle, comme l'illustre l'extrait suivant.

Listing 5.9 : Configuration de l'accès au modèle (extrait de .env)

```
# Production : modele local servi par la passerelle llms-gateway
OPENAI_BASE_URL=http://localhost:6060/v1
AGENT_MODEL=<modele-local>

# Tests de developpement : API externe compatible OpenAI
# OPENAI_BASE_URL=https://api.openai.com/v1
```

L'orchestrateur LangGraph et les appels du modèle utilisent ainsi le même contrat d'API, que le modèle soit servi localement par la passerelle ou, ponctuellement, par un fournisseur externe durant les tests. La passerelle assure également la gestion du cycle de vie des modèles : installation depuis le hub HuggingFace avec vérification d'intégrité, démarrage des conteneurs d'inférence à la demande et remplacement à chaud. Aucune donnée n'est transmise à un service externe en production, conformément à l'exigence de souveraineté identifiée au chapitre consacré à l'analyse des besoins.

5.8 Conclusion

Ce chapitre a présenté la réalisation effective de la plateforme, en répondant à la question de la mise en œuvre concrète de la solution. L'intelligence des données a été

introduite comme cadre structurant : la plateforme transforme les données hospitalières du GST-TTA en connaissances exploitables, accessibles en langage naturel et fondées sur des résultats vérifiables.

Les principales réalisations ont été détaillées : le backend FastAPI et son API REST, la chaîne d'ingestion et de profilage des jeux de données, le streaming temps réel, l'orchestration agentique et le mécanisme d'evidence garantissant la traçabilité des réponses, l'interface SvelteKit et son internationalisation en français et en anglais, ainsi que le déploiement conteneurisé et l'infrastructure de modèles de langage locaux. Les mesures de sécurité et de gestion des erreurs mises en œuvre ont également été présentées. Seuls les extraits de code les plus structurants ont été inclus, conformément à l'esprit d'un rapport académique.

La solution réalisée constitue ainsi une mise en œuvre complète des choix de conception du chapitre précédent. Le chapitre suivant est consacré aux tests effectués pour valider le bon fonctionnement de la plateforme, à la présentation des résultats obtenus ainsi qu'aux limites et perspectives d'amélioration.

Chapitre 6

Tests et validation

6.1 Introduction

Ce chapitre présente les tests réalisés pour valider le bon fonctionnement de la plateforme et évaluer sa conformité aux besoins identifiés au chapitre consacré à l'analyse des besoins. Dans le cadre de la démarche Agile adoptée, la validation a été menée de manière continue : chaque sprint livrait un incrément fonctionnel vérifié avant d'être intégré à la solution.

Le chapitre décrit la stratégie de test mise en œuvre et son environnement, les tests unitaires des composants, les tests d'intégration et la validation fonctionnelle, ainsi que les tests de la passerelle de modèles locaux. Il présente ensuite les aspects de sécurité, de performance et de qualité, avant de synthétiser les résultats obtenus, les limites identifiées et les perspectives d'amélioration envisageables.

6.2 Stratégie et environnement de test

La validation de la plateforme repose sur plusieurs niveaux de test complémentaires, conformes aux bonnes pratiques de développement logiciel :

- **Tests unitaires** : validation de composants isolés, notamment l'exécution des outils de l'agent et la gestion de leurs erreurs ;
- **Tests d'intégration** : vérification des interactions entre les composants — API, streaming, ingestion, orchestration ;
- **Validation fonctionnelle** : scénarios de bout en bout correspondant aux cas d'utilisation identifiés au chapitre consacré à l'analyse des besoins ;
- **Tests de sécurité et de performance** : contrôle des mécanismes de protection des données et de la réactivité des traitements.

L'environnement de test s'appuie sur `pytest` et le gestionnaire de paquets `uv` (`uv run pytest`), avec des scripts de validation localisés à la racine du dépôt ainsi que

dans les différents composants. Pour la passerelle de modèles locaux, une intégration continue est en place au travers d'un workflow GitHub Actions, comme détaillé dans la suite du chapitre. Cette stratégie garantit une validation à la fois locale (développement) et automatisée (intégration).

6.3 Tests unitaires des composants

Les tests unitaires visent à valider le comportement des composants fondamentaux de l'agent et du moteur d'analyse, indépendamment du reste du système.

6.3.1 Exécution des outils et tolérance aux pannes

Le premier volet vérifie l'exécution des outils au travers du nœud d'outils de l'agent : les fonctions enregistrées dans le registre doivent être correctement invoquées à partir des appels d'outils du modèle et leurs résultats correctement formatés. Le second volet porte sur la **tolérance aux pannes** : un outil qui échoue (par exemple une requête SQL invalide) doit produire une erreur structurée, sans interrompre le processus d'exécution. Ce comportement est validé à la fois avec le mode de gestion d'erreur du nœud d'outils et au sein du graphe de l'agent, comme l'illustre l'extrait suivant.

Listing 6.1 : Test de tolérance aux pannes d'un outil (extrait)

```
from langchain_core.messages import AIMessage, ToolCall

msg = AIMessage(content="", tool_calls=[ToolCall(name="bad_tool", args={}, id="
    call_123")])
res = tools_node.invoke({"messages": [msg]})
# L'echec de l'outil est capture et retourne sous forme d'erreur structuree,
# sans interrompre le processus d'execution.
```

Ces validations confirment que le mécanisme de reprise présenté au chapitre précédent — l'erreur renvoyée dans la mémoire de travail et corrigée par le planificateur — s'appuie sur un socle d'exécution robuste.

6.3.2 Tests de bornes et robustesse

Un troisième volet vérifie les **bornes** appliquées aux traitements analytiques : le volume maximal des résultats retournés par les requêtes DuckDB (afin d'éviter la saturation mémoire) et les seuils de taille des jeux de données manipulés. Ces tests valident que les limites de sécurité définies lors de la réalisation sont effectivement respectées par le moteur d'analyse.

Le moteur d'analyse (profilage, statistiques, chargement multi-formats) est par ailleurs exercé au travers de ces validations ainsi que dans les scénarios fonctionnels décrits ci-après.

6.4 Tests d'intégration et validation fonctionnelle

Les tests d'intégration et la validation fonctionnelle vérifient le comportement global de la plateforme selon les scénarios d'usage réels.

6.4.1 Validation de l'API et du streaming

Les points de terminaison de l'API REST (jeux de données, conversations, artefacts, catégories sémantiques) ont été validés, notamment la documentation OpenAPI générée automatiquement. Le flux de **streaming** (SSE) a été vérifié sur l'ensemble de ses événements : annonce du plan, étapes, appels d'outils, diffusion des tokens, artefacts, erreur et fin. La validation confirme que le frontend reçoit en temps réel les événements produits par l'orchestrateur, sans rupture de la connexion en cas d'erreur.

6.4.2 Validation de la chaîne d'ingestion

Le parcours d'importation d'un jeu de données a été validé de bout en bout : téléversement d'un fichier, enregistrement du fichier brut, profilage automatique des colonnes, persistance des métadonnées et affichage du schéma et des statistiques dans l'interface. Les formats pris en charge (CSV, Parquet, XLSX, JSON, etc.) et la détection des encodages ont été exercés sur des jeux de données variés.

6.4.3 Validation du workflow d'analyse

Le scénario central — une question posée en langage naturel aboutissant à une analyse complète — a été validé conformément au workflow présenté au chapitre consacré à la conception (figure 4.3) : génération du plan, exécution des étapes par les outils, accumulation de l'evidence et synthèse de la réponse. La validation vérifie en particulier que chaque conclusion est tracée vers des résultats réellement calculés, et que les graphiques produits constituent des preuves associées aux réponses.

6.4.4 Validation utilisateur

Enfin, des démonstrations ont été réalisées avec les parties prenantes du GST-TTA afin de valider l'adéquation de la plateforme aux besoins métiers. Les retours recueillis

ont été intégrés au fil des itérations, conformément à la démarche Agile présentée au chapitre consacré au contexte général, contribuant à l'amélioration continue de l'interface et des fonctionnalités.

6.5 Tests de la passerelle de modèles locaux

La passerelle d'inférence qui sert les modèles locaux en production (présentée au chapitre précédent) dispose de sa propre suite de tests, garantissant la disponibilité de l'endpoint utilisé par la plateforme.

6.5.1 Suite de tests de la passerelle

La passerelle (*llms-gateway*) est validée à plusieurs niveaux : des tests de santé des endpoints (`/health`, `/ready`), des tests de l'API de gestion des modèles (installation, listage, progression), des tests de recherche sur le hub HuggingFace et des tests du système. Les bibliothèques de base sont également couvertes : le registre et ses backends de stockage, la validation des fichiers téléchargés et le client HuggingFace pour le paquet central, ainsi que le cycle de vie des conteneurs et l'allocation des ports pour l'orchestration.

6.5.2 Intégration continue

Une intégration continue est mise en place au travers d'un workflow GitHub Actions qui, sur chaque modification, identifie les composants affectés (paquet central, interface en ligne de commande, orchestration) et exécute les cibles de test correspondantes. Cette automatisation garantit que les évolutions de la passerelle ne régressent pas les mécanismes de gestion des modèles sur lesquels s'appuie la plateforme en production.

6.6 Sécurité, performance et qualité

6.6.1 Sécurité

Les contrôles de sécurité mis en œuvre lors de la réalisation ont été vérifiés : le moteur DuckDB fonctionne en lecture seule, les volumes de résultats sont bornés, et les formats des fichiers importés sont validés. Un audit de la plateforme a par ailleurs été mené, attribuant les appréciations suivantes : **A-** pour l'architecture et la qualité du code, **A** pour la conception de la base de données et du stockage, et **B-**

pour la posture de sécurité, en raison de l'absence d'authentification et d'une politique CORS volontairement ouverte en environnement de développement. Ces réserves sont détaillées dans les limites de la solution.

6.6.2 Performance

La validation des performances s'est concentrée sur la réactivité du streaming temps réel (diffusion progressive des étapes et des tokens) et sur l'exécution des requêtes analytiques : le recours à DuckDB permet de traiter de grands volumes de données en mémoire, sans transiter par la base relationnelle pour les charges de calcul lourdes.

6.6.3 Qualité des réponses

La qualité des réponses produites par l'agent est évaluée au regard de leur fondement : chaque réponse s'appuie exclusivement sur l'evidence calculée par le moteur d'analyse, et les graphiques générés accompagnent les conclusions qu'ils illustrent. Ce mécanisme constitue une base objective d'évaluation, complétée par des critères d'appréciation automatisés (type *LLM-as-a-judge*) envisagés pour affiner le contrôle qualité.

6.7 Résultats, limites et perspectives

6.7.1 Résultats et synthèse

L'ensemble des validations a permis de confirmer le bon fonctionnement de la plateforme selon les scénarios prévus. Le tableau 6.1 synthétise les niveaux de test et les principaux constats.

TAB. 6.1 : Synthèse des tests et validation

Niveau de test	Objets validés	Constats
Tests unitaires	Exécution des outils, tolérance aux pannes, bornes des requêtes	Mécanismes d'exécution et de reprise validés
Intégration	API REST, streaming SSE, chaîne d'ingestion	Scénarios de bout en bout opérationnels
Passerelle de modèles	Santé, API modèles, bibliothèques, CI	Suite dédiée opérationnelle et automatisée
Sécurité et performance	Contrôles DuckDB, bornage, réactivité du streaming	Contrôles validés ; réserves identifiées (auth, CORS)
Validation utilisateur	Démonstrations GST-TTA	Retours intégrés au fil des itérations

6.7.2 Limites de la solution

Malgré les résultats obtenus, plusieurs limites ont été identifiées :

- **Authentification** : les points de terminaison de l'API ne sont actuellement pas protégés par un contrôle d'authentification ; la plateforme ne distingue pas les utilisateurs ni leurs droits d'accès ;
- **Politique CORS** : la configuration autorise volontairement toutes les origines en environnement de développement, ce qui doit être restreint en production ;
- **Nettoyage des fichiers** : les artefacts générés sont conservés indéfiniment, sans politique de purge automatisée ;
- **Interface héritée** : une ancienne interface (web-2) est maintenue en parallèle de l'application principale, engendrant une charge de maintenance redondante ;
- **Dépendance aux API externes en test** : les phases de test ont recours à des API externes compatibles OpenAI, alors que la production privilégie les modèles locaux.

6.7.3 Perspectives d'amélioration

Les limites identifiées ouvrent plusieurs perspectives d'amélioration :

- **Sécurité** : mettre en place une authentification (JWT) et une autorisation multi-tenant, et restreindre la politique CORS par variables d'environnement ;

- **Hygiène des données** : automatiser la purge des artefacts obsolètes et durcir l'accès aux fichiers ;
- **Consolidation** : déprécier l'interface héritée et généraliser l'usage des modèles locaux pour les tests ;
- **Qualité et intégration continue** : étendre l'intégration continue à l'ensemble de la plateforme et renforcer l'évaluation systématique des réponses de l'agent (critères objectifs, *LLM-as-a-judge*) ;
- **Extension des capacités** : enrichir le moteur d'analyse (nouveaux tests statistiques, connecteurs de données) et améliorer la gestion du contexte pour de très longues conversations.

Validation de la solution

Les tests réalisés confirment que la plateforme répond aux besoins fonctionnels identifiés et que ses mécanismes fondamentaux — orchestration, evidence, streaming, sécurité — fonctionnent conformément aux choix de conception. Les limites recensées, principalement liées à l'authentification et à l'industrialisation, constituent des axes d'amélioration prioritaires.

6.8 Conclusion

Ce chapitre a présenté la démarche de validation de la plateforme : la stratégie et l'environnement de test, les tests unitaires des composants, les tests d'intégration et la validation fonctionnelle, ainsi que les tests de la passerelle de modèles locaux. Les aspects de sécurité, de performance et de qualité ont été examinés, et les résultats ont été synthétisés.

Les tests confirment le bon fonctionnement de la solution et la conformité de ses mécanismes fondamentaux aux choix de conception. Les limites identifiées — principalement l'absence d'authentification, la politique CORS et l'industrialisation des tests — ainsi que les perspectives d'amélioration associées constituent des axes de travail futurs.

Ce chapitre clôt la présentation du projet. La conclusion générale du mémoire synthétise l'ensemble des travaux réalisés, de l'analyse des besoins à la validation de la plateforme, et rappelle les apports de la solution pour le GST-TTA.

Conclusion générale

Ce projet de fin d'année s'est inscrit dans la stratégie de transformation numérique du **Groupement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma (GST-TTA)**, dont l'objectif est de valoriser un patrimoine de données hospitalières de plus en plus riche. Face à la complexité des outils d'analyse traditionnels, aux contraintes strictes de confidentialité des données de santé et à la fragmentation des processus d'exploitation, le projet visait à concevoir une plateforme intelligente permettant aux professionnels de santé et aux décideurs d'interroger leurs données en langage naturel, tout en garantissant la fiabilité, la transparence et la souveraineté des traitements.

La démarche adoptée a combiné une analyse rigoureuse des besoins et de l'existant, une conception architecturale fondée sur une séparation claire entre le pilotage par l'IA et le calcul déterministe, puis une réalisation itérative en sprints jusqu'à la validation de la solution. La plateforme résultante intègre une interface conversationnelle internationalisée, un backend asynchrone et sécurisé, un orchestrateur agentique (LangGraph), un moteur d'analyse déterministe et une infrastructure de modèles de langage locaux garantissant qu'aucune donnée ne quitte l'infrastructure du groupement.

L'**apport principal** du projet réside dans la dissociation entre l'orchestration pilotée par le modèle de langage et l'exécution déterministe des traitements : le modèle comprend, planifie et interprète, tandis que le moteur d'analyse réalise les calculs de manière vérifiable. Le mécanisme d'*evidence* assure que chaque conclusion repose sur des résultats réellement calculés, répondant ainsi aux exigences de fiabilité, de transparence et de reproductibilité. L'infrastructure de modèles locaux concrétise quant à elle l'exigence de souveraineté, les API externes n'étant mobilisées qu'en phase de test.

La validation de la plateforme a confirmé son bon fonctionnement et sa conformité aux besoins identifiés, tout en mettant en évidence des axes d'amélioration : mise en place de l'authentification et du contrôle d'accès, industrialisation des tests, politique de nettoyage des fichiers et extension des capacités analytiques.

La réalisation de ce projet a également été confrontée à plusieurs difficultés techniques. L'une des principales a concerné l'intégration des modèles de langage dans un processus d'analyse fiable et reproductible, notamment en raison de la variabilité des réponses générées et de la nécessité de contrôler les différentes étapes d'exécution. La mise en place d'une séparation stricte entre l'orchestration pilotée par l'IA et les traitements déterministes a ainsi nécessité plusieurs ajustements architecturaux. L'utilisation de modèles de langage locaux a également soulevé des contraintes liées aux

ressources matérielles disponibles, aux performances et au choix des modèles adaptés aux besoins d'analyse. Par ailleurs, la conception de l'orchestration agentique et la mise en place du mécanisme d'*evidence*, permettant de relier les conclusions aux résultats effectivement calculés, ont constitué des défis importants. Ces difficultés ont néanmoins permis d'affiner les choix techniques et de mieux prendre en compte les contraintes liées à l'utilisation de l'intelligence artificielle générative dans un contexte d'analyse de données de santé.

Les perspectives sont néanmoins prometteuses, notamment la généralisation des modèles locaux, l'évaluation systématique de la qualité des réponses et l'intégration de nouvelles sources de données hospitalières.

Au-delà du GST-TTA, cette réalisation démontre la faisabilité d'une **analyse conversationnelle de données de santé souveraine**, accessible et fondée sur des preuves. Elle ouvre la voie à une adoption plus large de l'intelligence des données au service du pilotage des établissements de santé et de la qualité des soins.

Bilan du projet

Le projet a permis d'aboutir à une plateforme fonctionnelle d'analyse conversationnelle de données, conçue pour répondre aux contraintes spécifiques du contexte hospitalier. Au-delà de la mise en œuvre des différentes fonctionnalités, plusieurs contributions structurantes peuvent être retenues.

- **Une interface conversationnelle** permettant aux utilisateurs d'interroger leurs données en langage naturel, sans nécessiter la maîtrise préalable de langages d'analyse ou de requêtes complexes ;
- **Une architecture modulaire** séparant clairement les différentes responsabilités du système, notamment l'interface utilisateur, les services applicatifs, l'orchestration agentique, le moteur d'analyse et la gestion des modèles de langage ;
- **Une séparation entre intelligence et calcul**, dans laquelle le modèle de langage assure principalement la compréhension, la planification et l'interprétation, tandis que les opérations analytiques sont exécutées par des composants déterministes et vérifiables ;
- **Un mécanisme d'evidence** permettant d'associer les conclusions produites aux résultats effectivement calculés par le moteur d'analyse, renforçant ainsi la traçabilité, la transparence et la reproductibilité des analyses ;
- **Une architecture agentique** reposant sur un orchestrateur permettant de décomposer les demandes complexes en différentes étapes et de coordonner les outils nécessaires à leur exécution ;
- **Une infrastructure de modèles locaux** permettant de répondre aux exigences de souveraineté des données, les données sensibles pouvant rester au sein de l'infrastructure du GST-TTA ;
- **Une conception extensible**, permettant d'envisager l'intégration progressive de nouveaux outils analytiques, de nouveaux modèles de langage et de nouvelles sources de données hospitalières.

Ainsi, la contribution du projet ne se limite pas à la réalisation d'une interface conversationnelle. Elle réside également dans la conception d'une architecture permettant de combiner les capacités d'interprétation des modèles de langage avec la fiabilité des traitements déterministes. Cette approche constitue un compromis entre facilité d'utilisation, contrôle des traitements, traçabilité des résultats et maîtrise des données.

Les travaux réalisés constituent également une base pour des évolutions futures, notamment l'intégration de mécanismes d'authentification et de contrôle d'accès, l'enrichissement des capacités analytiques, l'évaluation systématique des réponses générées et la généralisation de l'utilisation des modèles locaux.

Annexe A

Annexes

A.1 Captures d'écran de la plateforme

Cette annexe présente les interfaces de la plateforme réalisée. Pour chaque capture, une brève description indique à quoi sert l'écran et son rôle dans le parcours utilisateur. Les captures sont organisées selon les espaces fonctionnels de l'application : tableau de bord, gestion des jeux de données, espace conversationnel et artefacts.

A.1.1 Tableau de bord

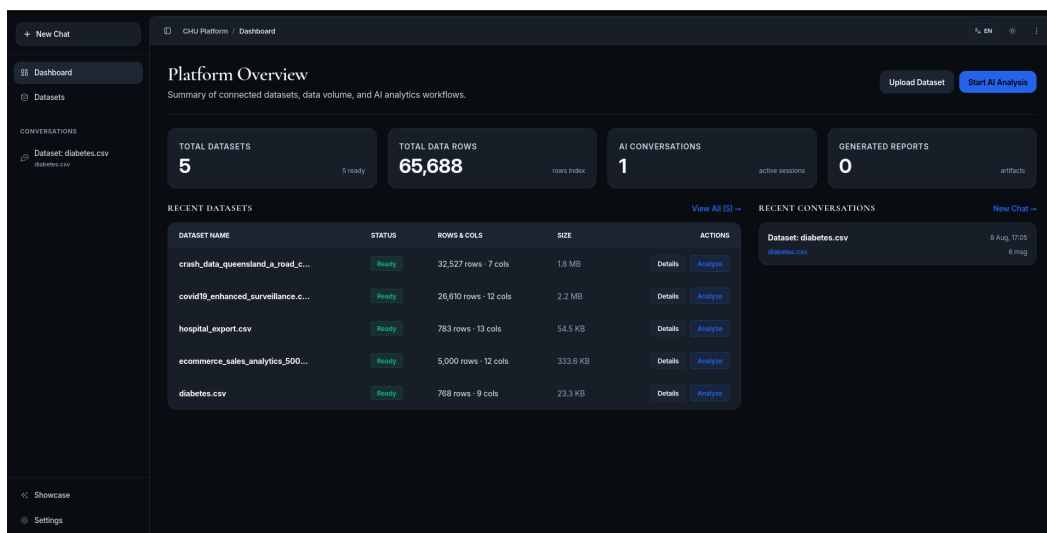


FIG. A.1 : Tableau de bord : vue d'ensemble de la plateforme. Cette page sert de point d'entrée et présente les jeux de données disponibles, les conversations récentes et les indicateurs clés de l'activité.

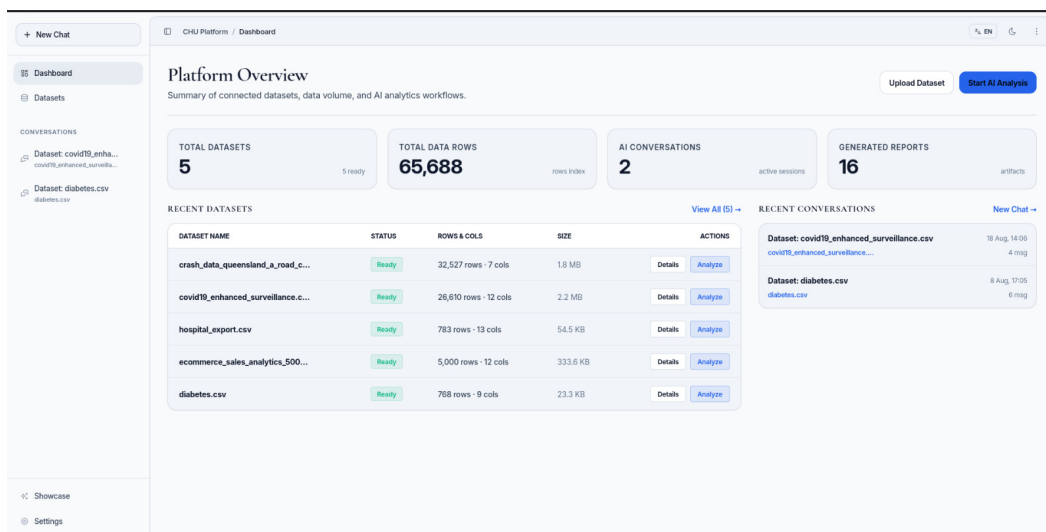


FIG. A.2 : Tableau de bord (thème clair) : indicateurs clés de la plateforme — jeux de données connectés, volume de lignes, conversations actives et rapports générés.

A.1.2 Gestion des jeux de données

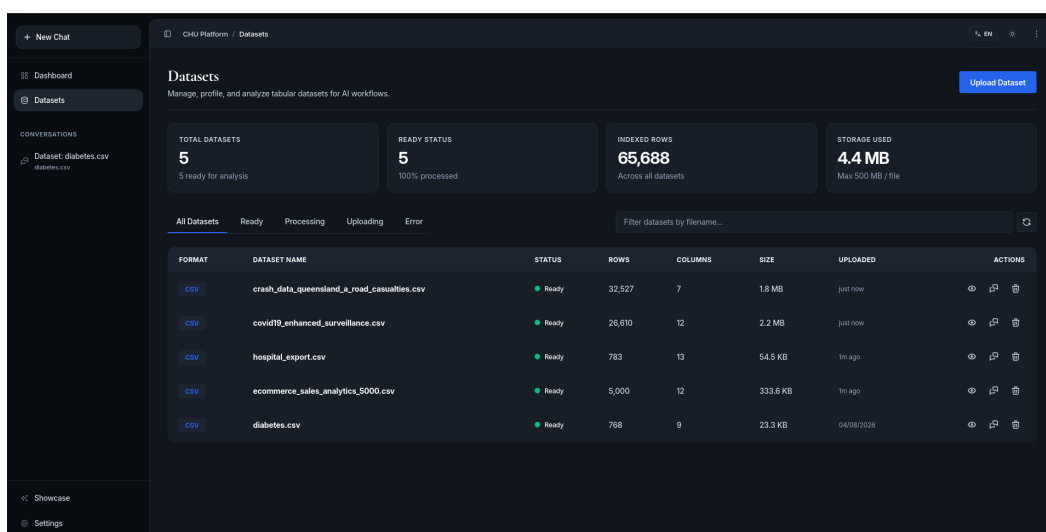


FIG. A.3 : Page de gestion des jeux de données : liste des jeux de données importés dans la plateforme, avec les actions disponibles (ouvrir, créer une conversation, supprimer).

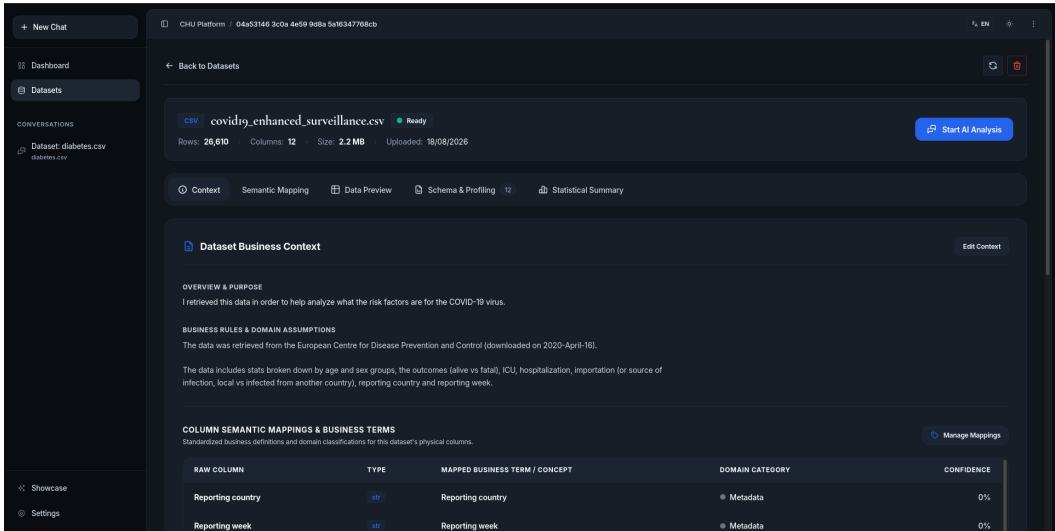


FIG. A.4 : Page de détail d’un jeu de données : informations générales (nom, taille, format, source) et accès aux différentes vues d’analyse (schéma, aperçu, statistiques).

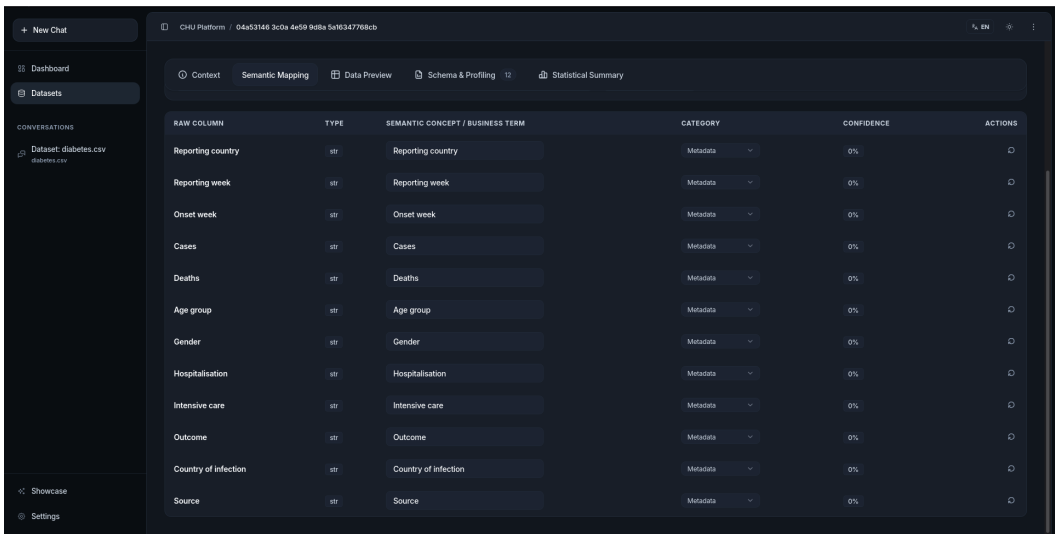


FIG. A.5 : Mappage des colonnes : lors de l’import, la plateforme associe les colonnes du fichier aux types de données attendus, garantissant une interprétation correcte du contenu.

A.1. CAPTURES D'ÉCRAN DE LA PLATEFORME

CHU Platform 04a53148 3c0a 4e59 9d8a 5a16347768cb

Back to Datasets

CSV covid19_enhanced_surveillance.csv Ready Start AI Analysis

Rows: 26,610 Columns: 12 Size: 2.2 MB Uploaded: 18/08/2026

Context Semantic Mapping Data Preview Schema & Profiling 12 Statistical Summary

Search cell values in preview...

Limit rows: 10 rows

#	REPORTING COUNTRY	REPORTING WEEK	ONSET WEEK	CASES	DEATHS	AGE GROUP	GENDER	HOSPITALISATION	INTENSIVE CARE	OUTCOME	COUNTRY OF INFECTION	SOURCE
1	Austria	2020-W17	2020-W10	<=5	0	25-49	Male	Not hospitalised	Unknown	Unknown	Austria	TESSy
2	Austria	2020-W17	2020-W11	<=5	0	50-64	Male	Not hospitalised	Unknown	Unknown	-	TESSy
3	Austria	2020-W17	2020-W11	<=5	0	65-79	Male	Not hospitalised	Unknown	Unknown	-	TESSy
4	Austria	2020-W17	2020-W12	<=5	0	25-49	Male	Not hospitalised	Unknown	Unknown	-	TESSy
5	Austria	2020-W17	2020-W14	<=5	0	15-24	Female	Not hospitalised	Unknown	Unknown	-	TESSy
6	Austria	2020-W17	2020-W14	<=5	0	25-49	Female	Not hospitalised	Unknown	Unknown	-	TESSy
7	Austria	2020-W17	2020-W14	<=5	0	25-49	Male	Not hospitalised	Unknown	Unknown	-	TESSy
8	Austria	2020-W17	2020-W14	<=5	0	50-64	Male	Not hospitalised	Unknown	Unknown	-	TESSy
9	Austria	2020-W17	2020-W14	<=5	0	65-79	Male	Not hospitalised	Unknown	Unknown	-	TESSy
10	Austria	2020-W17	2020-W14	<=5	0	80+	Male	Not hospitalised	Unknown	Unknown	-	TESSy

Showing 10 of 10 previewed rows Total: 26,610 rows x 12 columns

FIG. A.6 : Aperçu des données : les premières lignes du jeu de données sont affichées pour vérifier le contenu importé avant analyse.

CHU Platform 04a53148 3c0a 4e59 9d8a 5a16347768cb

Back to Datasets

CSV covid19_enhanced_surveillance.csv Ready Start AI Analysis

Rows: 26,610 Columns: 12 Size: 2.2 MB Uploaded: 18/08/2026

Context Semantic Mapping Data Preview Schema & Profiling 12 Statistical Summary

Filter columns by name or type...

Showing 12 of 12 columns

#	COLUMN NAME	DATA TYPE	NULL COUNT	NULL %	UNIQUE VALUES	SAMPLE VALUE
1	Reporting country	str	0	0%	26	Austria
2	Reporting week	str	0	0%	15	2020-W17
3	Onset week	str	0	0%	20	2020-W10
4	Cases	int	0	0%	501	<=5
5	Deaths	int	0	0%	121	0
6	Age group	str	0	0%	8	25-49
7	Gender	str	0	0%	4	Male
8	Hospitalisation	str	0	0%	3	Not hospitalised
9	Intensive care	str	0	0%	3	Unknown
10	Outcome	str	0	0%	5	Unknown
11	Country of infection	str	0	0%	115	Austria
12	Source	str	0	0%	1	TESSy

FIG. A.7 : Schéma du jeu de données : liste des colonnes avec leurs types et leurs caractéristiques, offrant une vision structurée des données.

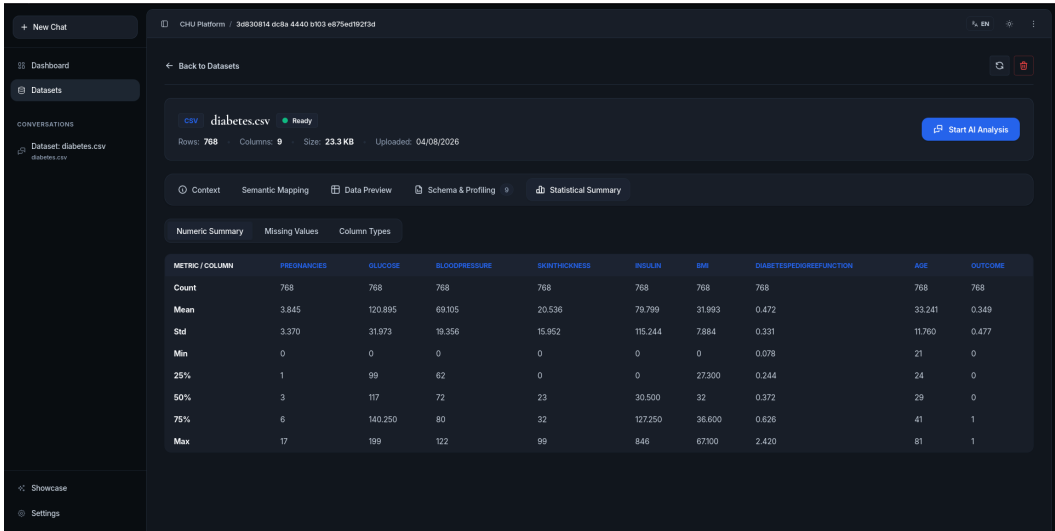


FIG. A.8 : Statistiques du jeu de données : indicateurs descriptifs calculés par le profilage (moyenne, médiane, écart-type, valeurs manquantes, valeurs uniques).

A.1.3 Espace conversationnel

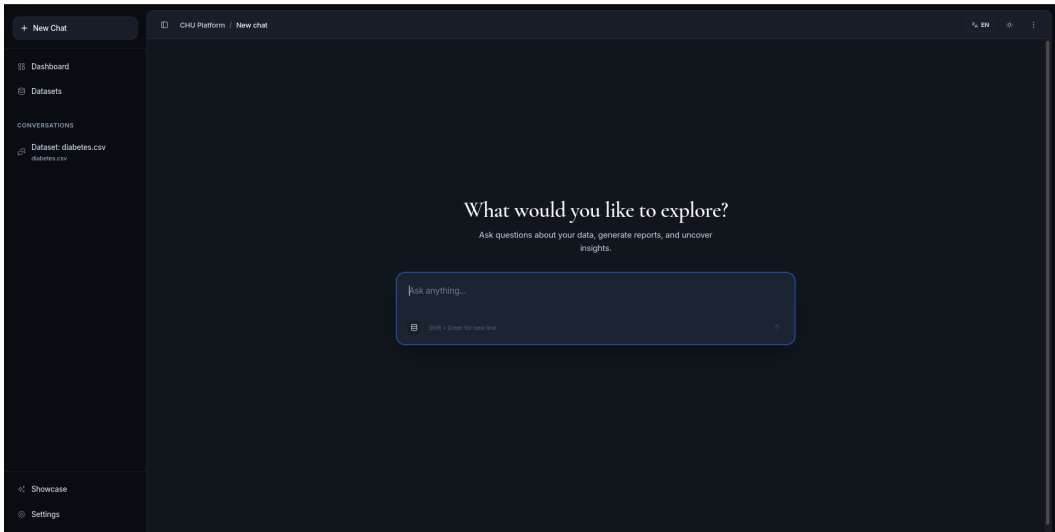


FIG. A.9 : Nouvelle conversation : espace de discussion vide, point de départ d’une analyse en langage naturel.

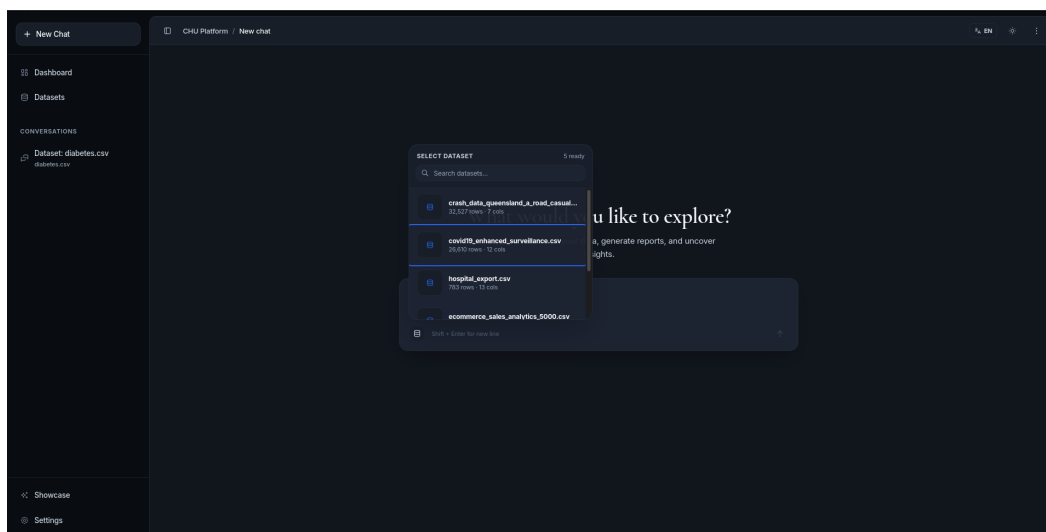


FIG. A.10 : Nouvelle conversation associée à un jeu de données : l'utilisateur sélectionne le contexte de l'analyse avant de poser sa question.

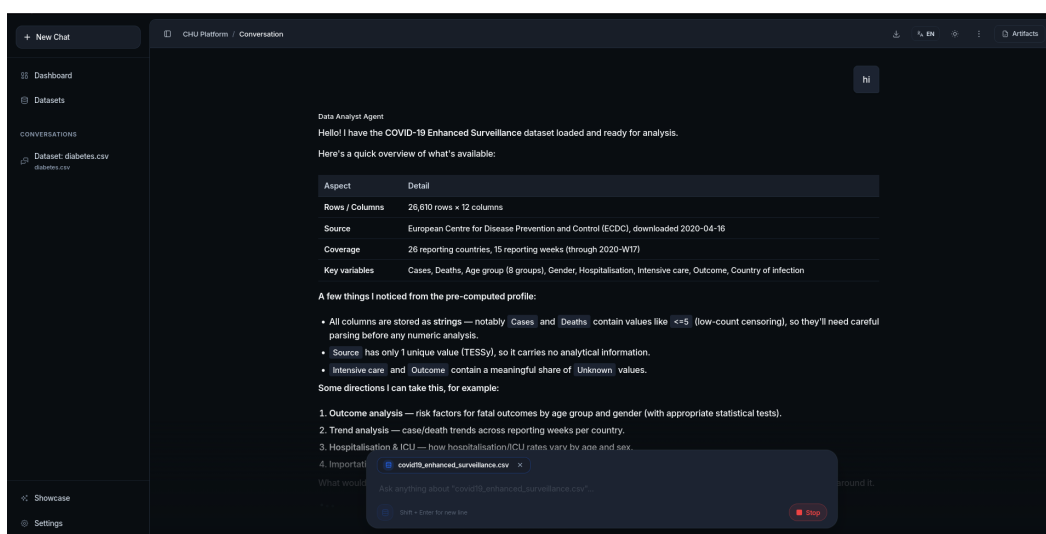


FIG. A.11 : Requête en langage naturel : exemple de question posée par l'utilisateur, qui sera décomposée par l'agent en étapes d'analyse.

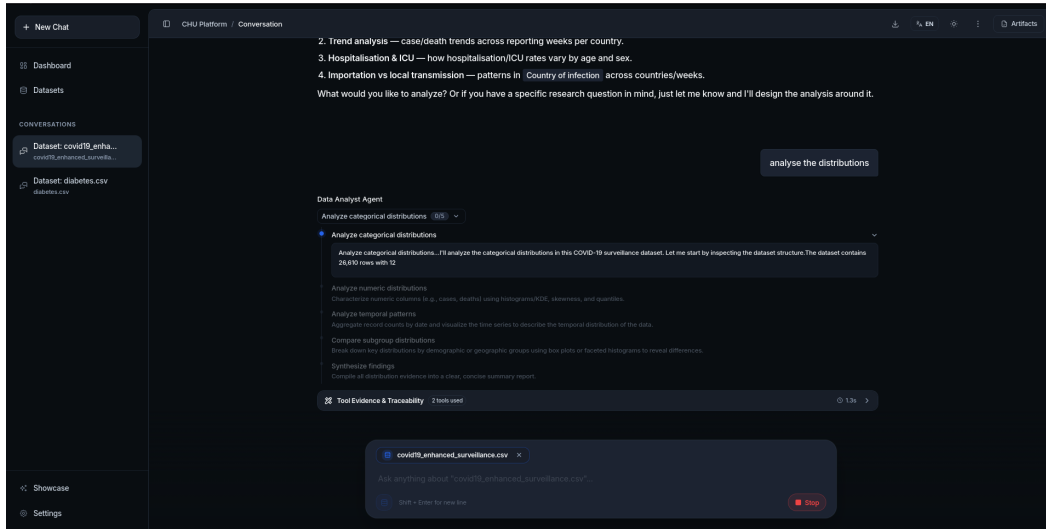


FIG. A.12 : Planification : l’agent décompose la requête en un plan d’étapes ordonné, conformément à l’orchestration LangGraph.

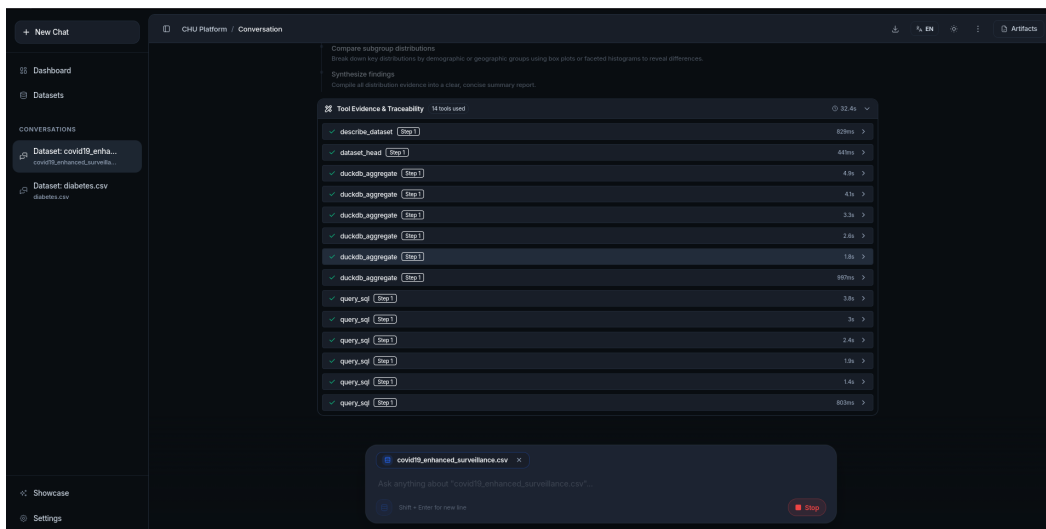
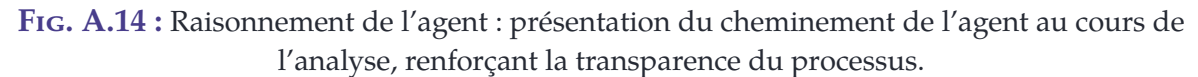


FIG. A.13 : Exécution des outils : déroulement des étapes du plan avec l’affichage des appels d’outils, garantissant la traçabilité du traitement.



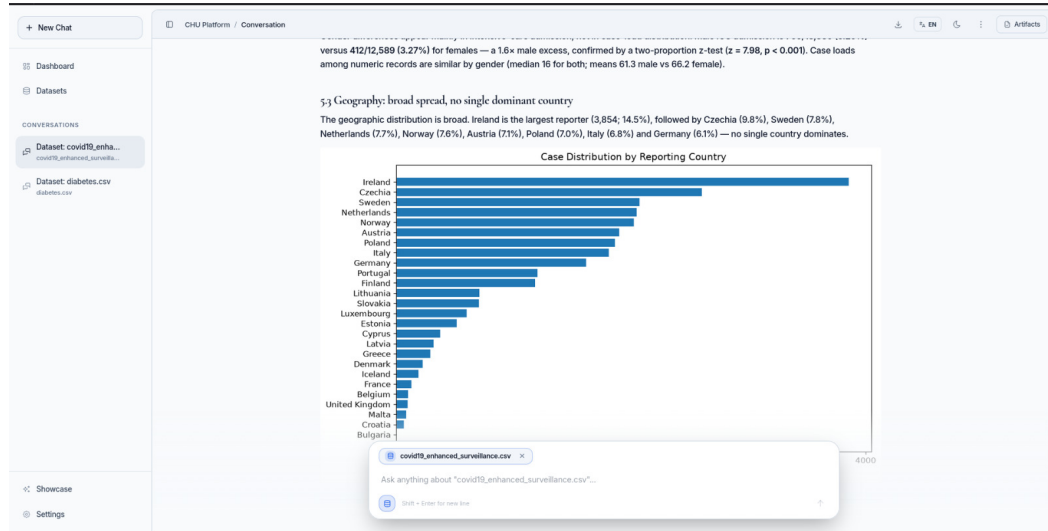


FIG. A.16 : Génération de graphiques (thème clair) : l’agent présente une visualisation (répartition des cas par pays) accompagnée de son interprétation directement dans la conversation.

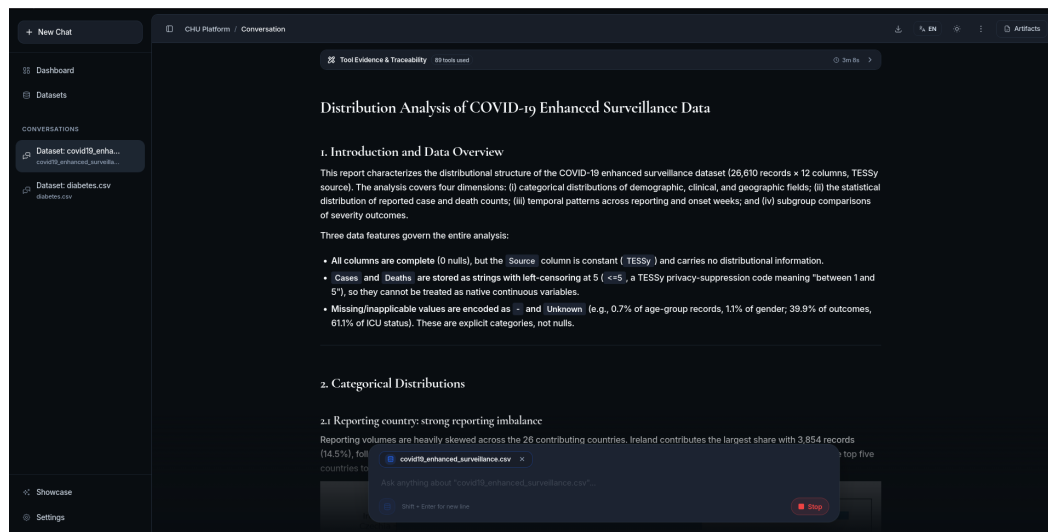


FIG. A.17 : Résultat de l’agent : réponse finale présentée à l’utilisateur, fondée sur l’evidence accumulée au cours de l’analyse.

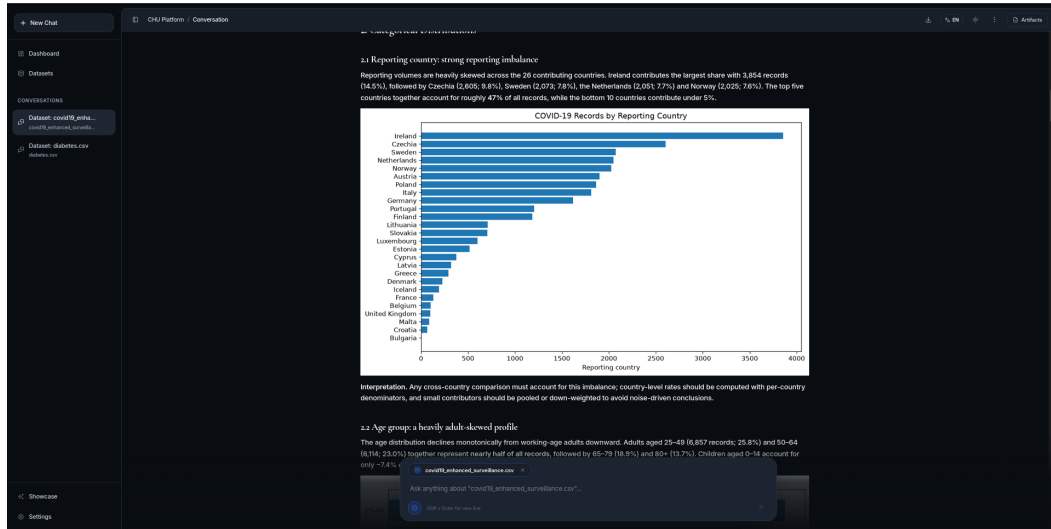


FIG. A.18 : Explication et interprétation : détail des conclusions de l’agent et de leur signification pour l’utilisateur.

A.1.4 Artefacts

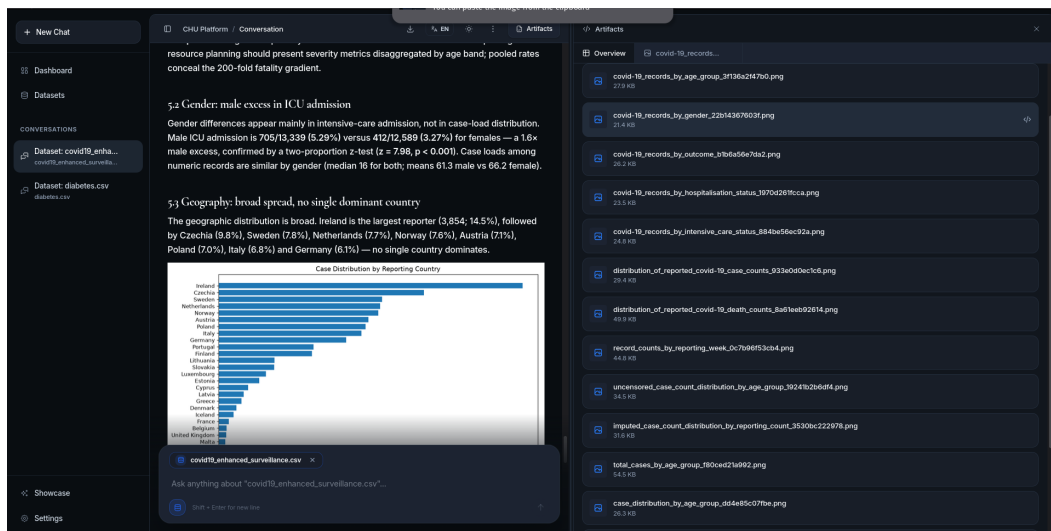


FIG. A.19 : Liste des artefacts : panneau présentant l’ensemble des artefacts produits par l’analyse (graphiques, exports, rapports).

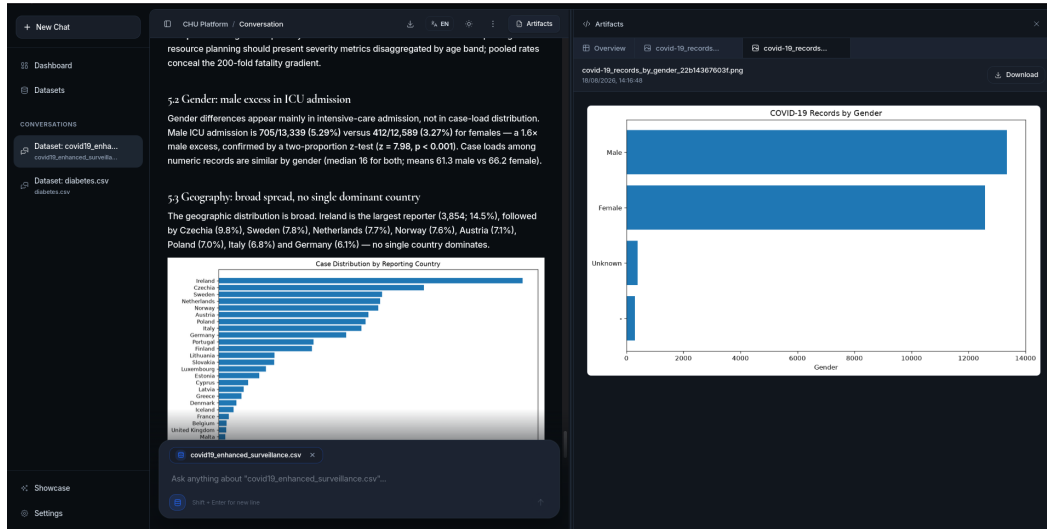


FIG. A.20 : Aperçu d'un artefact : consultation d'un graphique ou d'un rapport produit par l'agent, avec possibilité de le télécharger.

A.1.5 Paramètres et internationalisation

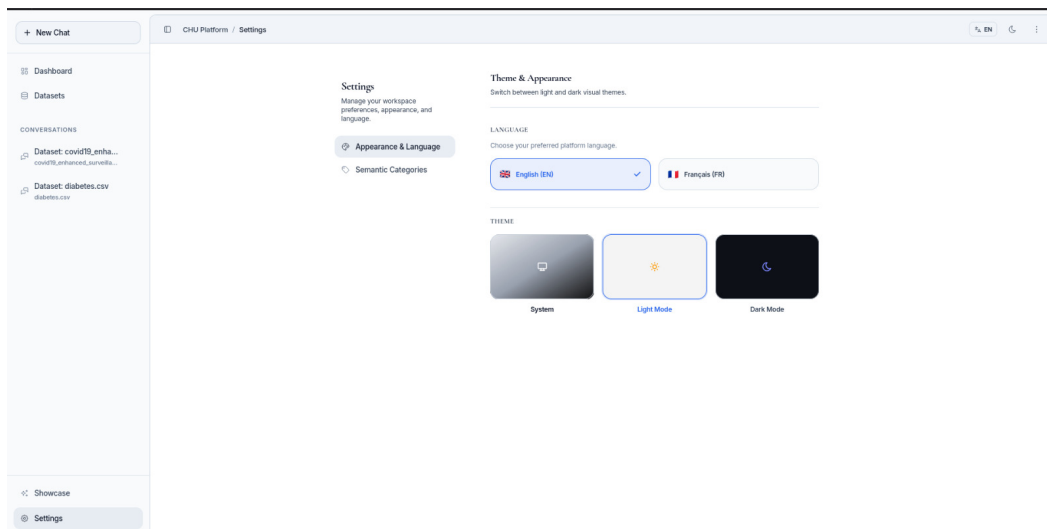


FIG. A.21 : Paramètres : gestion des préférences de l'espace de travail (langue et thème). Cette page permet de basculer l'interface entre le français et l'anglais.

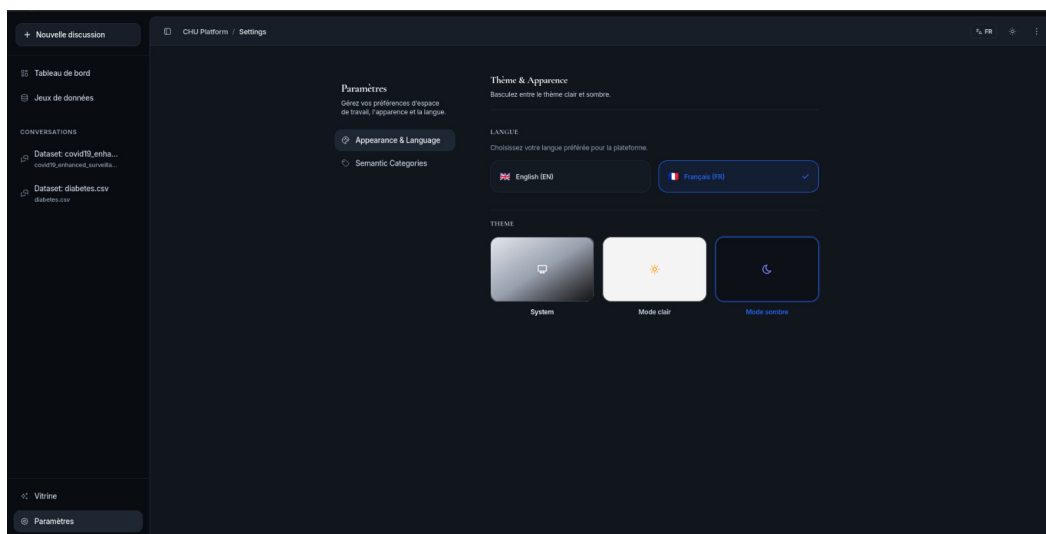


FIG. A.22 : Internationalisation : les mêmes paramètres affichés en français avec le thème sombre, illustrant la bascule de langue et d'apparence de l'interface.

A.1.6 Passerelle de modèles locaux

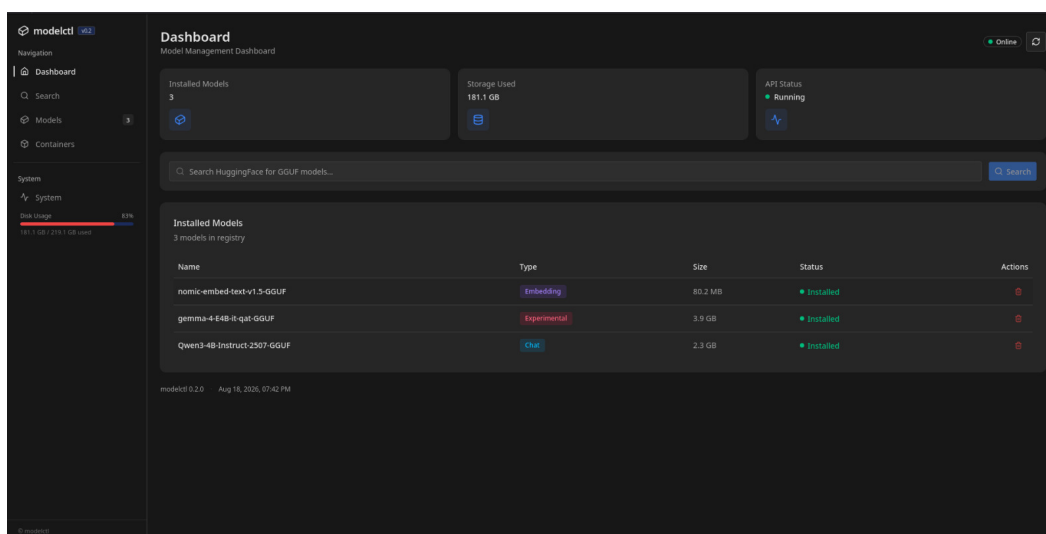


FIG. A.23 : Passerelle de modèles locaux (ModelCtl) : tableau de bord de gestion de l'infrastructure d'inférence — modèles GGUF installés, espace de stockage consommé et état de l'API.

A.2 Structure du projet

Listing A.1 : Organisation du monorepo


```
CHU-Platform/
|-- apps/
|   |-- api/          # API FastAPI (routers, services, modèles)
|   |-- ui/           # Mockup / maquette de l'interface utilisateur
|   |-- web/          # Application SvelteKit (interface)
|-- packages/
|   |-- agents/       # Orchestration agentique (LangGraph)
|   |-- analysis/     # Moteur d'analyse (statistiques, profilage)
|   |-- tools/        # Outils accessibles aux agents
|-- nginx/            # Reverse proxy
`-- docs/             # Documentation du projet
```

A.3 Projets liés

La plateforme s'appuie, pour le service des modèles de langage locaux, sur une passerelle d'inférence auto-hébergée. Le code source de ce projet est disponible publiquement :

<https://github.com/regisx001/llms-gateway>

Cette passerelle permet de rechercher, télécharger et servir des modèles GGUF via `llama.cpp`, en exposant une API compatible OpenAI utilisée par la plateforme en production.

Bibliographie

- [1] GROUPEMENT SANITAIRE TERRITORIAL TANGER-TÉTOUAN-AL HOCEÏMA. *Groupelement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma*. 2026. URL : <https://gst-tta.ma/>.
- [2] GROUPEMENT SANITAIRE TERRITORIAL TANGER-TÉTOUAN-AL HOCEÏMA. *Présentation du Groupelement Sanitaire Territorial Tanger-Tétouan-Al Hoceïma*. 2026. URL : <https://gst-tta.ma/>.
- [3] GROUPEMENT SANITAIRE TERRITORIAL TANGER-TÉTOUAN-AL HOCEÏMA. *Organigramme du GST-TTA*. 2026. URL : <https://gst-tta.ma/organigramme/>.
- [4] Ashish VASWANI et al. « Attention Is All You Need ». In : *Advances in Neural Information Processing Systems*. T. 30. 2017.
- [5] OPENAI. « GPT-4 Technical Report ». In : *arXiv preprint arXiv :2303.08774* (2023). URL : <https://arxiv.org/abs/2303.08774>.
- [6] Albert Q. JIANG et al. « Mistral 7B ». In : *arXiv preprint arXiv :2310.06825* (2023). URL : <https://arxiv.org/abs/2310.06825>.
- [7] AI@META. *Llama 3 Model Card*. <https://github.com/meta-llama/llama-models>. Documentation technique. 2024.
- [8] Patrick LEWIS et al. « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». In : *Advances in Neural Information Processing Systems*. T. 33. 2020.
- [9] Lei WANG et al. « A Survey on Large Language Model based Autonomous Agents ». In : *Frontiers of Computer Science* (2024).
- [10] Shunyu YAO et al. « ReAct : Synergizing Reasoning and Acting in Language Models ». In : *International Conference on Learning Representations* (2023). URL : <https://arxiv.org/abs/2210.03629>.
- [11] LANGCHAIN. *LangGraph Documentation*. 2026. URL : <https://docs.langchain.com/oss/python/langgraph/overview>.
- [12] MODEL CONTEXT PROTOCOL. *Model Context Protocol Specification*. 2026. URL : <https://modelcontextprotocol.io/specification/>.
- [13] Zui CHEN et al. « A Survey on Text-to-SQL Parsing : From Text to SQL ». In : *ACM Computing Surveys* (2022).
- [14] Wes MCKINNEY. « Data Structures for Statistical Computing in Python ». In : *Proceedings of the 9th Python in Science Conference*. 2010, p. 51-56.

- [15] Mark RAASVELDT et Hannes MÜHLEISEN. « DuckDB : an Embeddable Analytical Database ». In : *Proceedings of the 2019 International Conference on Management of Data* (2019).
- [16] FASTAPI. *FastAPI Documentation*. 2026. URL : <https://fastapi.tiangolo.com/>.
- [17] SVELTE. *SvelteKit Documentation*. 2026. URL : <https://svelte.dev/docs/kit>.
- [18] OPENAI. *ChatGPT Data Analysis*. 2026. URL : <https://help.openai.com/>.
- [19] MICROSOFT. *Copilot in Power BI*. 2026. URL : <https://learn.microsoft.com/power-bi/create-reports/copilot-introduction>.
- [20] SALESFORCE. *Tableau AI*. 2026. URL : <https://www.tableau.com/products/tableau-ai>.
- [21] ANTHROPIC. *Claude*. 2026. URL : <https://www.anthropic.com/claude>.
- [22] JULIUS AI. *Julius AI*. 2026. URL : <https://julius.ai/>.